



ELEC 5690 – Advanced Topics in Artificial Intelligence for Medical Image Analysis

Lecture 04: Training Strategies and Medical Images (Retinal Images)

Sep. 28, 2026



- Review what we have learned last time
- Data preprocessing
- Data augmentation
- Loss function
- Open-source tools for image analysis
- Advanced Medical Image Analysis Topic (Retinal Images)



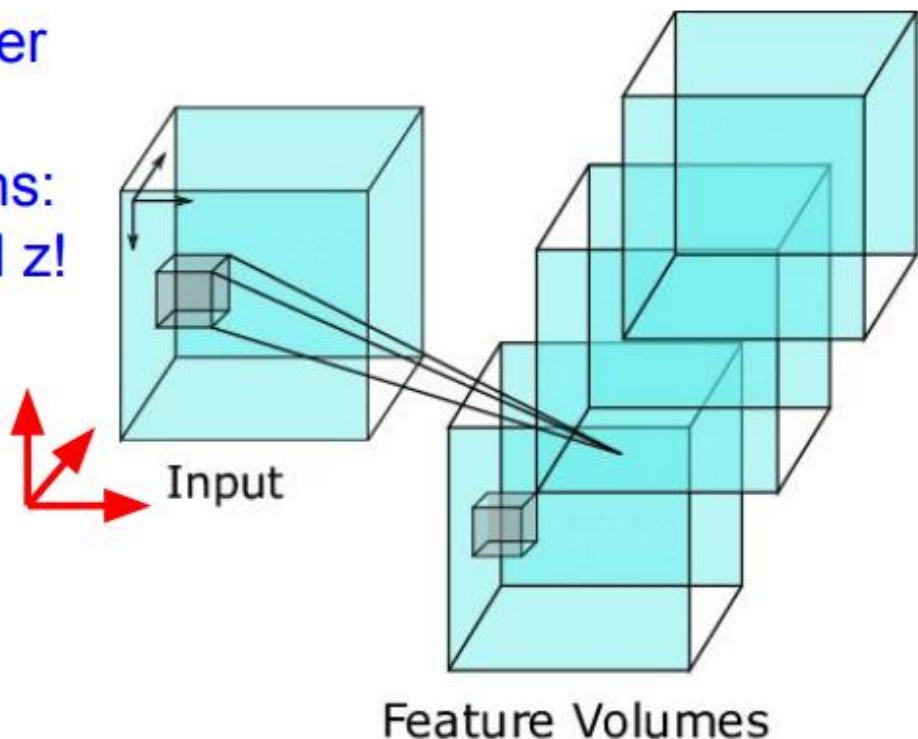
- Review what we have learned last time
- Data preprocessing
- Data augmentation
- Loss function
- Open-source tools for image analysis
- Advanced Medical Image Analysis Topic (Retinal Images)

1. Last Time



3D convolutions

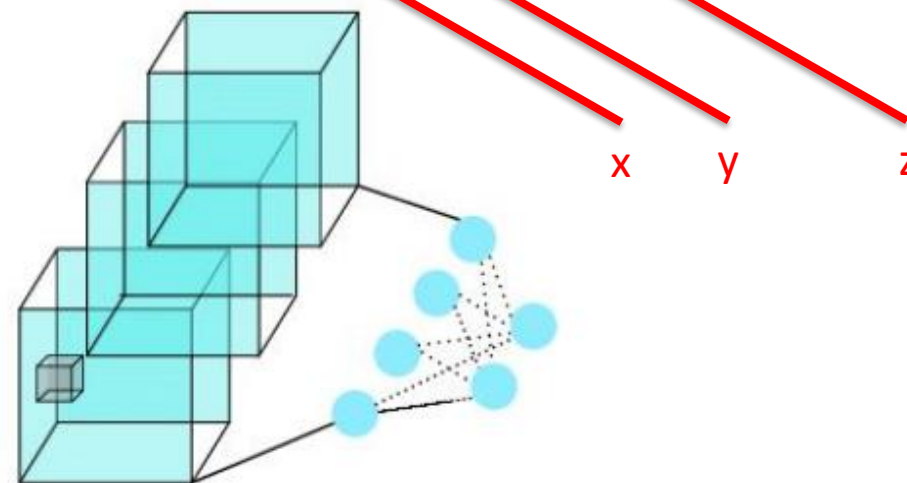
Slide filter
along 3
directions:
x, y, and z!



When might you use 3D convolutions?

Ex: $224 \times 224 \times 1 \times 256$ 3D CT scan (with 256 slices)

Ex: $224 \times 224 \times 3 \times 500$ video data (with 500 temporal frames)



x,y,z are spatial and/or channel temporal dimensions.

Filter (e.g. $5 \times 5 \times 3 \times 10$ filter) goes all the way through the “channels” dimension as before.

Figure credit:

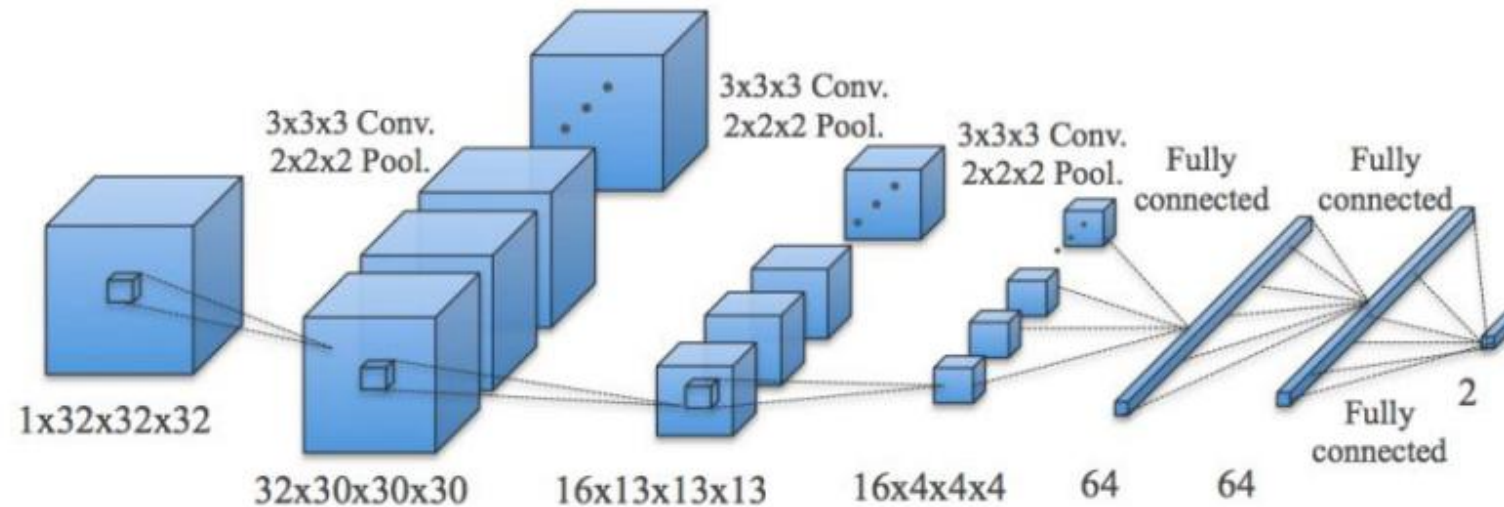
https://www.researchgate.net/profile/Deepak_Mishra19/publication/330912338/figure/fig1/AS:723363244810254@1549474645742/Basic-3D-CNN-architecture-the-3D-filter-is-convolved-with-the-video-in-three-dimensions.png

1. Last Time



Huang et al. 2017

- Simple 3D CNN for lung nodule classification
- Used image processing approaches to extract candidate nodules, then 3D CNN to classify the surrounding volume
- Used the Lung Image Database Consortium (LIDC) Dataset, with 99 3D CT scans



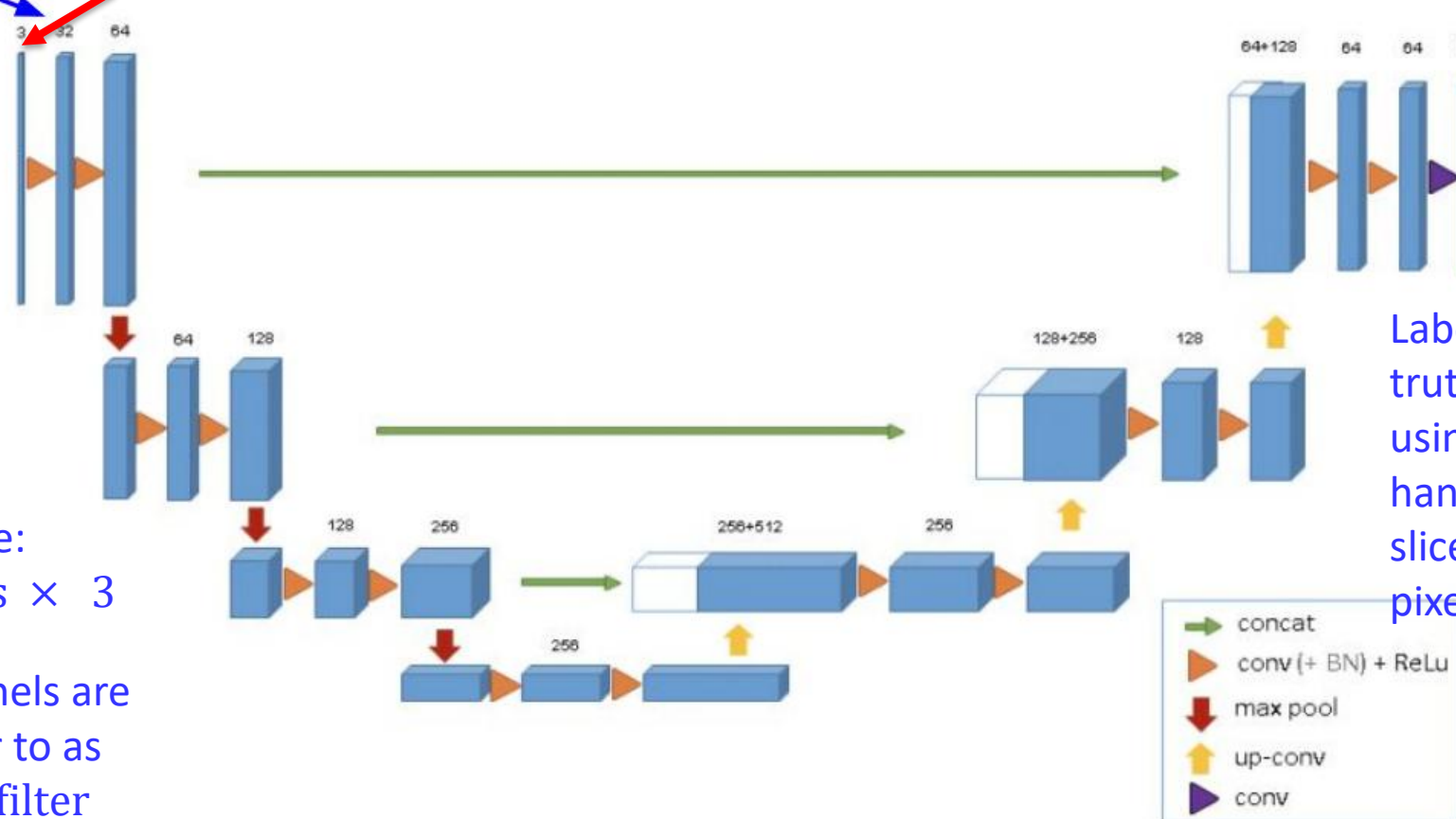
Huang et al. Lung Nodule Detection in CT Using 3D Convolutional Neural Networks. ISBI 2017.

1. Last Time



E.g. 3D U-Net

Channels Ex. input: 132 x 132 x 3 x 116



Labor-intensive to provide ground truth 3D annotation. Train instead using sparse annotations: a handful of annotated xy, xz, yz 2D slices. All others are “unlabeled” pixels with no weight in the loss.

3D conv filters are:
 $3 \times 3 \times \text{channels} \times 3$

Sometimes channels are implicit, can refer to as
 $3 \times 3 \times 3$ conv filter

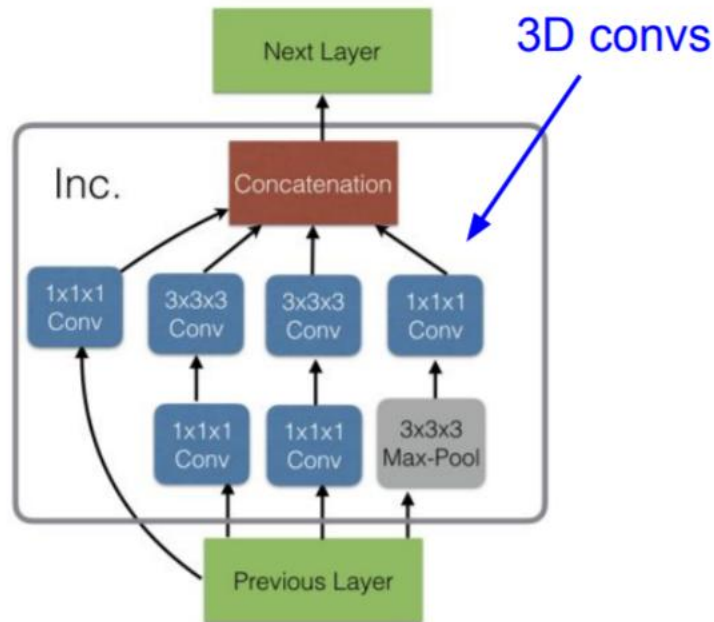
1. Last Time



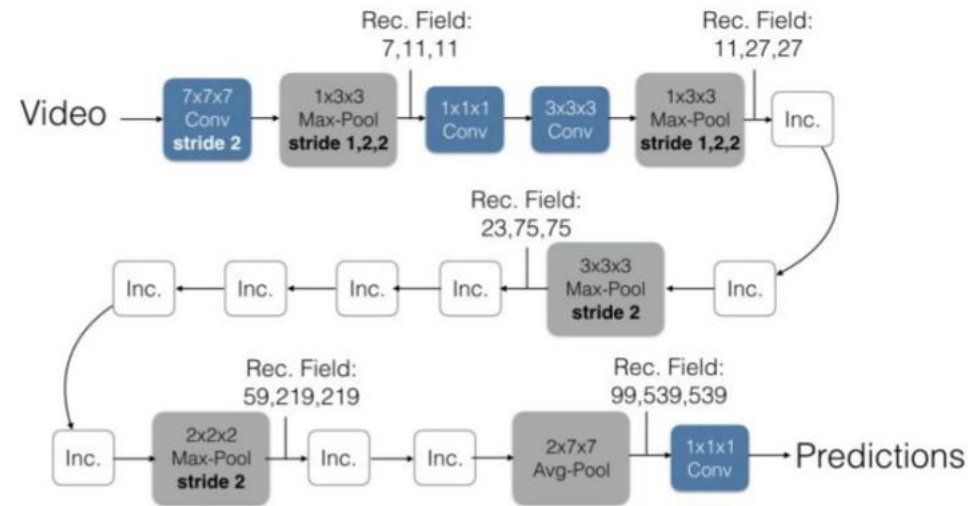
I3D: 3D convolutional network for video data

Note: in general,
can 3D-ify many
2D architectures!

Inception Module (Inc.) w/
3D convolutions



3D Inception Module used in Inception
Network (also known as GoogLeNet)



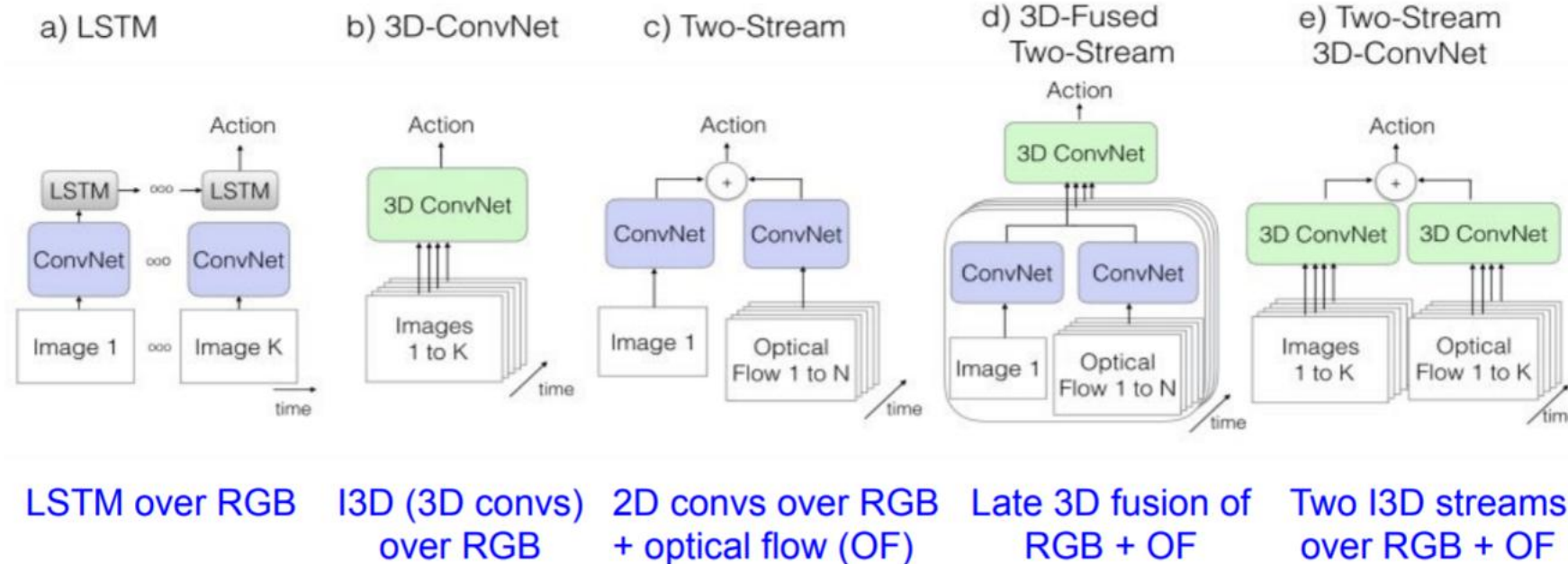
Can pre-train from 2D datasets e.g. ImageNet by replicating
and normalizing 2D weights over additional dimension!

Carreira and Zisserman. Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset. CVPR 2017.

1. Last Time



Video classifiers (including I3D) often enhanced with optical flow

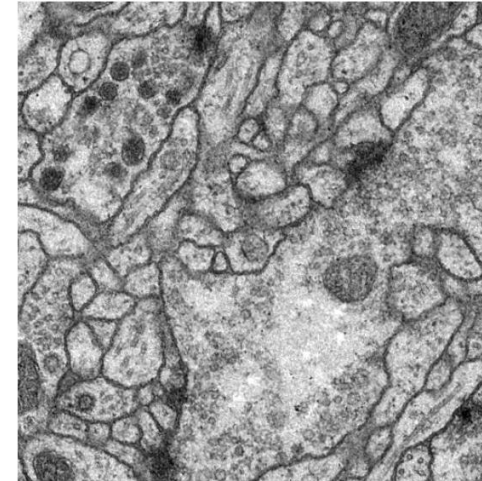
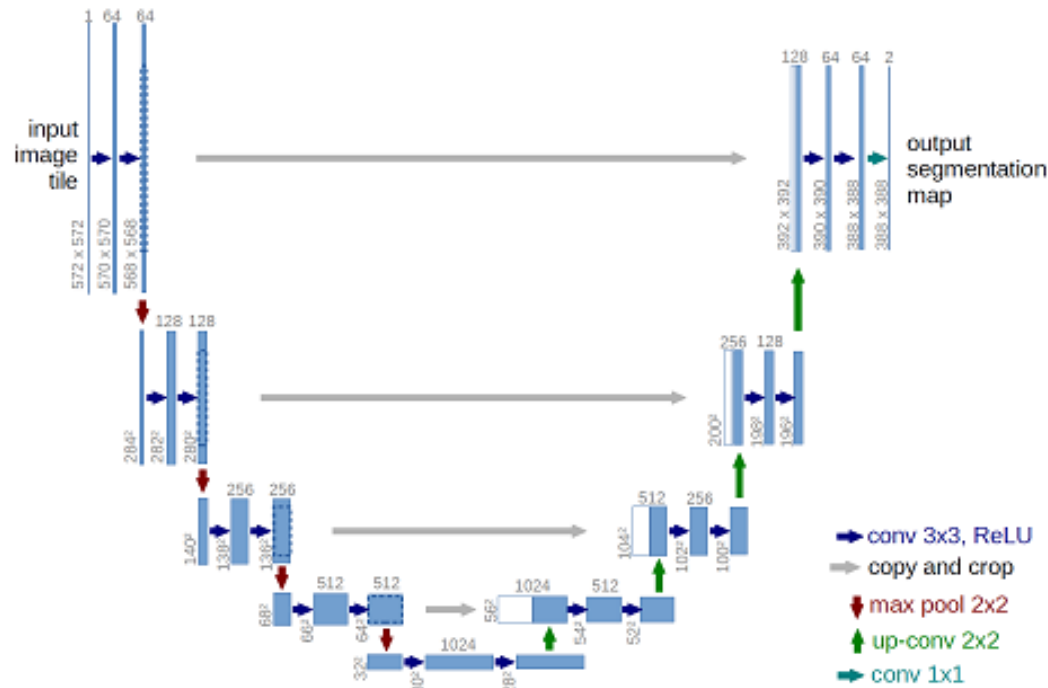


Carreira and Zisserman. Quo Vadis, Action Recognition? A New Model and the Kinetics Dataset. CVPR 2017.

1. Last Time



Implementation of deep learning framework -- Unet, using Keras



1. Last Time



Implementation of deep learning framework -- Unet, using Keras

master unet / main.py / <> Jump to

zhixuhao update ...

1 contributor

22 lines (17 sloc) | 821 Bytes

```
1 from model import *
2 from data import *
3
4 #os.environ["CUDA_VISIBLE_DEVICES"] = "0"
5
6
7 data_gen_args = dict(rotation_range=0.2,
8                       width_shift_range=0.05,
9                       height_shift_range=0.05,
10                      shear_range=0.05,
11                      zoom_range=0.05,
12                      horizontal_flip=True,
13                      fill_mode='nearest')
14 myGene = trainGenerator(2, 'data/membrane/train', 'image', 'label', data_gen_args, save_to_dir = None)
15
16 model = unet()
17 model_checkpoint = ModelCheckpoint('unet_membrane.hdf5', monitor='loss', verbose=1, save_best_only=True)
18 model.fit_generator(myGene, steps_per_epoch=300, epochs=1, callbacks=[model_checkpoint])
19
20 testGene = testGenerator("data/membrane/test")
21 results = model.predict_generator(testGene, 30, verbose=1)
22 saveResult("data/membrane/test", results)
```

Model init

Load pretrained model

Start training model

Data generator

Test data generator

Predict results

Save results

1. Last Time



Implementation of deep learning framework -- Unet, using Keras

```
1 import numpy as np
2 import os
3 import skimage.io as io
4 import skimage.transform as trans
5 import numpy as np
6 from keras.models import *
7 from keras.layers import *
8 from keras.optimizers import *
9 from keras.callbacks import ModelCheckpoint, LearningRateScheduler
10 from keras import backend as keras
11
12
13 def unet(pretrained_weights = None, input_size = (256,256,1)):
14     inputs = Input(input_size)
15     conv1 = Conv2D(64, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(inputs)
16     conv1 = Conv2D(64, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv1)
17     pool1 = MaxPooling2D(pool_size=(2, 2))(conv1)
18     conv2 = Conv2D(128, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(pool1)
19     conv2 = Conv2D(128, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv2)
20     pool2 = MaxPooling2D(pool_size=(2, 2))(conv2)
21     conv3 = Conv2D(256, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(pool2)
22     conv3 = Conv2D(256, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv3)
23     pool3 = MaxPooling2D(pool_size=(2, 2))(conv3)
24     conv4 = Conv2D(512, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(pool3)
25     conv4 = Conv2D(512, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv4)
26     drop4 = Dropout(0.5)(conv4)
27     pool4 = MaxPooling2D(pool_size=(2, 2))(drop4)
28
29     conv5 = Conv2D(1024, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(pool4)
30     conv5 = Conv2D(1024, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv5)
31     drop5 = Dropout(0.5)(conv5)
32
33     up6 = Conv2D(512, 2, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(UpSampling2D(size = (2,2))(drop5))
34     merge6 = concatenate([drop4, up6], axis = 3)
35     conv6 = Conv2D(512, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(merge6)
36     conv6 = Conv2D(512, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv6)
37
38     up7 = Conv2D(256, 2, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(UpSampling2D(size = (2,2))(conv6))
39     merge7 = concatenate([conv3, up7], axis = 3)
40     conv7 = Conv2D(256, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(merge7)
41     conv7 = Conv2D(256, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv7)
```

```
42
43     up8 = Conv2D(128, 2, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(UpSampling2D(size = (2,2))(conv7))
44     merge8 = concatenate([conv2, up8], axis = 3)
45     conv8 = Conv2D(128, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(merge8)
46     conv8 = Conv2D(128, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv8)
47
48     up9 = Conv2D(64, 2, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(UpSampling2D(size = (2,2))(conv8))
49     merge9 = concatenate([conv1, up9], axis = 3)
50     conv9 = Conv2D(64, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(merge9)
51     conv9 = Conv2D(64, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv9)
52     conv9 = Conv2D(2, 3, activation = 'relu', padding = 'same', kernel_initializer = 'he_normal')(conv9)
53     conv10 = Conv2D(1, 1, activation = 'sigmoid')(conv9)
54
55     model = Model(input = inputs, output = conv10)
56
57     model.compile(optimizer = Adam(lr = 1e-4), loss = 'binary_crossentropy', metrics = ['accuracy'])
58
59     #model.summary()
60
61     if(pretrained_weights):
62         model.load_weights(pretrained_weights)
63
64     return model
65
```

1. Last Time



2D Unet

```
class UNet2D(nn.Module):
    def __init__(self, in_channels, out_channels, conv_depths=(64, 128, 256, 512, 1024)):
        assert len(conv_depths) > 2, 'conv_depths must have at least 3 members'

        super(UNet2D, self).__init__()

        # defining encoder layers
        encoder_layers = []
        encoder_layers.append(First2D(in_channels, conv_depths[0], conv_depths[0]))
        encoder_layers.extend([Encoder2D(conv_depths[i], conv_depths[i + 1], conv_depths[i + 1])
                                for i in range(len(conv_depths)-2)])

        # defining decoder layers
        decoder_layers = []
        decoder_layers.extend([Decoder2D(2 * conv_depths[i + 1], 2 * conv_depths[i], 2 * conv_depths[i], conv_depths[i])
                                for i in reversed(range(len(conv_depths)-2))])
        decoder_layers.append>Last2D(conv_depths[1], conv_depths[0], out_channels))

        # encoder, center and decoder layers
        self.encoder_layers = nn.Sequential(*encoder_layers)
        self.center = Center2D(conv_depths[-2], conv_depths[-1], conv_depths[-1], conv_depths[-2])
        self.decoder_layers = nn.Sequential(*decoder_layers)

    def forward(self, x, return_all=False):
        x_enc = [x]
        for enc_layer in self.encoder_layers:
            x_enc.append(enc_layer(x_enc[-1]))

        x_dec = [self.center(x_enc[-1])]
        for dec_layer_idx, dec_layer in enumerate(self.decoder_layers):
            x_opposite = x_enc[-1-dec_layer_idx]
            x_cat = torch.cat(
                [pad_to_shape(x_dec[-1], x_opposite.shape), x_opposite],
                dim=1
            )
            x_dec.append(dec_layer(x_cat))

        if not return_all:
            return x_dec[-1]
        else:
            return x_enc + x_dec
```

```
1 import torch.nn as nn
2 from torch.nn.modules.loss import _Loss
3
4
5 class SoftDiceLoss(_Loss):
6     def __init__(self, size_average=None, reduce=None, reduction='mean'):
7         super(SoftDiceLoss, self).__init__(size_average, reduce, reduction)
8
9     def forward(self, y_pred, y_gt):
10         numerator = torch.sum(y_pred*y_gt)
11         denominator = torch.sum(y_pred*y_pred + y_gt*y_gt)
12         return numerator/denominator
13
14
15 class First2D(nn.Module):
16     def __init__(self, in_channels, middle_channels, out_channels, dropout=False):
17         super(First2D, self).__init__()
18
19         layers = [
20             nn.Conv2d(in_channels, middle_channels, kernel_size=3, padding=1),
21             nn.BatchNorm2d(middle_channels),
22             nn.ReLU(inplace=True),
23             nn.Conv2d(middle_channels, out_channels, kernel_size=3, padding=1),
24             nn.BatchNorm2d(out_channels),
25             nn.ReLU(inplace=True)
26         ]
27
28         if dropout:
29             assert 0 <= dropout <= 1, 'dropout must be between 0 and 1'
30             layers.append(nn.Dropout2d(p=dropout))
31
32         self.first = nn.Sequential(*layers)
33
34     def forward(self, x):
35         return self.first(x)
36
37
```

1. Last Time



```
class UNet2D(nn.Module):
```

```
    def __init__(self, in_channels, out_channels, conv_depths=(64, 128, 256, 512, 1024)):
```

```
        assert len(conv_depths) > 2, 'conv_depths must have at least 3 members'
```

```
        super(UNet2D, self).__init__()
```

```
        # defining encoder layers
```

```
        encoder_layers = []
```

```
        encoder_layers.append(First2D(in_channels, conv_depths[0], conv_depths[0]))
```

```
        encoder_layers.extend([Encoder2D(conv_depths[i], conv_depths[i + 1], conv_depths[i + 1])  
                                for i in range(len(conv_depths)-2)])
```

```
        # defining decoder layers
```

```
        decoder_layers = []
```

```
        decoder_layers.extend([Decoder2D(2 * conv_depths[i + 1], 2 * conv_depths[i], 2 * conv_depths[i], conv_depths[i])  
                                for i in reversed(range(len(conv_depths)-2))])
```

```
        decoder_layers.append>Last2D(conv_depths[1], conv_depths[0], out_channels))
```

```
        # encoder, center and decoder layers
```

```
        self.encoder_layers = nn.Sequential(*encoder_layers)
```

```
        self.center = Center2D(conv_depths[-2], conv_depths[-1], conv_depths[-1], conv_depths[-2])
```

```
        self.decoder_layers = nn.Sequential(*decoder_layers)
```

```
    def forward(self, x, return_all=False):
```

```
        x_enc = [x]
```

```
        for enc_layer in self.encoder_layers:
```

```
            x_enc.append(enc_layer(x_enc[-1]))
```

```
        x_dec = [self.center(x_enc[-1])] 
```

```
        for dec_layer_idx, dec_layer in enumerate(self.decoder_layers):
```

```
            x_opposite = x_enc[-1-dec_layer_idx]
```

```
            x_cat = torch.cat(
```

```
                [pad_to_shape(x_dec[-1], x_opposite.shape), x_opposite],  
                dim=1
```

```
            )
```

```
            x_dec.append(dec_layer(x_cat))
```

```
        if not return_all:
```

```
            return x_dec[-1]
```

```
        else:
```

```
            return x_enc + x_dec
```

2D Unet

```
38 class Encoder2D(nn.Module):
```

```
39     def __init__(
```

```
40         self, in_channels, middle_channels, out_channels,
```

```
41         dropout=False, downsample_kernel=2
```

```
42     ):
```

```
43         super(Encoder2D, self).__init__()
```

```
44         layers = [
```

```
45             nn.MaxPool2d(kernel_size=downsample_kernel),
```

```
46             nn.Conv2d(in_channels, middle_channels, kernel_size=3, padding=1),
```

```
47             nn.BatchNorm2d(middle_channels),
```

```
48             nn.ReLU(inplace=True),
```

```
49             nn.Conv2d(middle_channels, out_channels, kernel_size=3, padding=1),
```

```
50             nn.BatchNorm2d(out_channels),
```

```
51             nn.ReLU(inplace=True)
```

```
52         ]
```

```
53         if dropout:
```

```
54             assert 0 <= dropout <= 1, 'dropout must be between 0 and 1'
```

```
55             layers.append(nn.Dropout2d(p=dropout))
```

```
56         self.encoder = nn.Sequential(*layers)
```

```
57     def forward(self, x):
```

```
58         return self.encoder(x)
```

```
61 class Center2D(nn.Module):
```

```
62     def __init__(self, in_channels, middle_channels, out_channels, deconv_channels, dropout=False):
```

```
63         super(Center2D, self).__init__()
```

```
64         layers = [
```

```
65             nn.MaxPool2d(kernel_size=2),
```

```
66             nn.Conv2d(in_channels, middle_channels, kernel_size=3, padding=1),
```

```
67             nn.BatchNorm2d(middle_channels),
```

```
68             nn.ReLU(inplace=True),
```

```
69             nn.Conv2d(middle_channels, out_channels, kernel_size=3, padding=1),
```

```
70             nn.BatchNorm2d(out_channels),
```

```
71             nn.ReLU(inplace=True),
```

```
72             nn.ConvTranspose2d(out_channels, deconv_channels, kernel_size=2, stride=2)
```

```
73         ]
```

```
74         if dropout:
```

```
75             assert 0 <= dropout <= 1, 'dropout must be between 0 and 1'
```

```
76             layers.append(nn.Dropout2d(p=dropout))
```

```
77         self.center = nn.Sequential(*layers)
```

```
78     def forward(self, x):
```

```
79         return self.center(x)
```

1. Last Time



2D Unet

```
class UNet2D(nn.Module):
    def __init__(self, in_channels, out_channels, conv_depths=(64, 128, 256, 512, 1024)):
        assert len(conv_depths) > 2, 'conv_depths must have at least 3 members'

        super(UNet2D, self).__init__()

        # defining encoder layers
        encoder_layers = []
        encoder_layers.append(First2D(in_channels, conv_depths[0], conv_depths[0]))
        encoder_layers.extend([Encoder2D(conv_depths[i], conv_depths[i + 1], conv_depths[i + 1])
                               for i in range(len(conv_depths)-2)])

        # defining decoder layers
        decoder_layers = []
        decoder_layers.extend([Decoder2D(2 * conv_depths[i + 1], 2 * conv_depths[i], 2 * conv_depths[i], conv_depths[i])
                               for i in reversed(range(len(conv_depths)-2))])
        decoder_layers.append>Last2D(conv_depths[1], conv_depths[0], out_channels))

        # encoder, center and decoder layers
        self.encoder_layers = nn.Sequential(*encoder_layers)
        self.center = Center2D(conv_depths[-2], conv_depths[-1], conv_depths[-1], conv_depths[-2])
        self.decoder_layers = nn.Sequential(*decoder_layers)

    def forward(self, x, return_all=False):
        x_enc = [x]
        for enc_layer in self.encoder_layers:
            x_enc.append(enc_layer(x_enc[-1]))

        x_dec = [self.center(x_enc[-1])]
        for dec_layer_idx, dec_layer in enumerate(self.decoder_layers):
            x_opposite = x_enc[-1-dec_layer_idx]
            x_cat = torch.cat(
                [pad_to_shape(x_dec[-1], x_opposite.shape), x_opposite],
                dim=1
            )
            x_dec.append(dec_layer(x_cat))

        if not return_all:
            return x_dec[-1]
        else:
            return x_enc + x_dec
```

```
90 class Decoder2D(nn.Module):
91     def __init__(self, in_channels, middle_channels, out_channels, deconv_channels, dropout=False):
92         super(Decoder2D, self).__init__()
93
94         layers = [
95             nn.Conv2d(in_channels, middle_channels, kernel_size=3, padding=1),
96             nn.BatchNorm2d(middle_channels),
97             nn.ReLU(inplace=True),
98             nn.Conv2d(middle_channels, out_channels, kernel_size=3, padding=1),
99             nn.BatchNorm2d(out_channels),
100             nn.ReLU(inplace=True),
101             nn.ConvTranspose2d(out_channels, deconv_channels, kernel_size=2, stride=2)
102         ]
103
104         if dropout:
105             assert 0 <= dropout <= 1, 'dropout must be between 0 and 1'
106             layers.append(nn.Dropout2d(p=dropout))
107
108         self.decoder = nn.Sequential(*layers)
109
110     def forward(self, x):
111         return self.decoder(x)
112
113
114 class Last2D(nn.Module):
115     def __init__(self, in_channels, middle_channels, out_channels, softmax=False):
116         super>Last2D, self).__init__()
117
118         layers = [
119             nn.Conv2d(in_channels, middle_channels, kernel_size=3, padding=1),
120             nn.BatchNorm2d(middle_channels),
121             nn.ReLU(inplace=True),
122             nn.Conv2d(middle_channels, middle_channels, kernel_size=3, padding=1),
123             nn.BatchNorm2d(middle_channels),
124             nn.ReLU(inplace=True),
125             nn.Conv2d(middle_channels, out_channels, kernel_size=1),
126             nn.Softmax(dim=1)
127         ]
128
129         self.first = nn.Sequential(*layers)
130
131     def forward(self, x):
132         return self.first(x)
133
```

1. Last Time



**Showcase of 2D and 3D Unet
implemented in Pytorch**

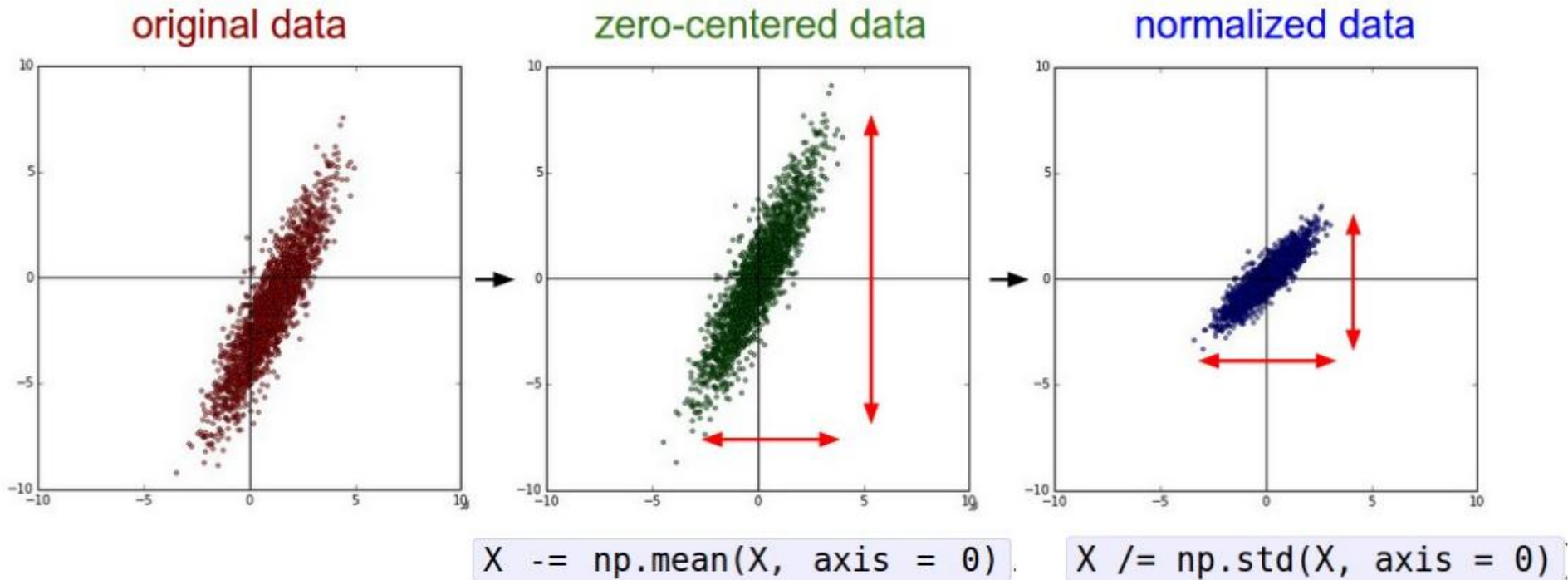


- Review what we have learned last time
- Data preprocessing
- Data augmentation
- Loss function
- Advanced Medical Image Analysis Topic (Retinal Images)

2. Data Preprocessing



Data Preprocessing



(Assume X [NxD] is data matrix,
each example in a row)

2. Data Preprocessing

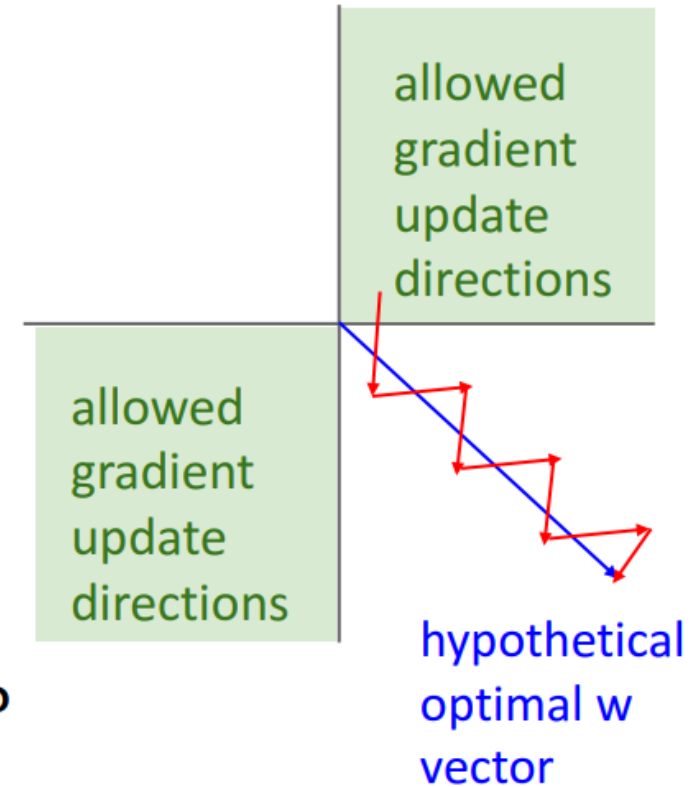


Remember: Consider what happens when the input to a neuron is always positive...

$$f\left(\sum_i w_i x_i + b\right)$$

What can we say about the gradients on $\mathbf{w}$?

Always all positive or all negative :(
(this is also why you want zero-mean data!)

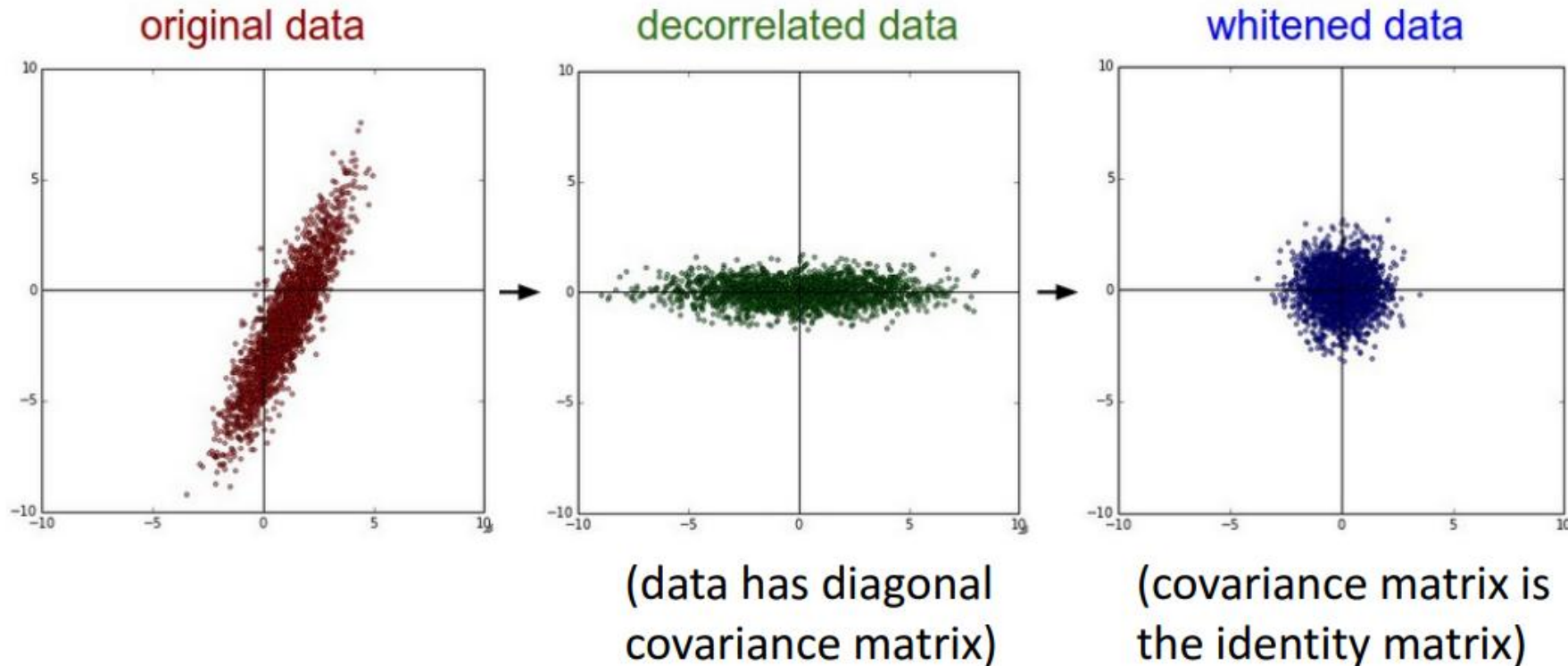


2. Data Preprocessing



Data Preprocessing

In practice, you may also see **PCA** and **Whitening** of the data

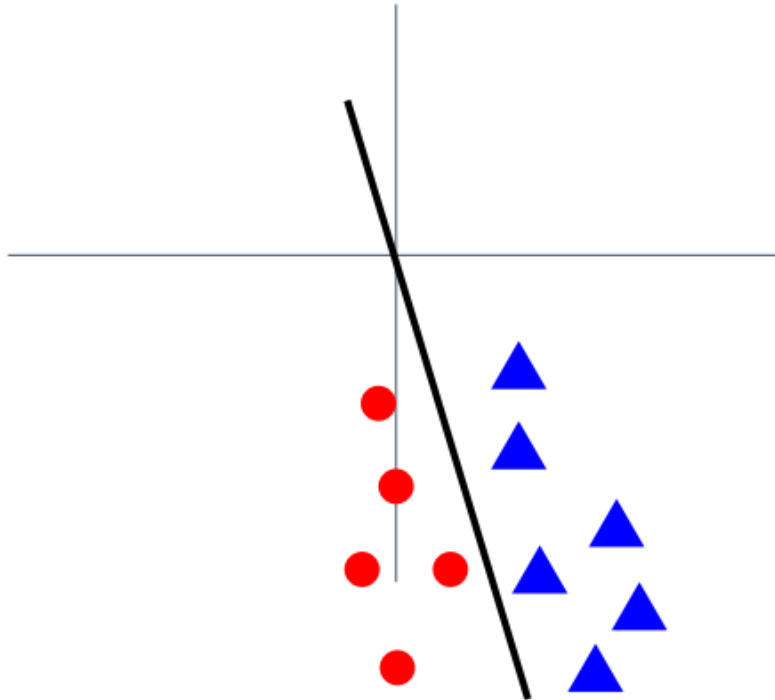


2. Data Preprocessing

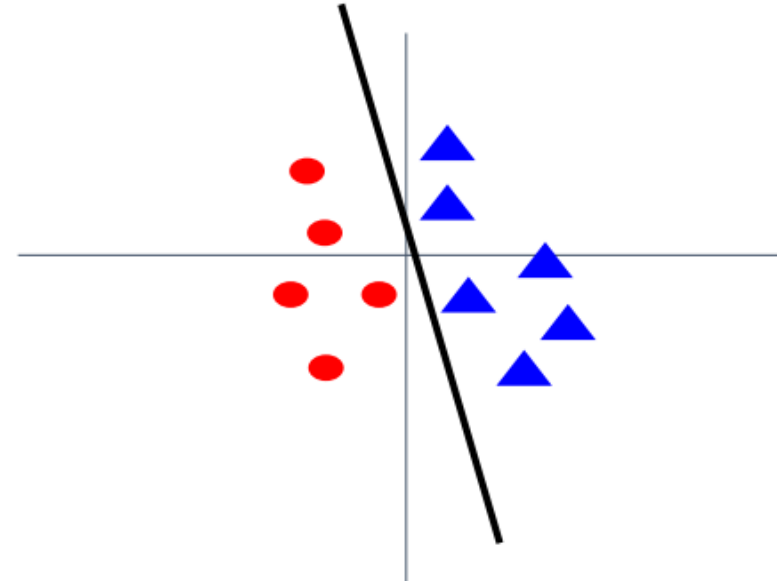


Data Preprocessing

Before normalization: classification loss very sensitive to changes in weight matrix; hard to optimize



After normalization: less sensitive to small changes in weights; easier to optimize



2. Data Preprocessing



Data Preprocessing for Images

e.g. consider CIFAR-10 example with [32,32,3] images

- Subtract the mean image (e.g. AlexNet)
(mean image = [32,32,3] array)
- Subtract per-channel mean (e.g. VGGNet)
(mean along each channel = 3 numbers)
- Subtract per-channel mean and
Divide by per-channel std (e.g. ResNet)
(mean along each channel = 3 numbers)

if you average over the entire dataset [n, 32, 32, 3], then you can get a mean image: [32, 32, 3]

Not common to do PCA or whitening

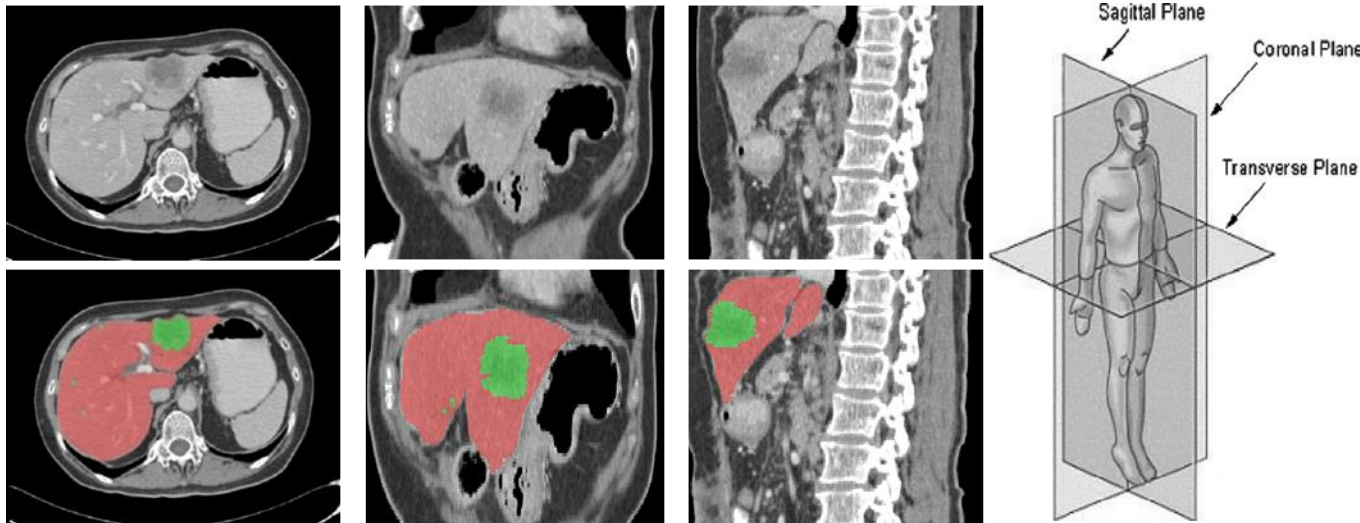
2. Data Preprocessing



Data preprocessing in medical images

Patching

Patching size is one of the most important factors affecting model performance, since medical images such as MRI and pathology images tend to be very large.



For example, in 3D-Unet, the training patch size is set to $96 * 96 * 96$ without overlap

The patching size is set to $96 * 96 * 96$ with 50% region overlap in the inference phase.

Liver and tumors shown in three orthogonal planes

Sizes: $512 * 512 * n$

2. Data Preprocessing



Data preprocessing in medical images

Oversam

To address problem of class imbalance between the positive and negative samples, including **random oversampling**, **synthetic minority oversampling**, **borderline SMOTE**, and **the adaptive synthetic sampling**.

One commonly used strategy

70% of the selected training samples are from the random image locations, while 30% of the image patches are guaranteed to hold at least one training foreground class.

2. Data Preprocessing



Data preprocessing in medical images

ReSam

To improve the representation ability of the used dataset via the machine learning model.

Consists of four steps: 1) spacing interpolation; 2) window transform; 3) acquisition of mask effective range and 4) generation of sub-images.

One commonly used strategy

The image pixels are directly interpolated and scaled, such that the actual distances represented by the pixels are the same.

2. Data Preprocessing



Data preprocessing in medical images

IntesNorm

Z-scores are a way to translate individual data points into terms of a standard deviation.

$$Z = (X - X_{\text{bar}}) / \sigma$$

- X : individual data point
- X_{bar} : the arithmetic mean
- σ : the standard deviation



- Review what we have learned last time
- Data preprocessing
- Data augmentation
- Loss function
- Advanced Medical Image Analysis Topic (Retinal Image)

3. Data Augmentation



Data Augmentation in medical images

Two categories of data augmentation

- Geometric transformation-based data augmentation (GTAug)
- Generative adversarial network (GAN)-based data augmentation (GANAug)

GTAug: to eliminate the effect of some geometric object variations in training images, e.g., positions, scales and viewing angles.

The prevalently used GTAug includes flipping, cropping, rotations, translating, color jittering, contrast, simulation of low resolution, Gaussian noise injection, mixing images, random erasing, Gaussian noise injection, mixing images, random erasing, Gaussian blue, mixup, and cutmix.

Commonly used strategies.

In 2D-Unet: random brightness contrast, random gamma, shift scale rotation horizontal and vertical flip.

In 3D-Unet: random noise and gamma adjust, random scale, rotation and mirroring.

3. Data Augmentation



Data Augmentation in medical images

Two categories of data augmentation

- Geometric transformation-based data augmentation (GTAug)
- Generative adversarial network (GAN)-based data augmentation (GANAug)

GANAug: guide the network to generate some real data that is close to the source domain



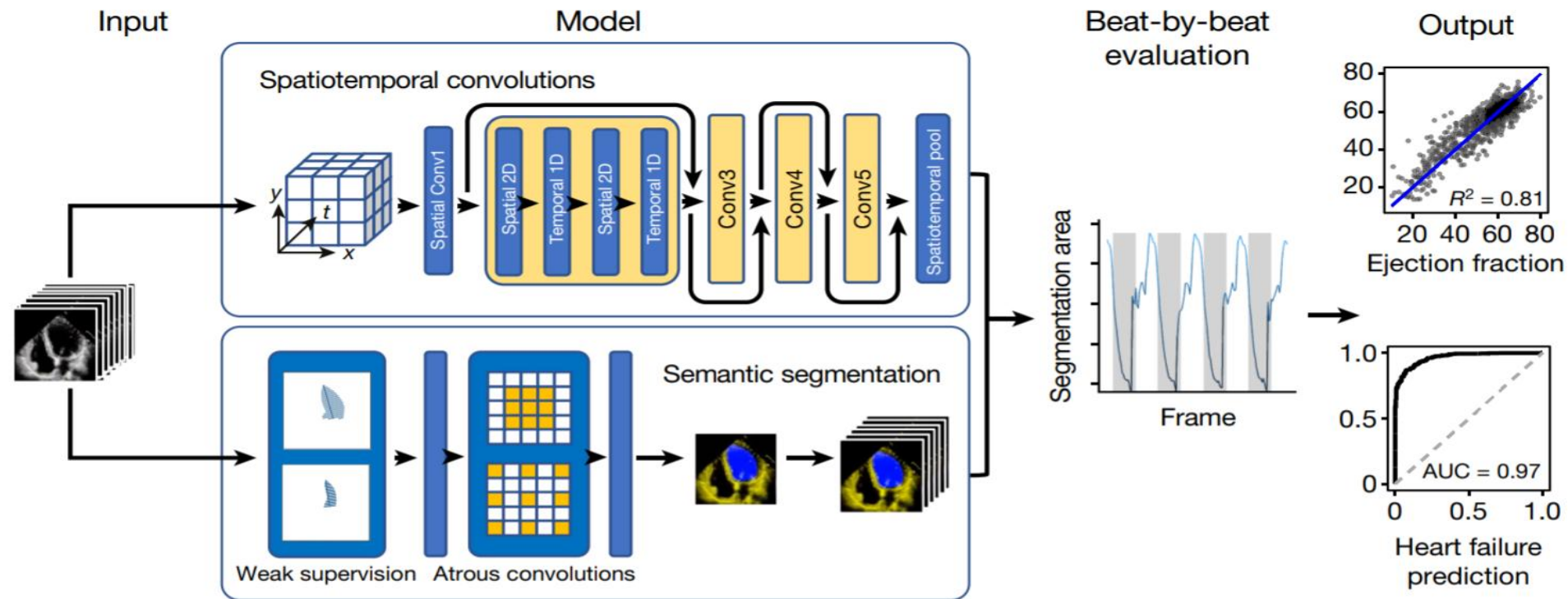
- Review what we have learned last time
- Data preprocessing
- Data augmentation
- Loss function
- Advanced Medical Image Analysis Topic (Retinal Image)

4. Loss Functions



Loss functions in medical images

Ejection fraction prediction from echocardiogram videos

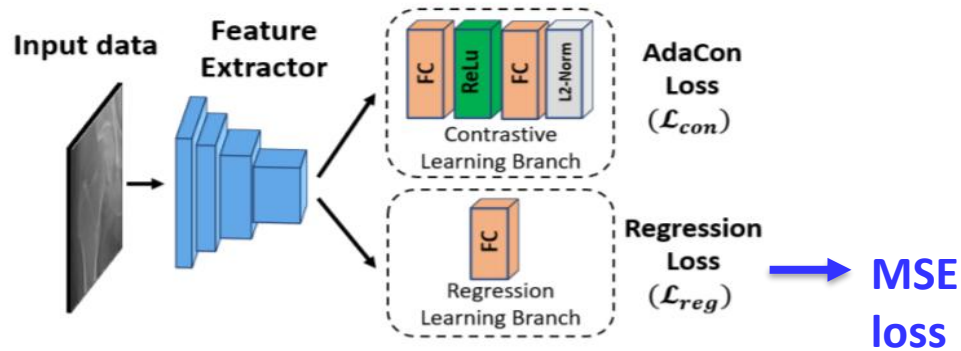


4. Loss Functions



Loss functions in medical images

Ejection fraction prediction from echocardiogram videos: how to train the network?



How to estimate ejection fraction from a video?

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

Can we optimize the network in feature space?

Contrastive loss for image classification

$$\mathcal{L}_{sup} = \sum_{i \in \mathcal{I}} \frac{-1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \log \frac{\exp(s \cos(\theta_{i,p}))}{\sum_{a \in \mathcal{I} \setminus i} \exp(s \cos(\theta_{i,a}))}, \quad (1)$$

where s denotes the temperature scaling factor, usually determined empirically, and $\cos(\theta_{i,j}) = z_i^T z_j$.

$\mathcal{P}(i) := \{j \in \mathcal{I} \mid y_j = y_i, j \neq i\}$ The set of indices for all positive samples for anchor sample i .

$\mathcal{Q}(i) := \{j \in \mathcal{I} \mid y_j \neq y_i\}$ The set of indices for all negative pair samples for anchor sample i .

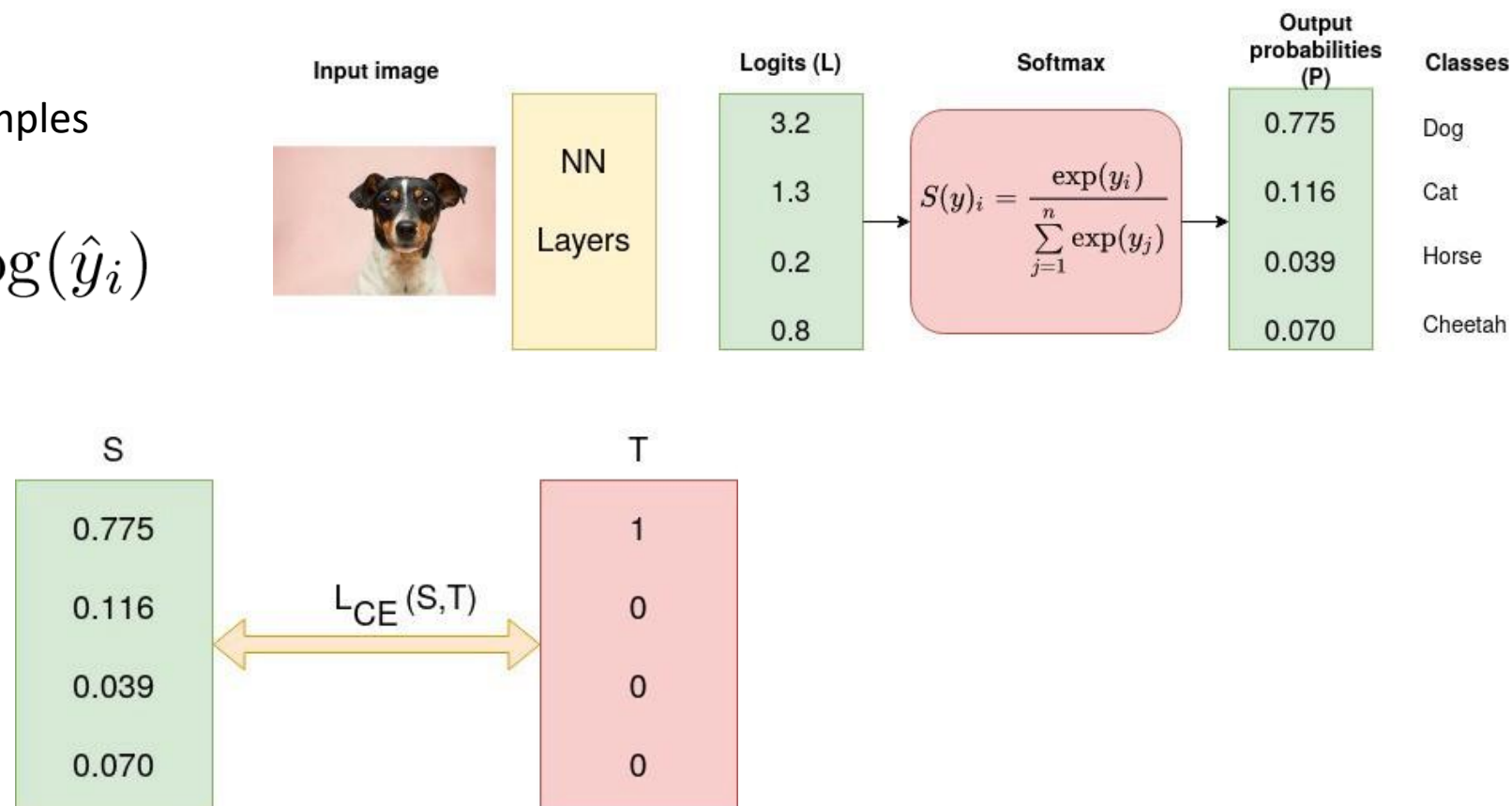
4. Loss Functions



Cross-entropy loss

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

m examples



$$L_{CE} = -(1 \times \log(0.775) + 0 \times \log(0.116) + 0 \times \log(0.039) + 0 \times \log(0.07)) = 0.255$$

4. Loss Functions



How to implement it?

```
from torch import nn
criterion = nn.CrossEntropyLoss()
input = torch.tensor([[3.2, 1.3, 0.2, 0.8]], dtype=torch.float)
target = torch.tensor([0], dtype=torch.long)
criterion(input, target)
Out[55]: tensor(0.2547)
```

It calculates SoftMax to input automatically.

[1, 0, 0, 0]

Apply softmax to input: [0.7751, 0.1159, 0.0386, 0.0703]

$$\mathcal{L} = -(1 \times \log(0.7751)) = 0.2547$$

$\log_e$

4. Loss Functions



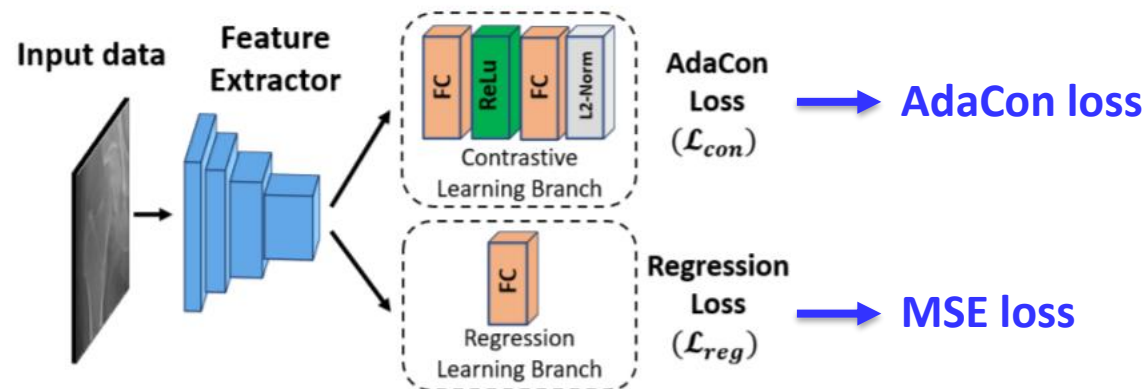
Compare cross-entropy and contrastive loss

$$L = -\frac{1}{m} \sum_{i=1}^m y_i \cdot \log(\hat{y}_i)$$

$$\mathcal{L}_{sup} = \sum_{i \in \mathcal{I}} \frac{-1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \log \frac{\exp(s \cos(\theta_{i,p}))}{\sum_{a \in \mathcal{I} \setminus i} \exp(s \cos(\theta_{i,a}))}, \quad (1)$$

where s denotes the temperature scaling factor, usually determined empirically, and $\cos(\theta_{i,j}) = z_i^T z_j$.

How to perform contrastive learning on regression?



4. Loss Functions



Classification

SupCon

A positive sample, p , is correctly classified if $z_i^T z_p$ is greater than $z_i^T z_{q_j}$ for negative samples q_1, q_2, q_3 . Decision boundaries for all negative samples are the same.

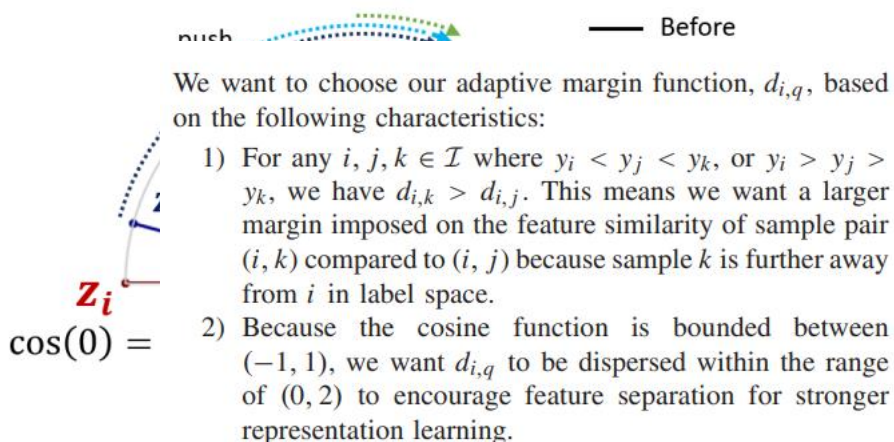


Fig. 1: (a) For Based on these characteristics, we define the adaptive margin $d_{i,q}$ as follows:

$$d_{i,q} = 2 \times |\phi(y_i) - \phi(y_q)|. \quad (4)$$

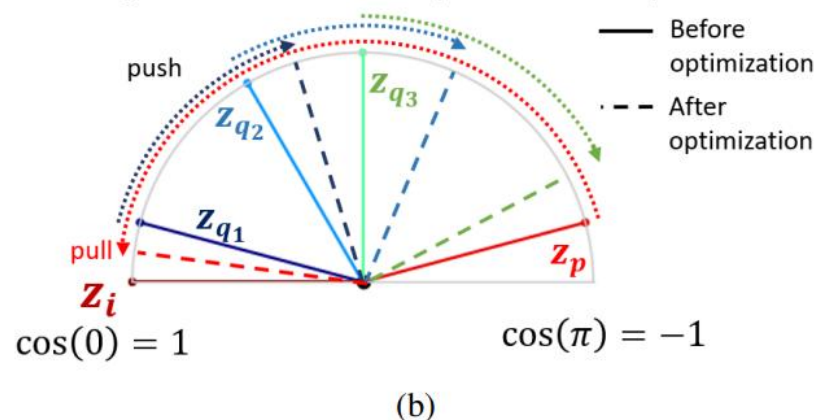
where ϕ is the empirical cumulative distribution function (ECDF) [68], defined by:

$$\phi(y_i) = \frac{1}{N} \sum_{j=1}^N \mathbb{1}_{y_j \leq y_i}. \quad (5)$$

Regression

AdaCon

A positive sample, p , is correctly identified if $z_i^T z_p$ is greater than $z_i^T z_{q_j} + d_{i,q_j}$ for negative samples q_1, q_2, q_3 . Samples with larger label differences are pushed further apart



SupCon [28], samples that form positive pairs are pulled closer together and further apart. The boundary condition for correct positive-negative pairs, where $p \in \mathcal{P}$ and $q_j \in \mathcal{Q}$; negative pairs are pushed apart. For regression conditions for classification, we enforce the model such that the positive sample p is further away from the negative samples q_j in the feature space compared to z_{q_1} if $y_p < y_{q_1} < y_{q_2}$ or $y_{q_2} > y_{q_1} > y_p$.

$$\mathcal{L}_{con} = \sum_{i \in \mathcal{I}} \frac{-1}{|\mathcal{P}(i)|} \sum_{p \in \mathcal{P}(i)} \log \frac{\exp(s \cos(\theta_{i,p}))}{\sum_{a \in \mathcal{I} \setminus i} \exp(s(\cos(\theta_{i,a}) + d_{i,a}))},$$

5. Open-source tools



Open-source tools for image analysis:

- ImageJ <http://rsbweb.nih.gov/ij>
- ITK-SNAP <http://itksnap.org>
- FSL <http://www.fmrib.ox.ac.uk/fsl>
- OsiriX <http://www.osirix-viewer.com>
- 3D Slicer <http://slicer.org>

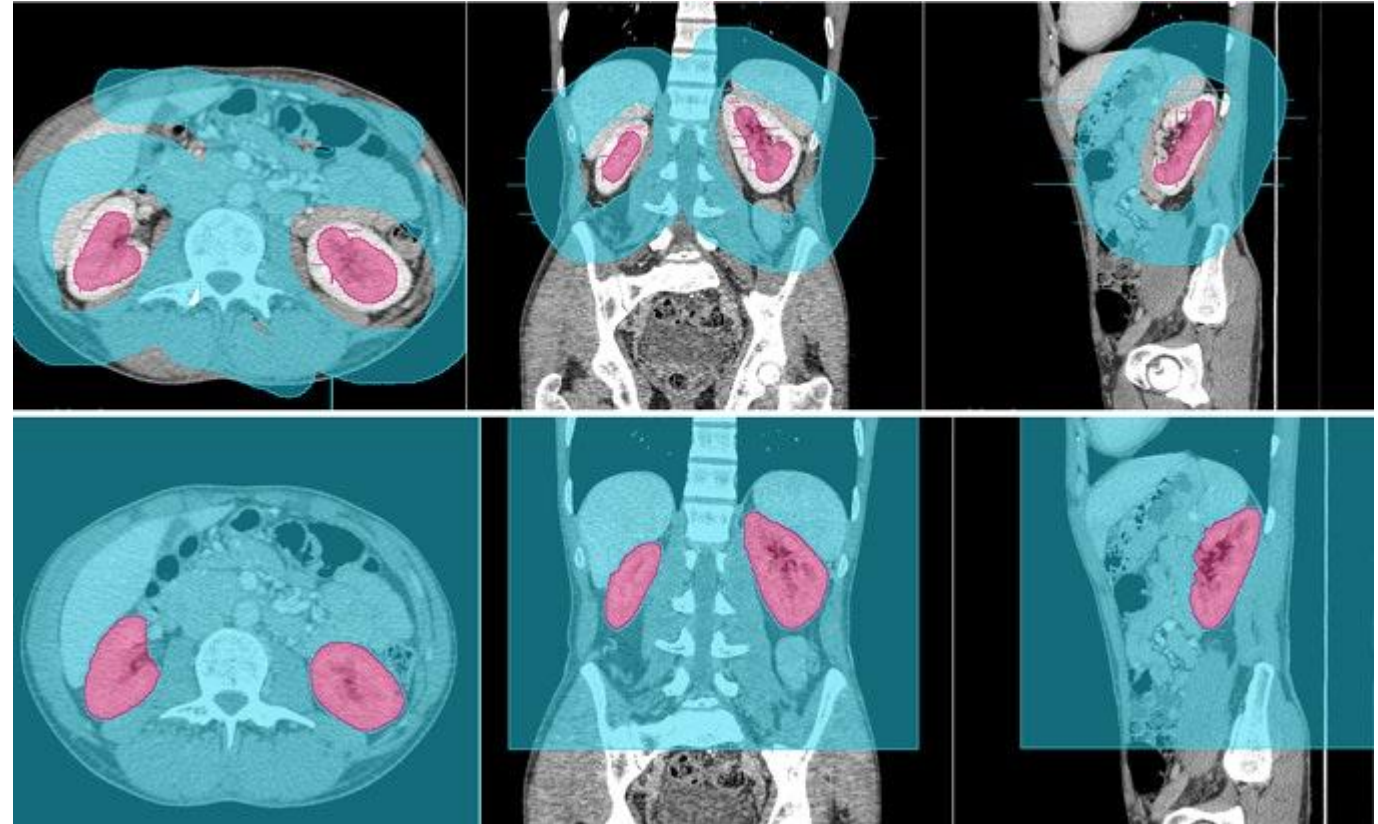
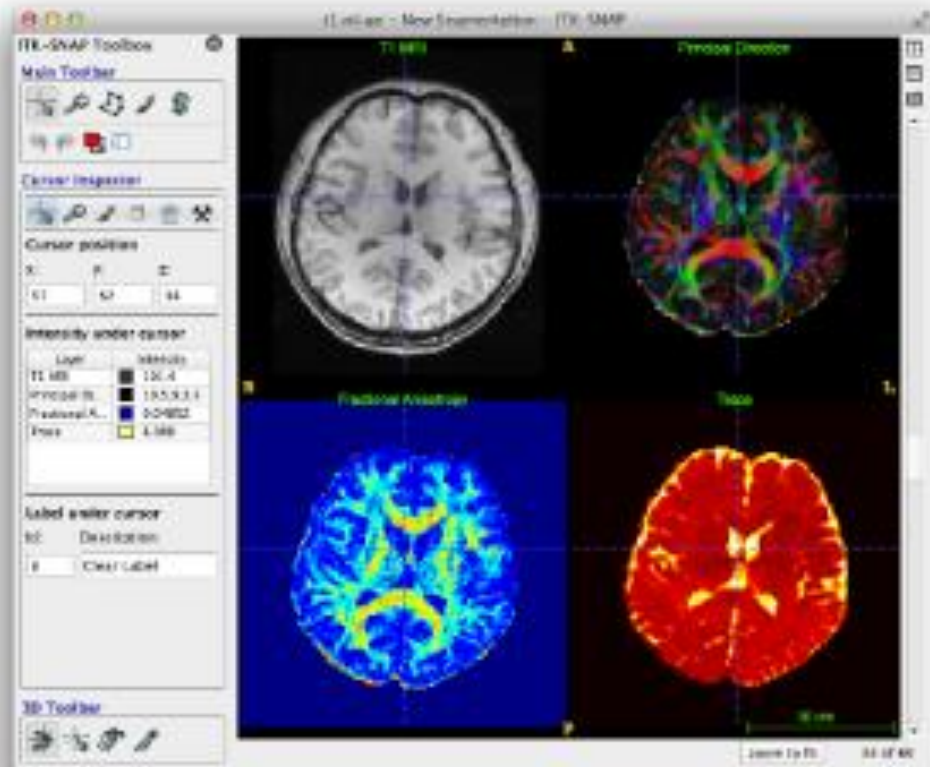
5. Open-source tools



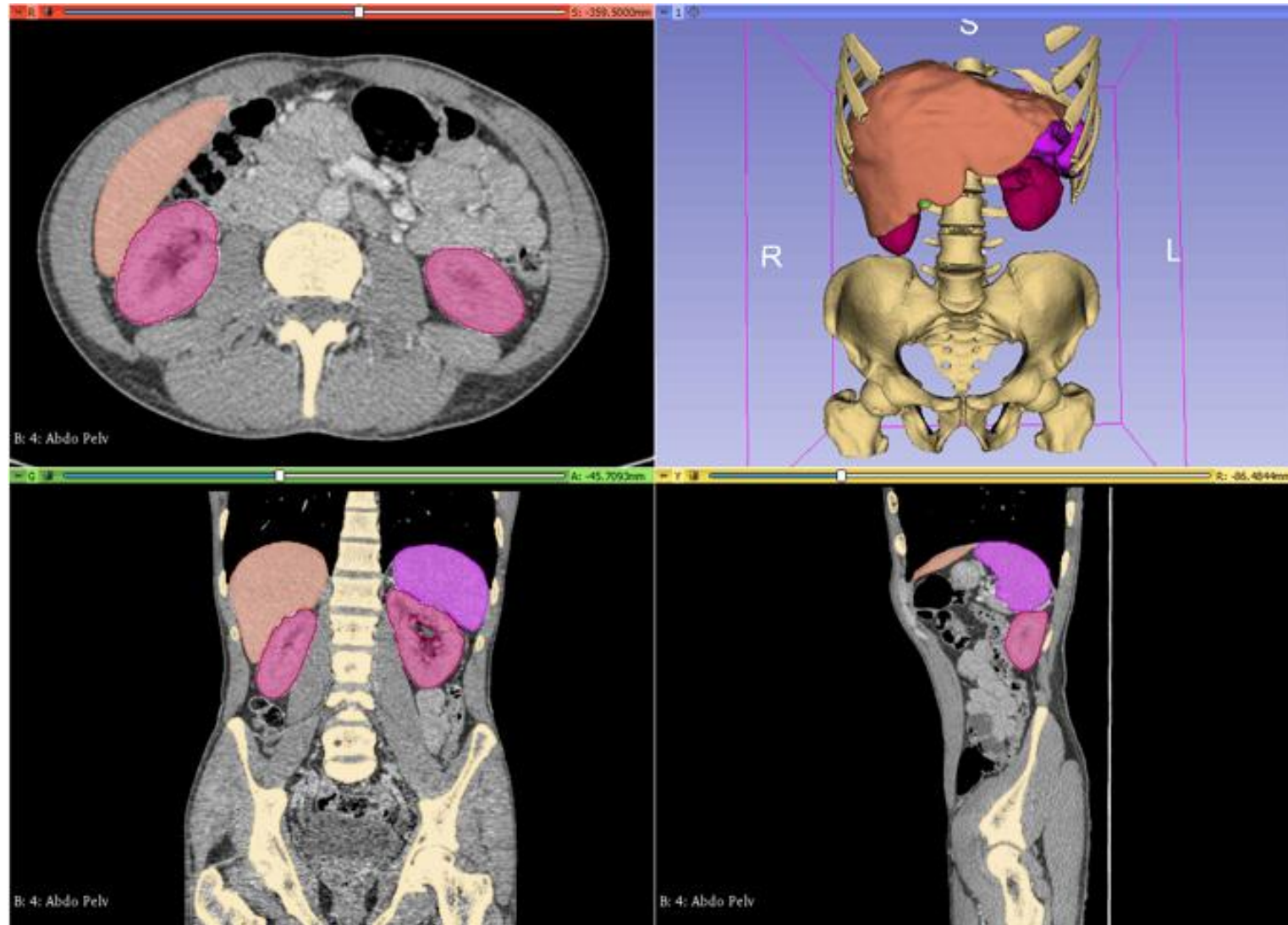
ITK-SNAP

- ITK-SNAP is a tool for navigation and segmentation of 3D medical imaging data
- ITK-SNAP implements the active contour automatic segmentation algorithms by Caselles et al (1997) and Zhu and Yuille (1996)
- It also provides a dynamic interface for navigation in 3D images
- Several tools for manual delineation are also provided. ITK-SNAP can open many 3D image file formats, including DICOM, NifTI and Analyze.

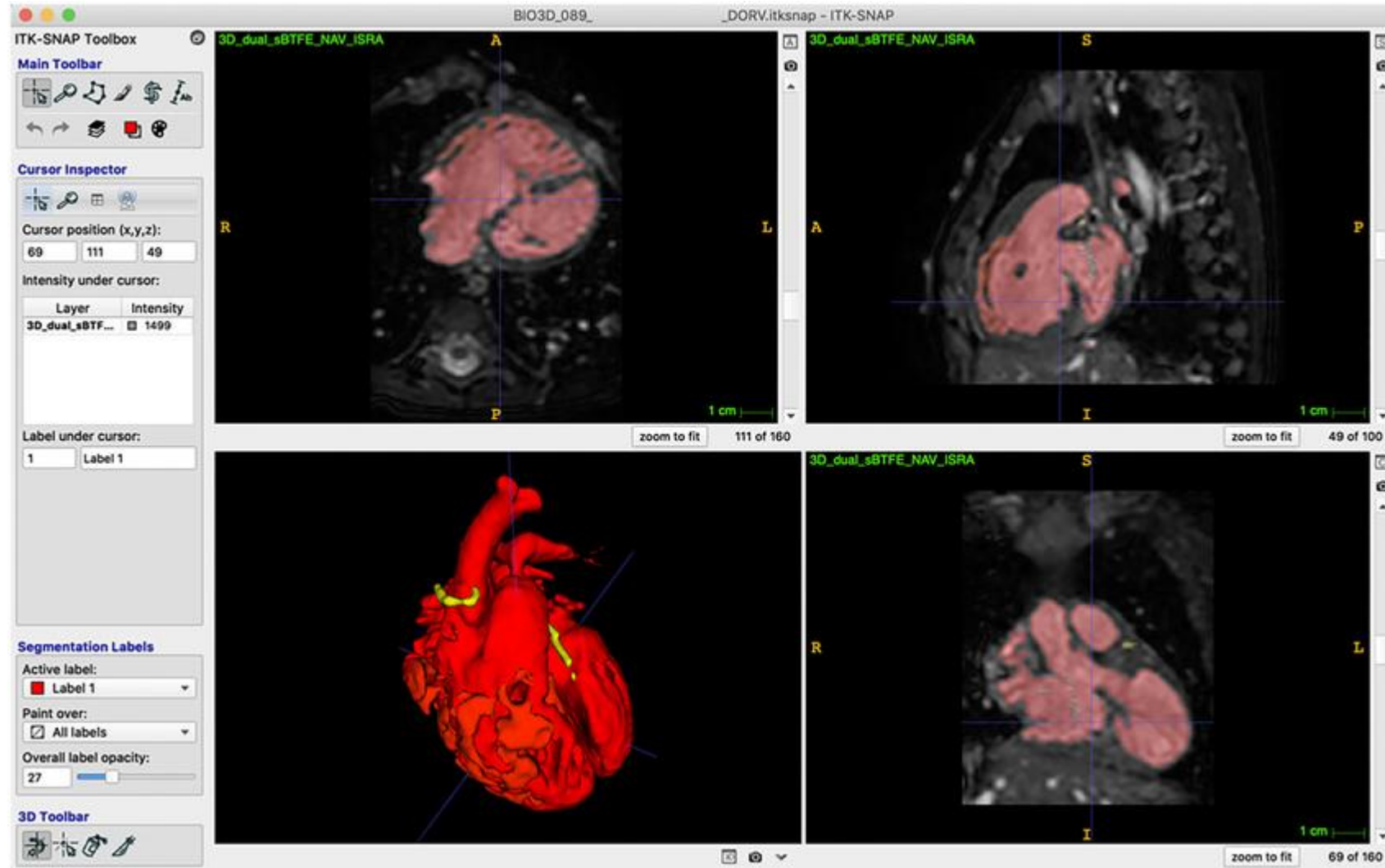
5. Open-source tools



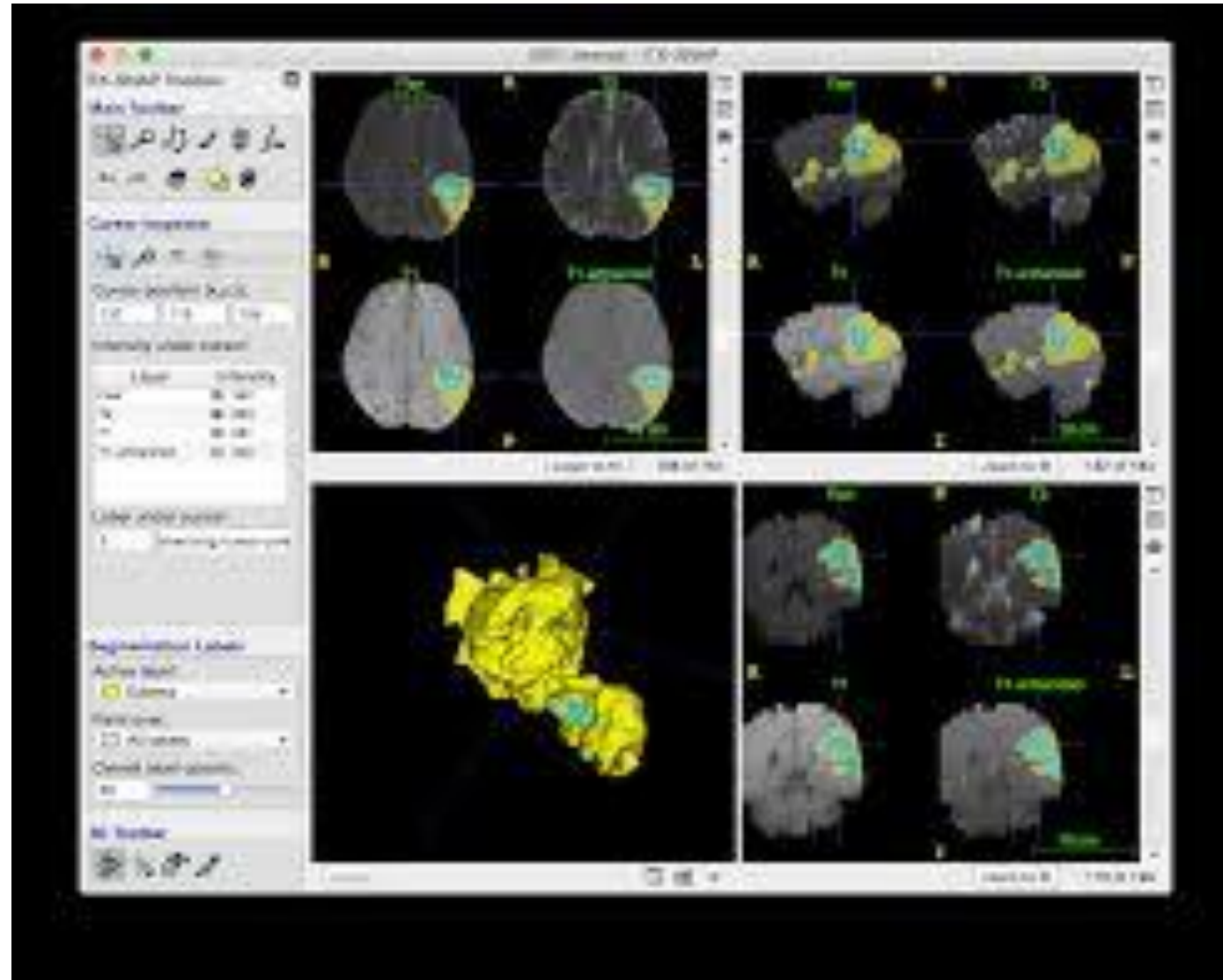
5. Open-source tools



5. Open-source tools



5. Open-source tools





- Review what we have learned last time
- Data preprocessing
- Data augmentation
- Loss function
- Advanced Medical Image Analysis Topic (Retinal Images)



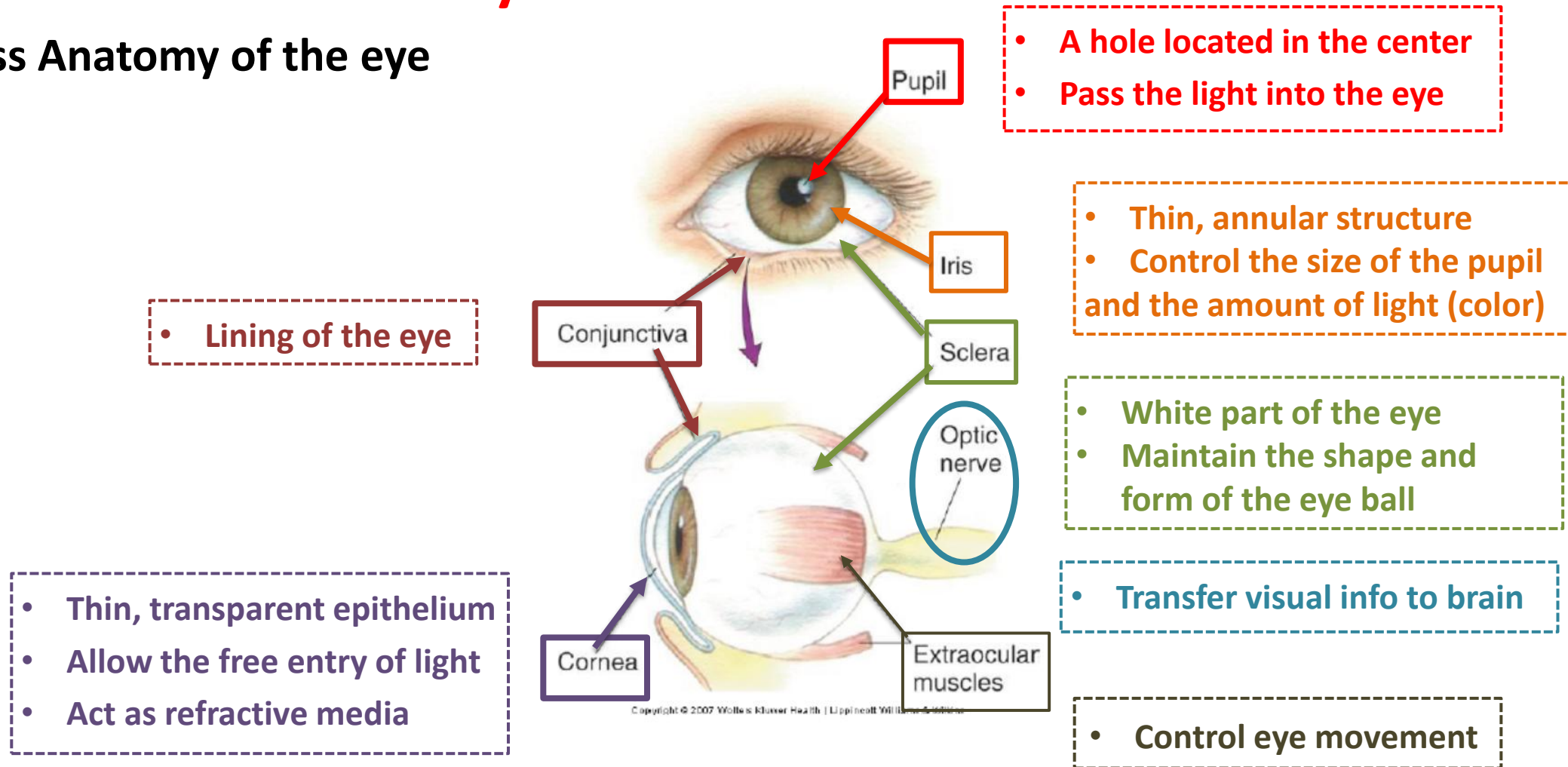
Next Topic: Introduction to eye physiology

2. Introduction to eye physiology



The structure of the eye

Gross Anatomy of the eye



2. Introduction to eye physiology

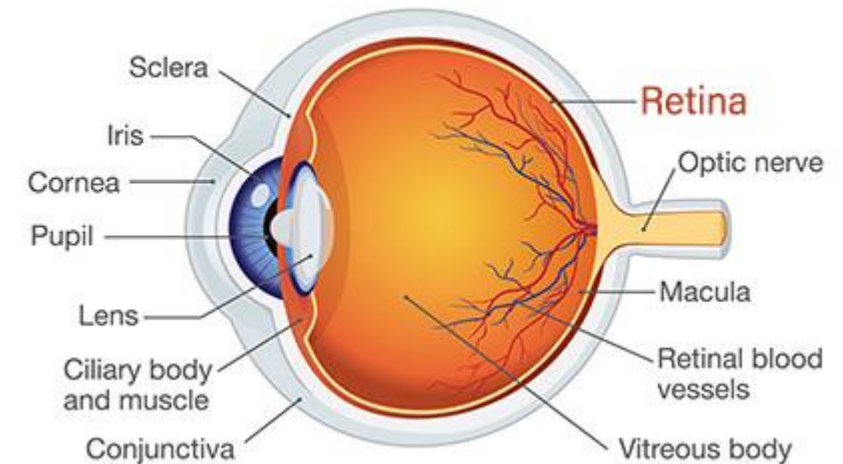


The structure of the eye

Retina

- very thin layer of tissue in the **inner** part of the eye
- capture light rays that enter the eye
- like **film's role** in camera
- light impulses are then sent to the **brain** via **optic nerve**

Human Eye Anatomy



2. Introduction to eye physiology



Physiology of the eye

Image formation

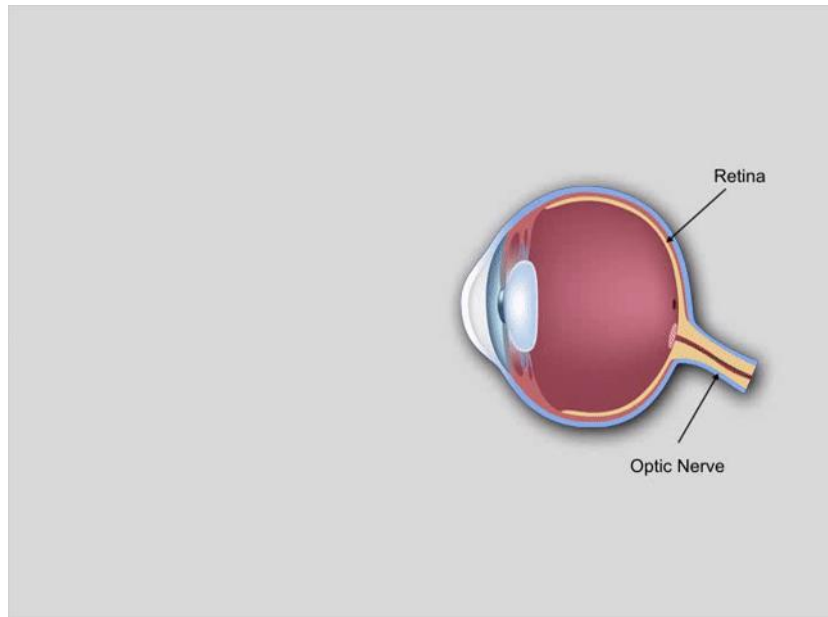


Image on the retina is inverted;
sent to brain by optic nerve

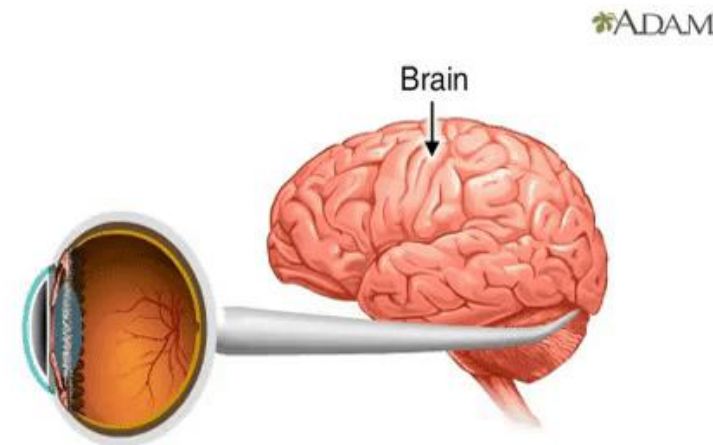


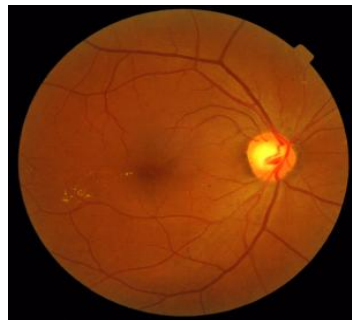
Image is reverted in the occipital
lobe in the visual processing center

2. Introduction to eye physiology



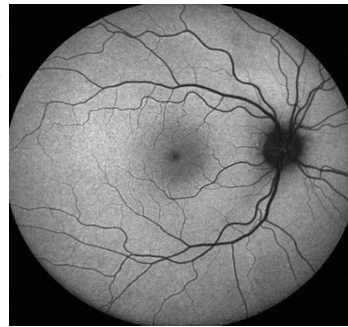
Diagnosis of eye diseases

Digital Retinal Photography



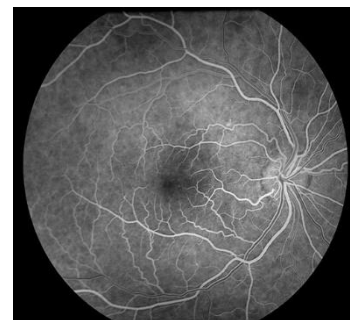
captures fine details of the retina

Fundus Autofluorescence Imaging



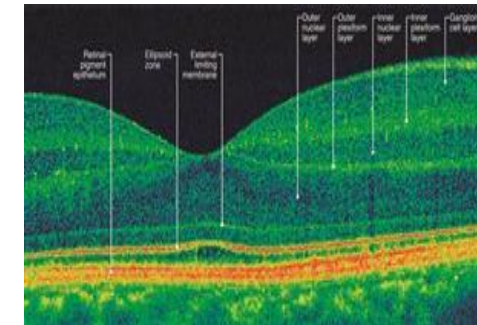
specialized filters to study certain layers

specialized filters to study the blood vessels



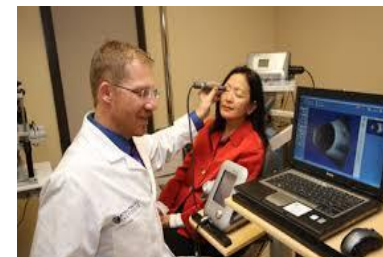
Fluorescein and ICG Angiography

Optical Coherence Tomography (OCT)



create cross-sectional images of the retina and surrounding tissues.

Ocular Ultrasound



assess the retina and other structures at the back of the eye

2. Introduction to eye physiology

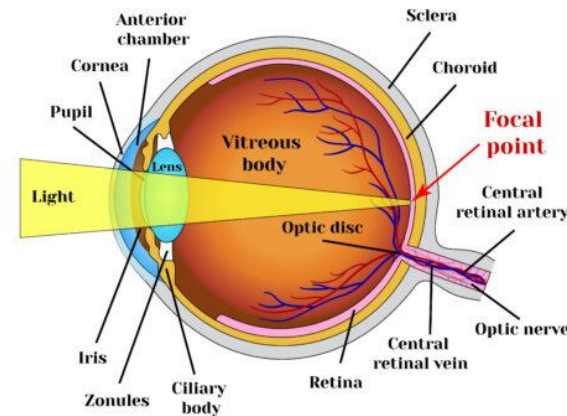


Common eye diseases

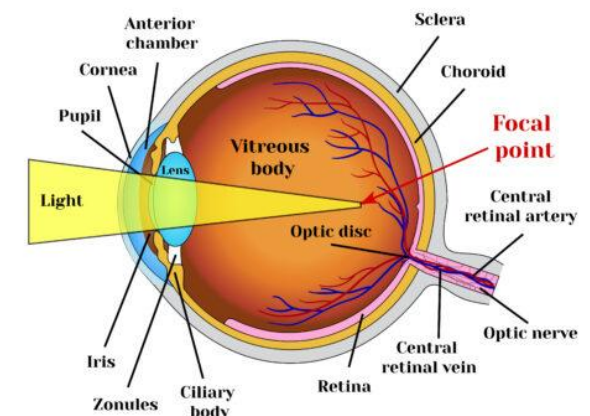
Myopia

- very common vision condition
- see objects near to you clearly
- objects farther away blurry

Normal vision



Myopia



light rays focus **at a point in front of the retina**, rather than directly on its **surface**.

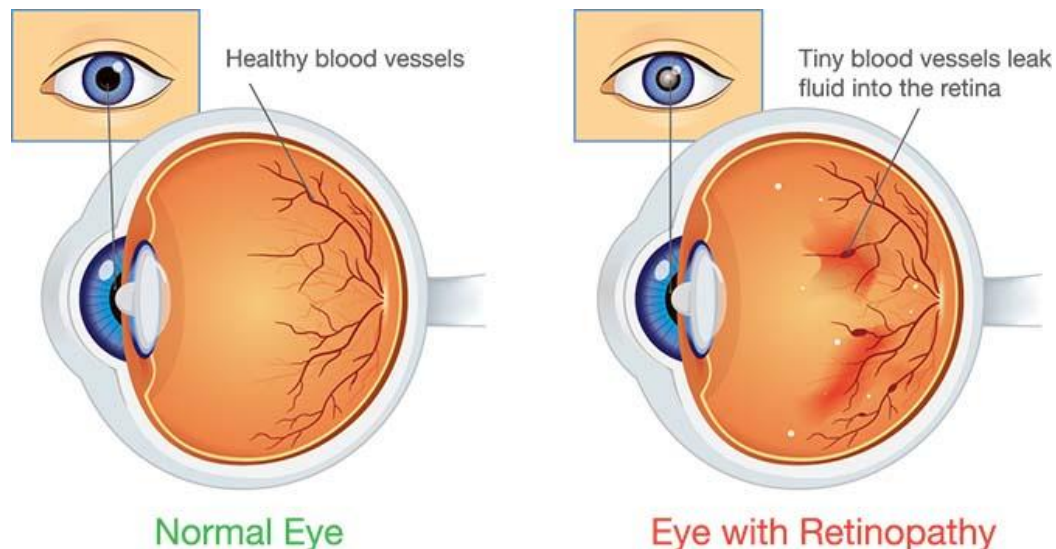
2. Introduction to eye physiology



Common eye diseases

Diabetic retinopathy

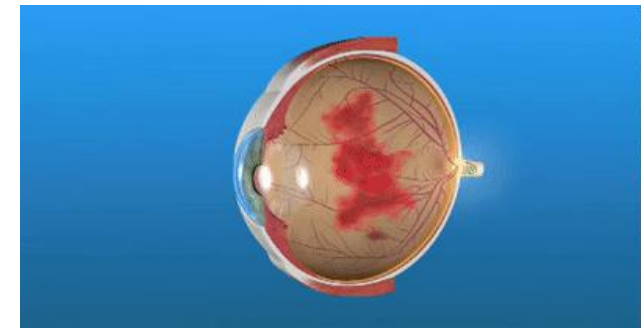
- **diabetes complication** that affects eyes
- high blood sugar levels cause damage to **blood vessels** in the retina



NORMAL

Diabetic Retinopathy

Impaired vision, dark or empty areas



Vitrectomy surgery for diabetic retinopathy

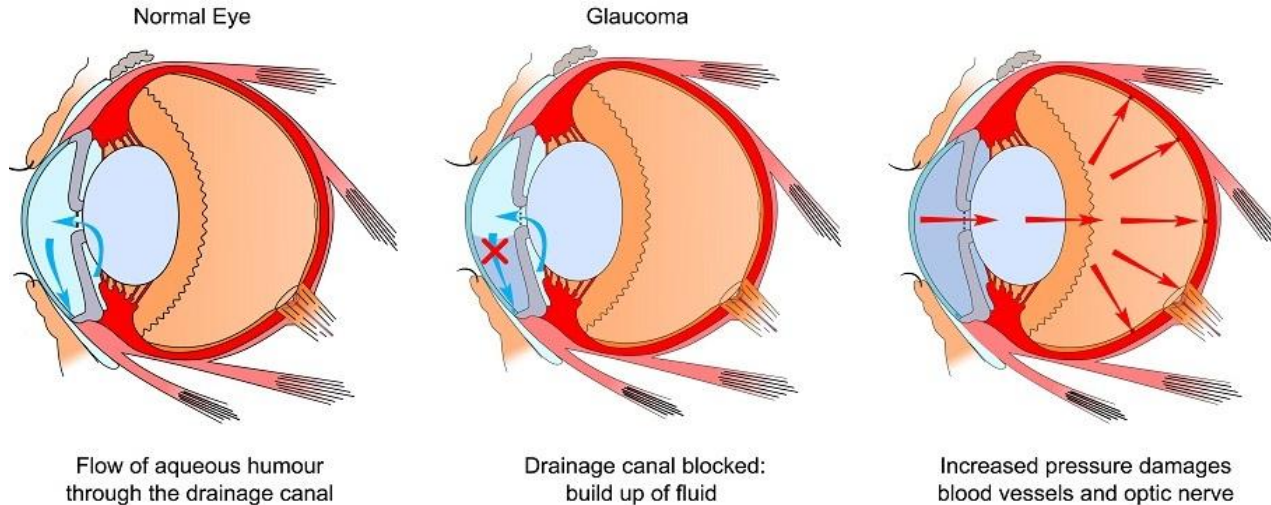
2. Introduction to eye physiology



Common eye diseases

Glaucoma

- one of the leading causes of **blindness** for age above 60
- abnormally **high pressure** in the eye
- if damage to optic nerve, may cause permanent vision loss or even blindness



Prescription eye drops or Glaucoma lasers

2. Introduction to eye physiology



Common eye diseases

Age-related macular degeneration

- it happens when the **macula** is damaged
- very **common**
- often happens in **elder** people > 50
- **blur the sharp**, central vision you need for activities like reading and driving

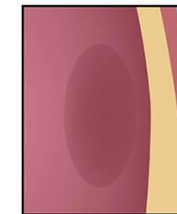
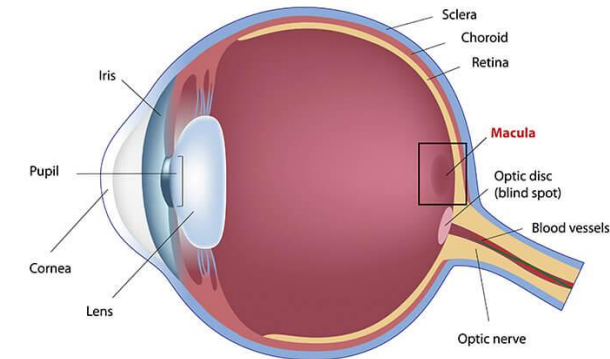


Normal Vision

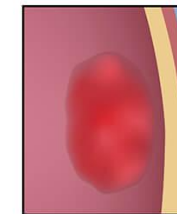


The same scene affected by age-related macular degeneration

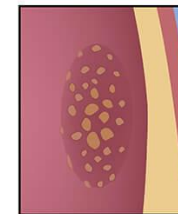
Macular Degeneration



Normal



"Wet" Macular Degeneration



"Dry" Macular Degeneration

Treatment:
medications called anti-VEGF drugs

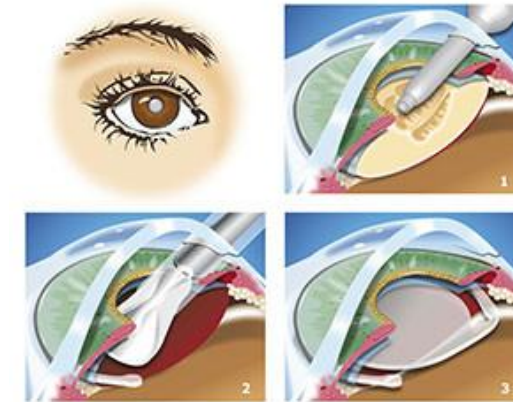
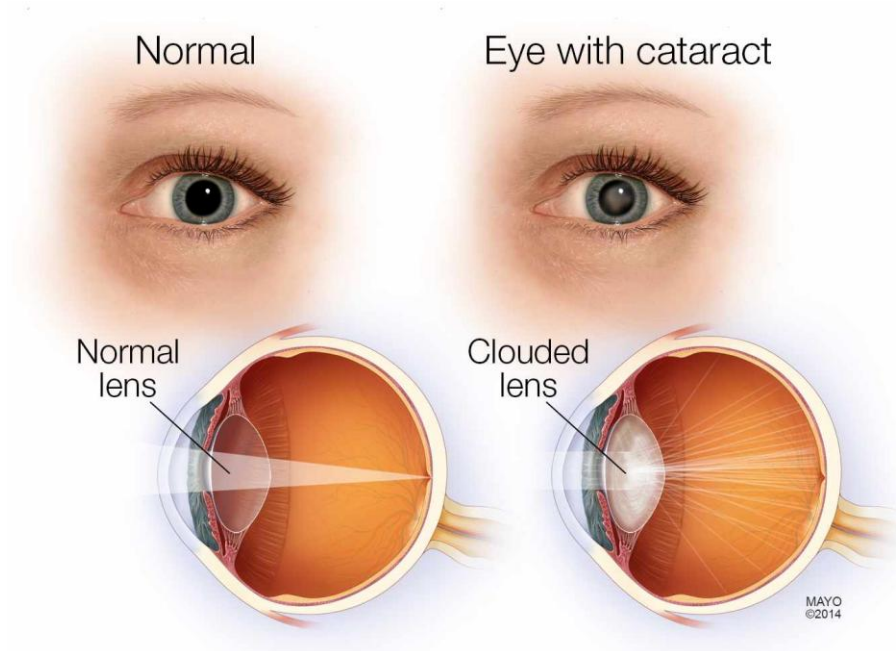
2. Introduction to eye physiology



Common eye diseases

Cataract

- is a cloudy area in the lens of your eye
- very **common** in **older** people
- **clouded, blurred or dim vision**

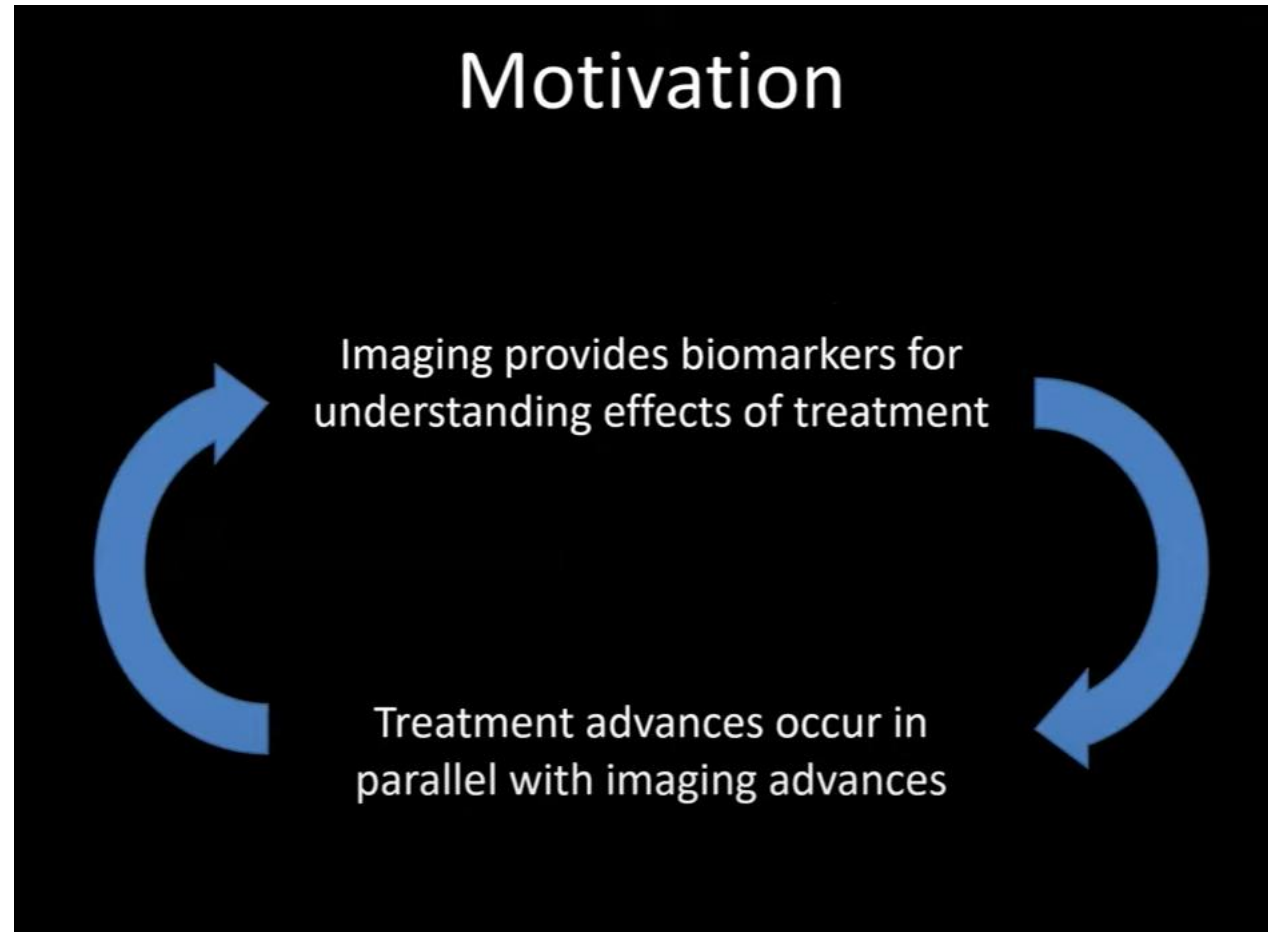


Cataract surgery involves replacing clouded lens with artificial lens

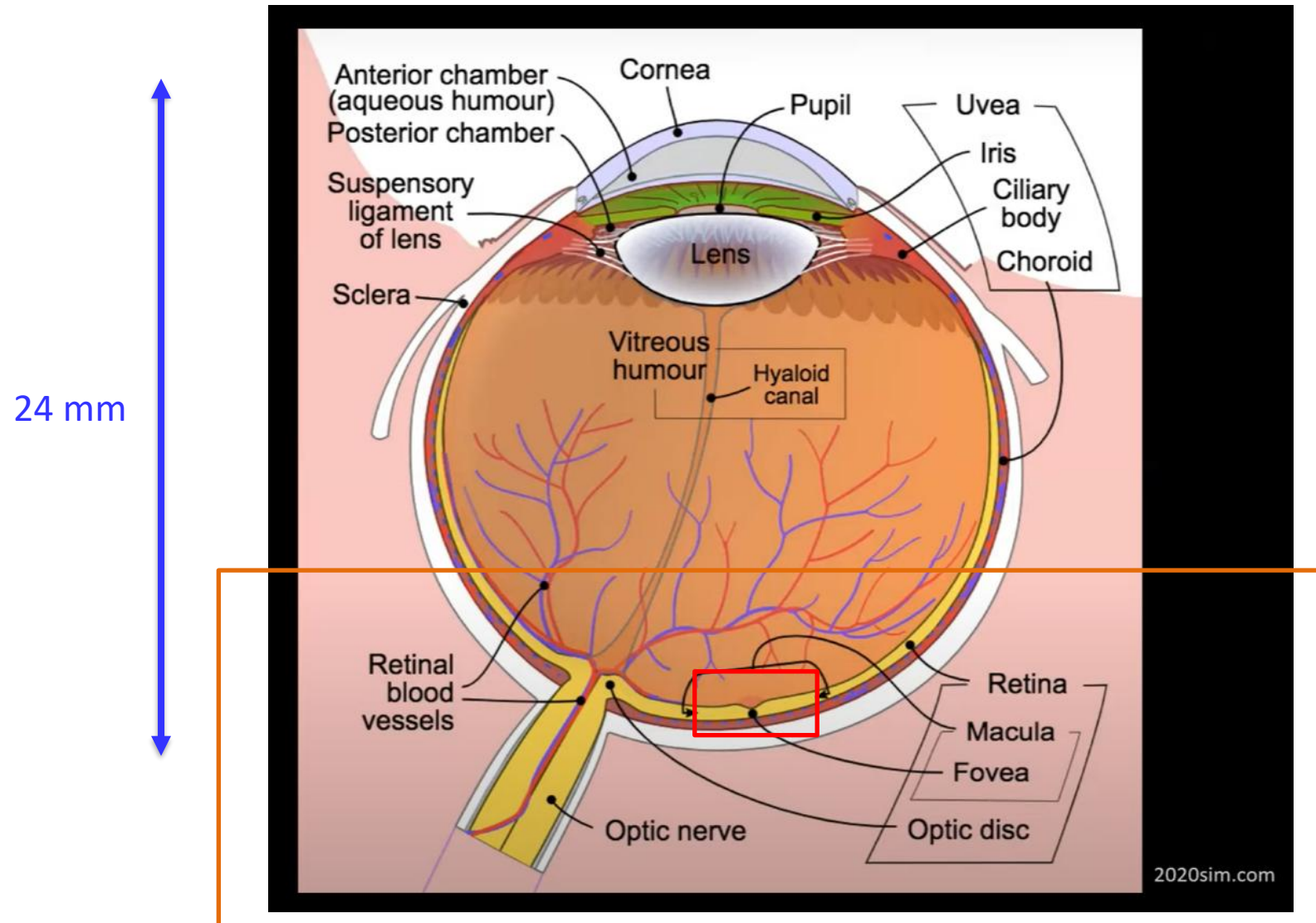


Next Topic: Fundus images and OCT

3. Fundus images and OCT



3. Fundus images and OCT



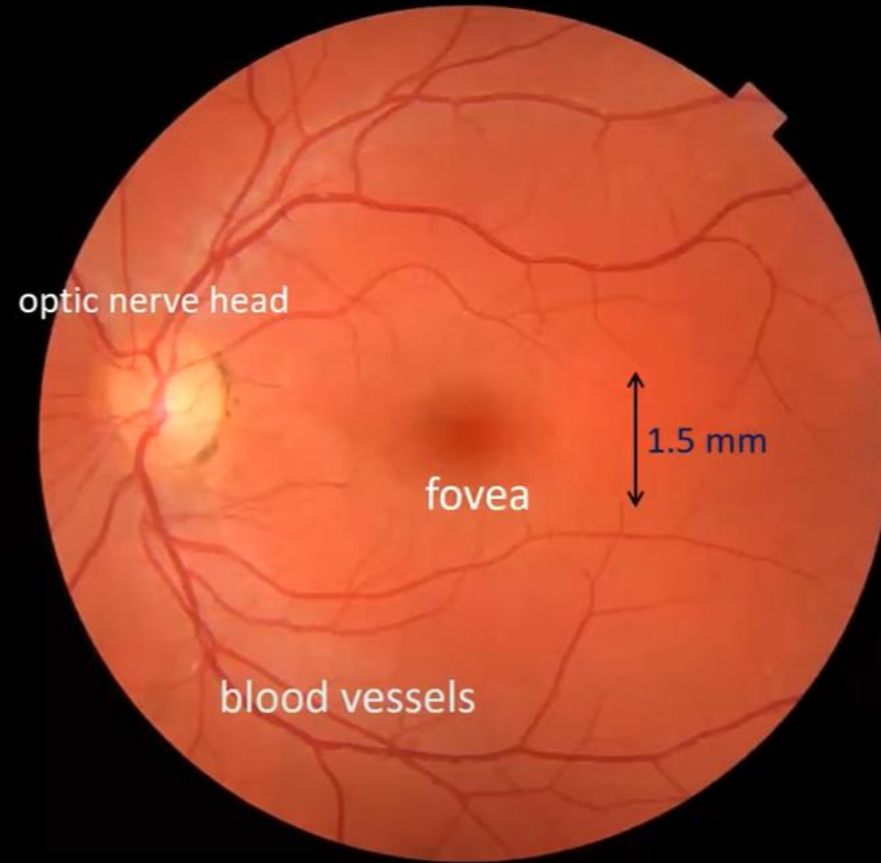
Posterior pole

3. Fundus images and OCT



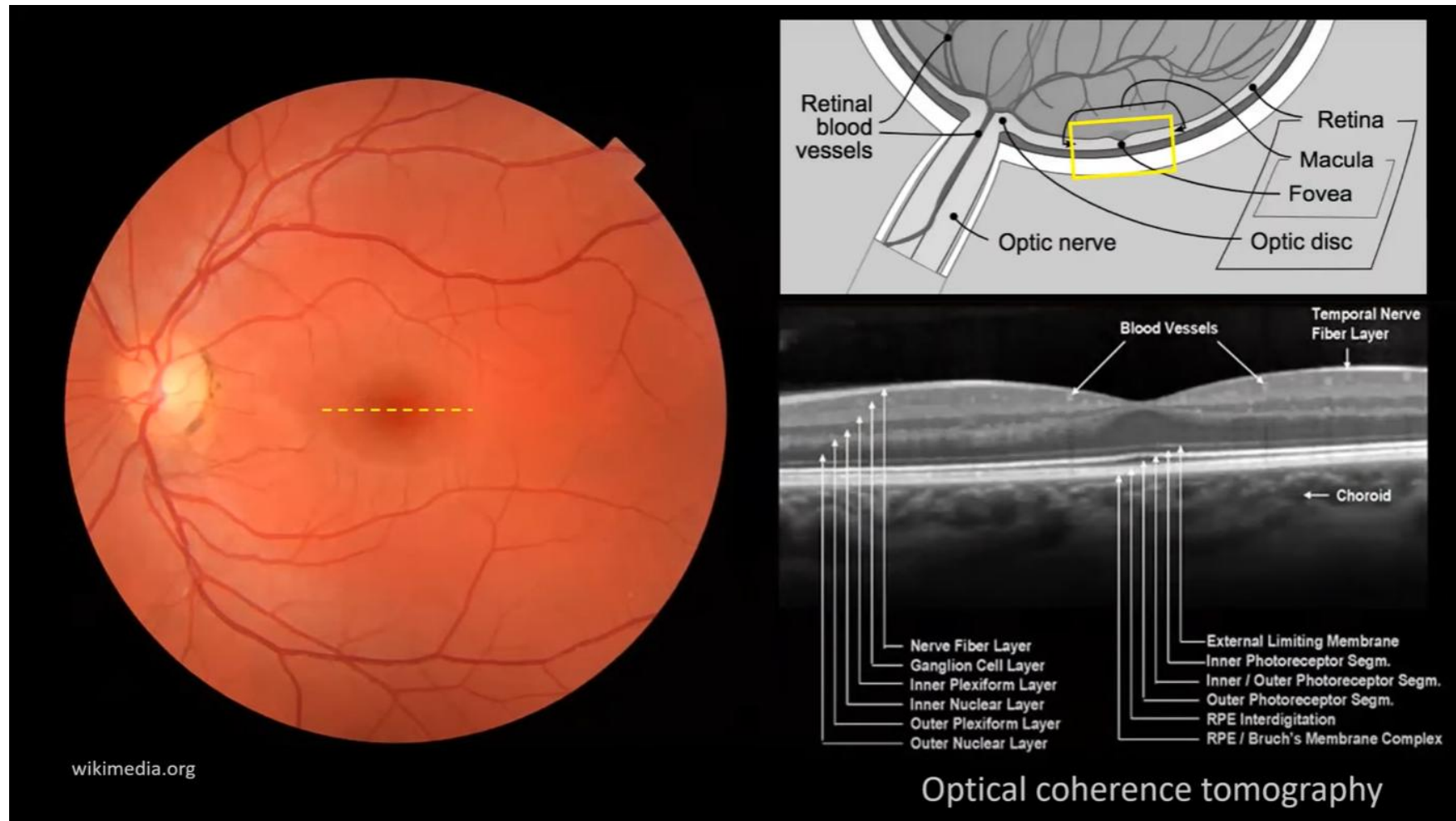
Normal
posterior pole

color
photograph



wikimedia.org

3. Fundus images and OCT



3. Fundus images and OCT



Light-based imaging modalities

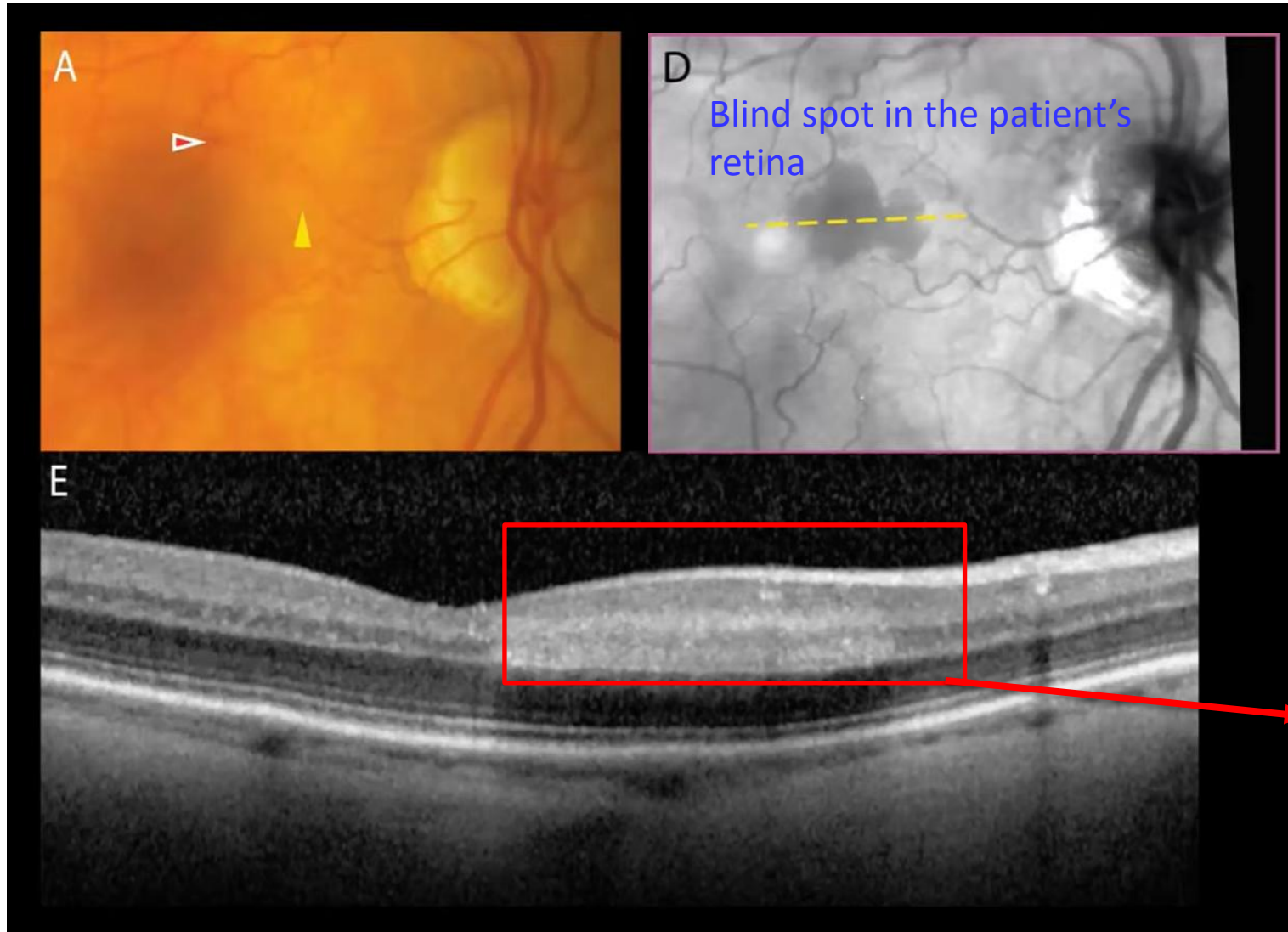
REFLECTANCE	FLUORESCENCE	3D	TECHNOLOGY
color	autofluorescence	optical coherence tomography (OCT)	photography
red-free	dye angiography	SD-OCT and SS-OCT	scanning laser ophthalmoscopy
blue		OCT angiography	frame averaging
near-infrared		directional OCT	adaptive optics

3. Fundus images and OCT



color photograph of a patient, who has an interruption of blood flow on the retina

However, color photo can not contribute a lot in understanding this case.



infrared photography

reflects one of very deep layers on behind of retina.

In addition to standard fundus photography, serial IR imaging may be an important adjunct to detecting outer retinal pathology with patients with peripheral retinoschisis.

A band of tissue within the retina which has increased brightness.

Also causes a shadow behind it.

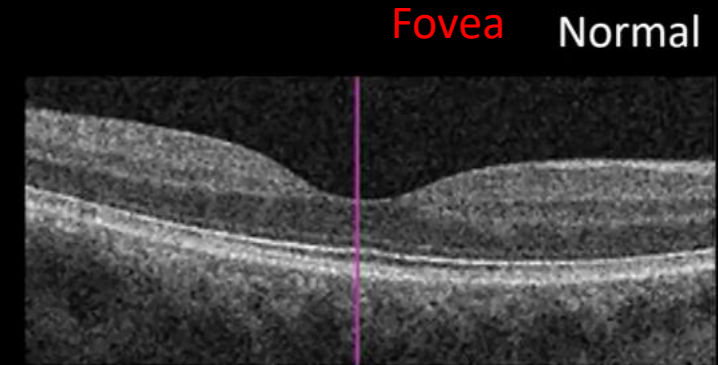
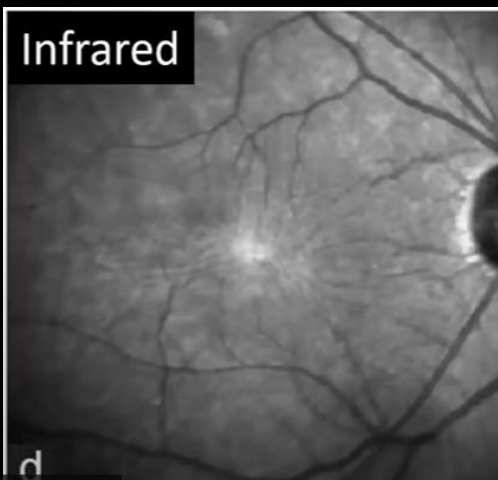
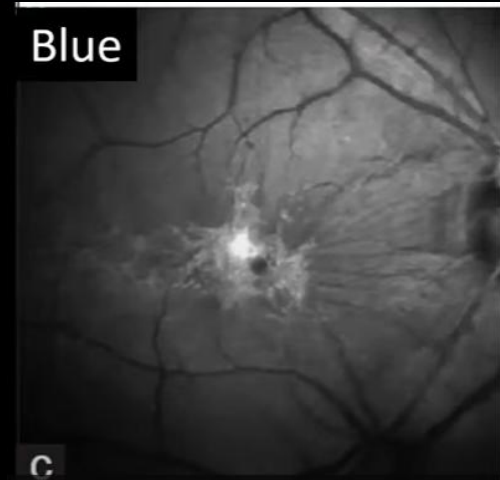
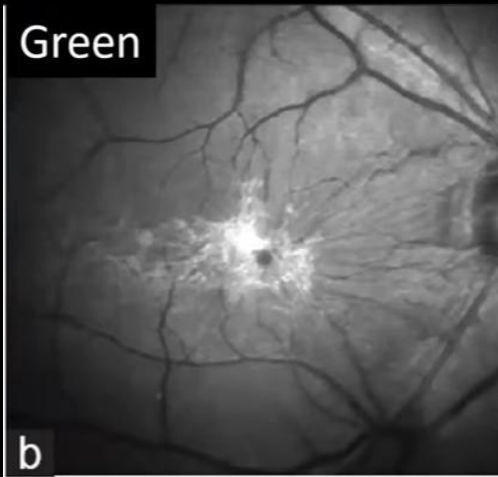
3. Fundus images and OCT



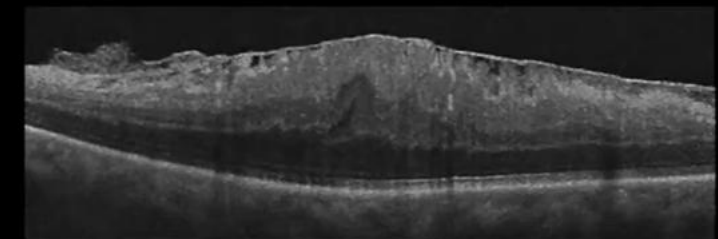
A patient with
epiretinal
membrane.

A layer of scar tissue
which has gradually
grown the surface of
the retina.

Blue flash in the
camera that
delineates the
edges of the
membrane.

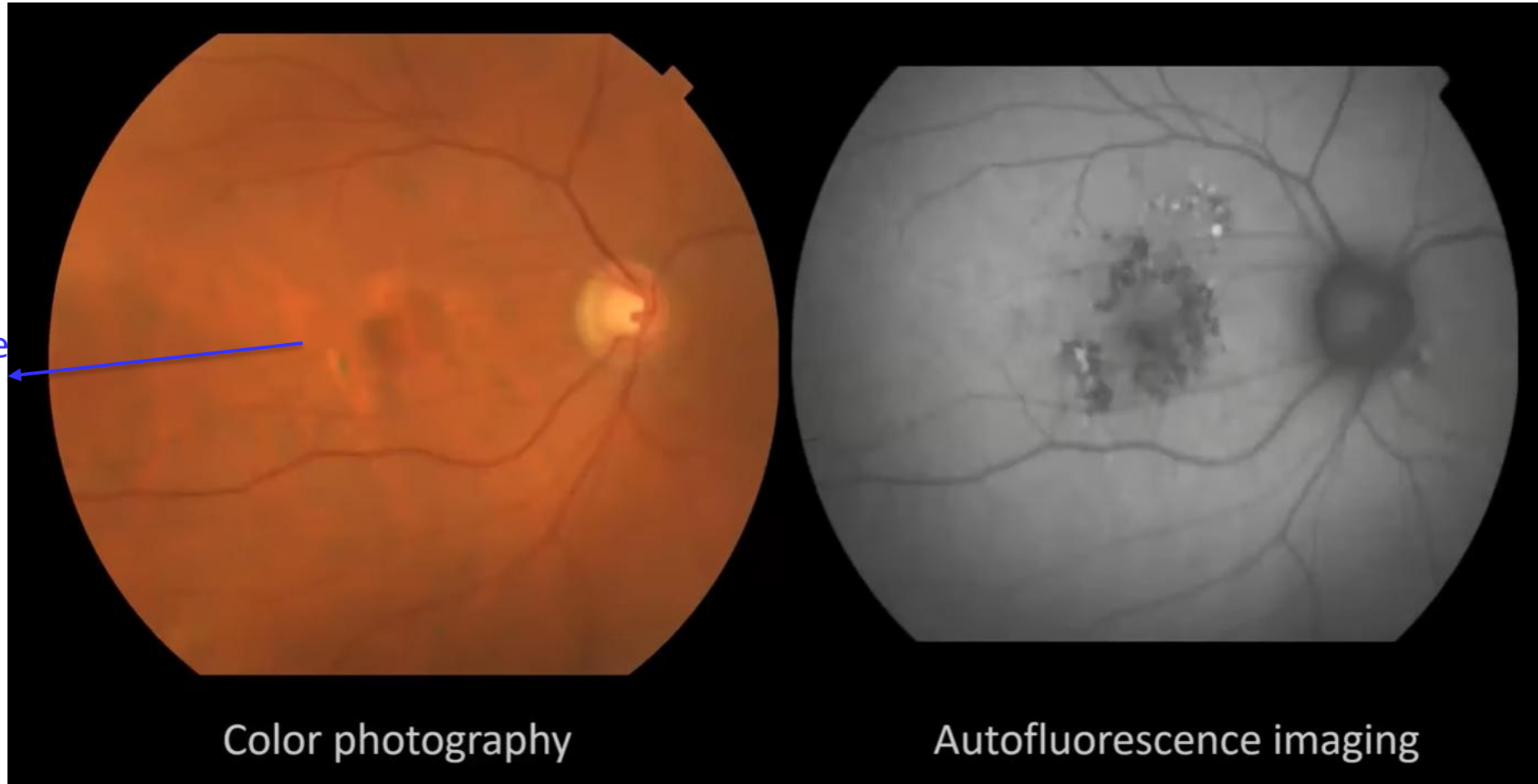


Has grown over the surface of the fovea
Epiretinal membrane



Giridhar and Ranjith, 2018

3. Fundus images and OCT



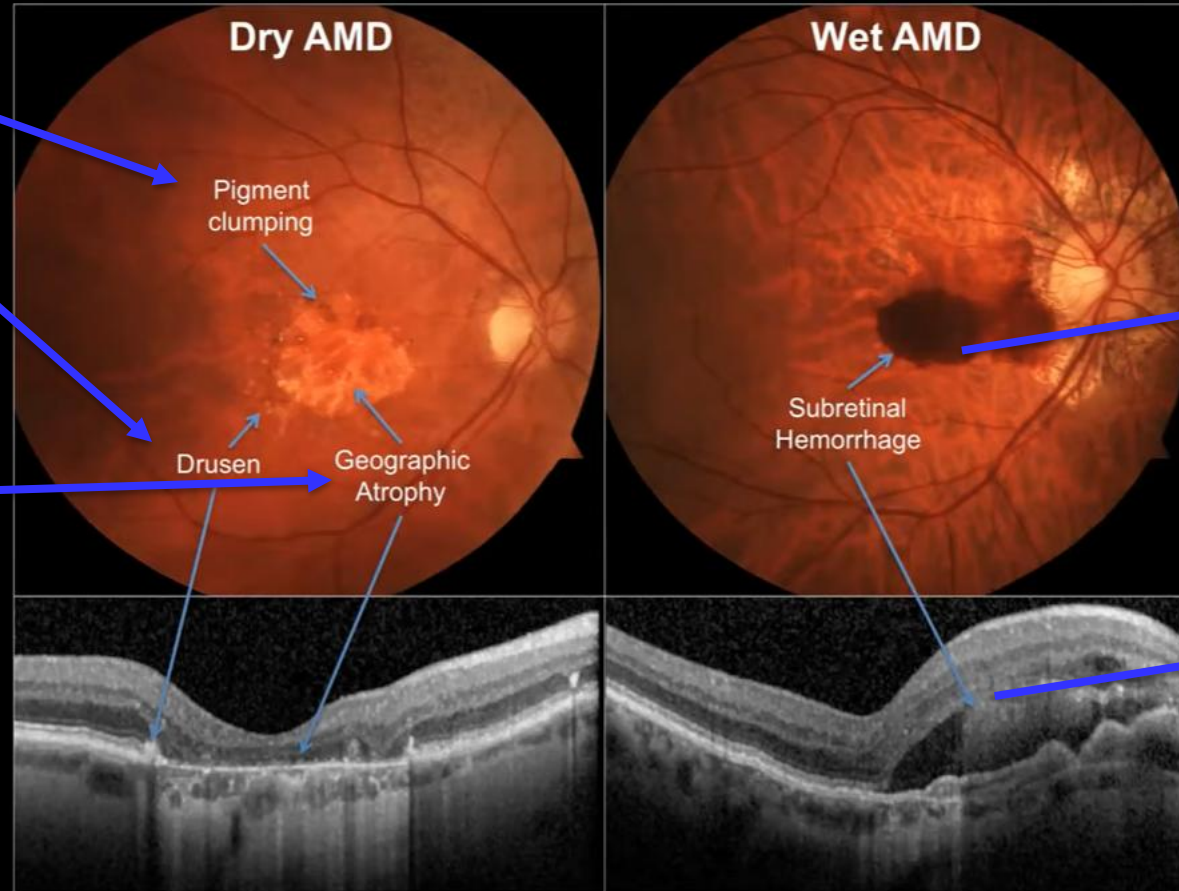
Not so much a structural image. It is an image of the distribution of certain molecules in the retinal pigment epithelium.

We can see not only structural info. but also, molecular info what is inside the cell.

3. Fundus images and OCT



Age-related macular degeneration



Very characteristic finding in macular degeneration.

Retinal tissue has really thinned out.

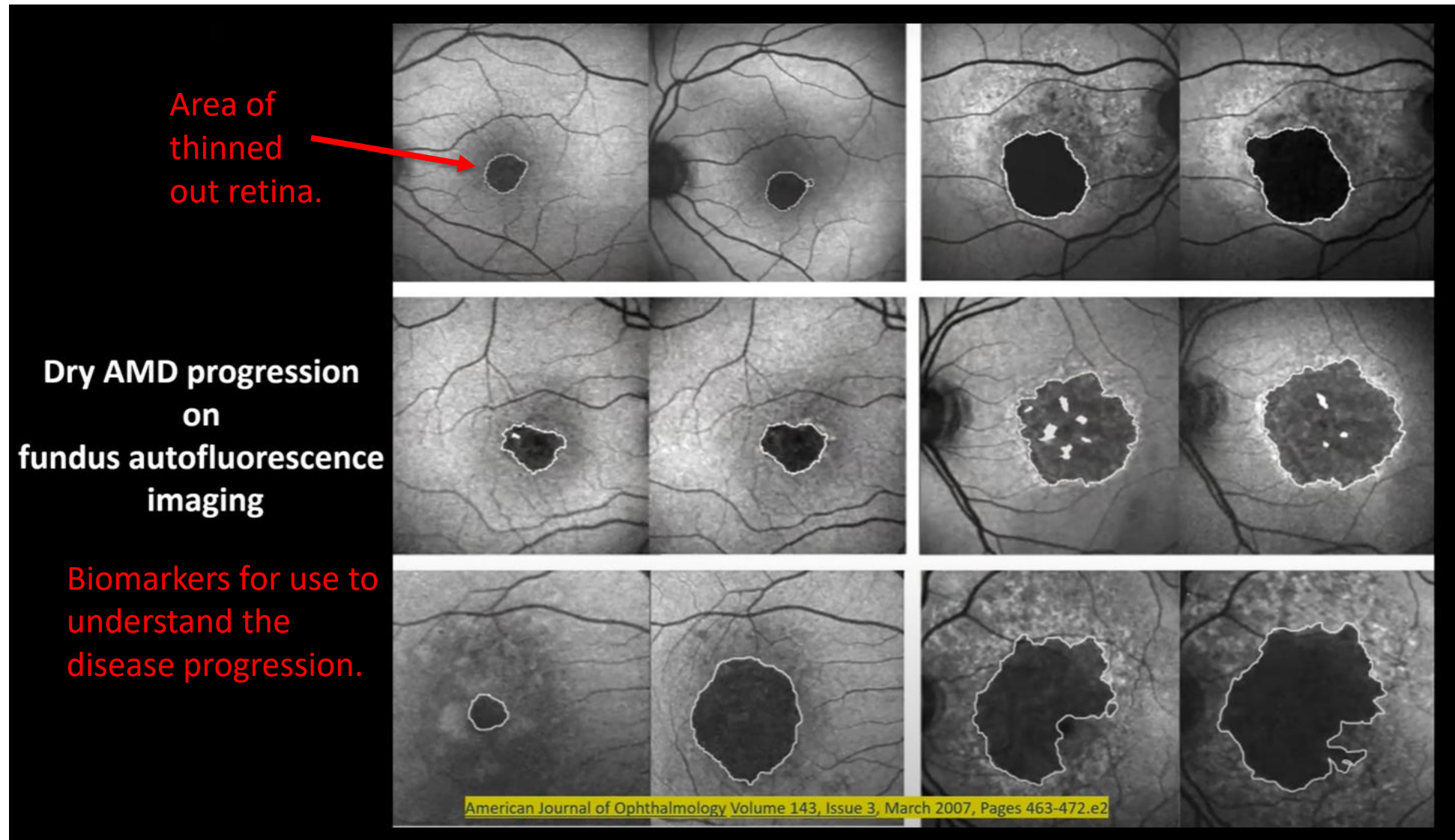
Confirmed by OCT

Some bleeding under the retina.

By using the OCT, we can look at the hemorrhage and see what level of hemorrhage is.

Knickelbein, 2019

3. Fundus images and OCT

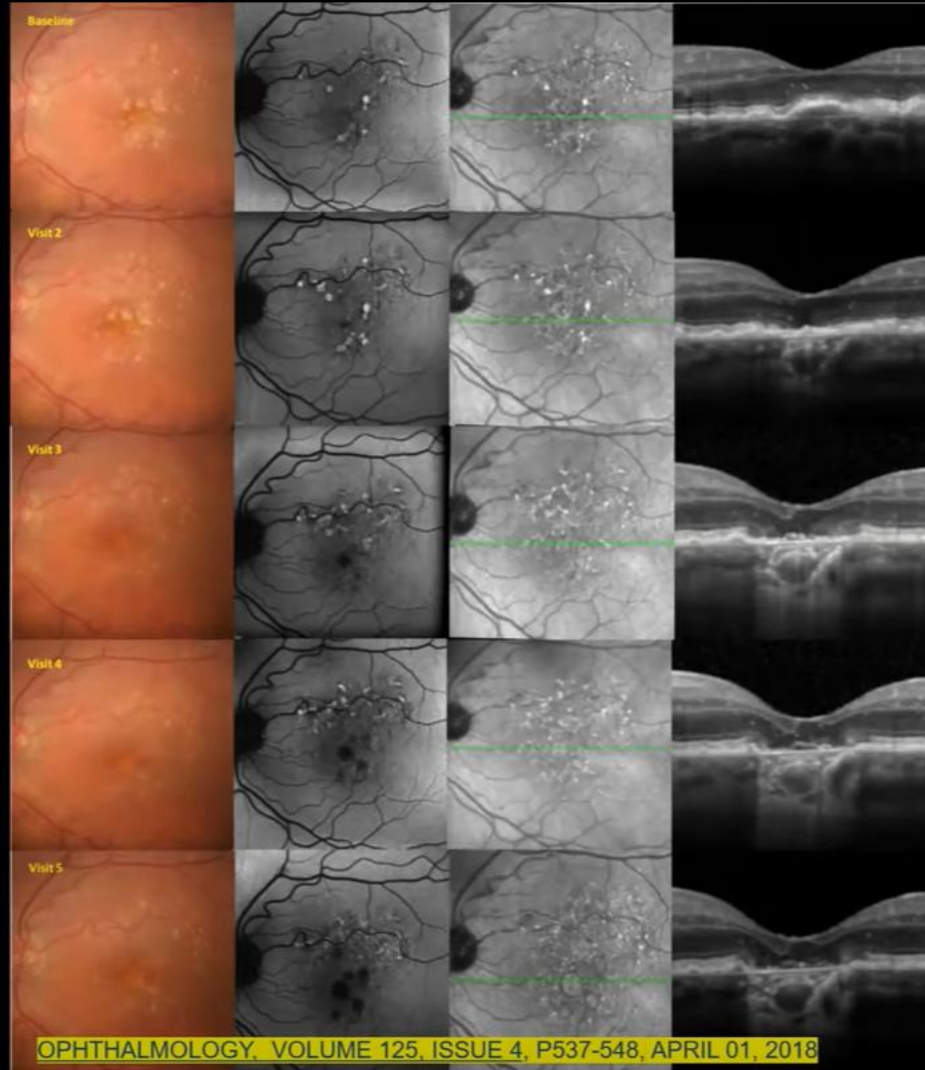


3. Fundus images and OCT



Fundus photograph,
Autofluorescence image,
Infrared image,
OCT scan which pass
through the central part
of the retina.

**Dry AMD progression
on
Multimodal
imaging**



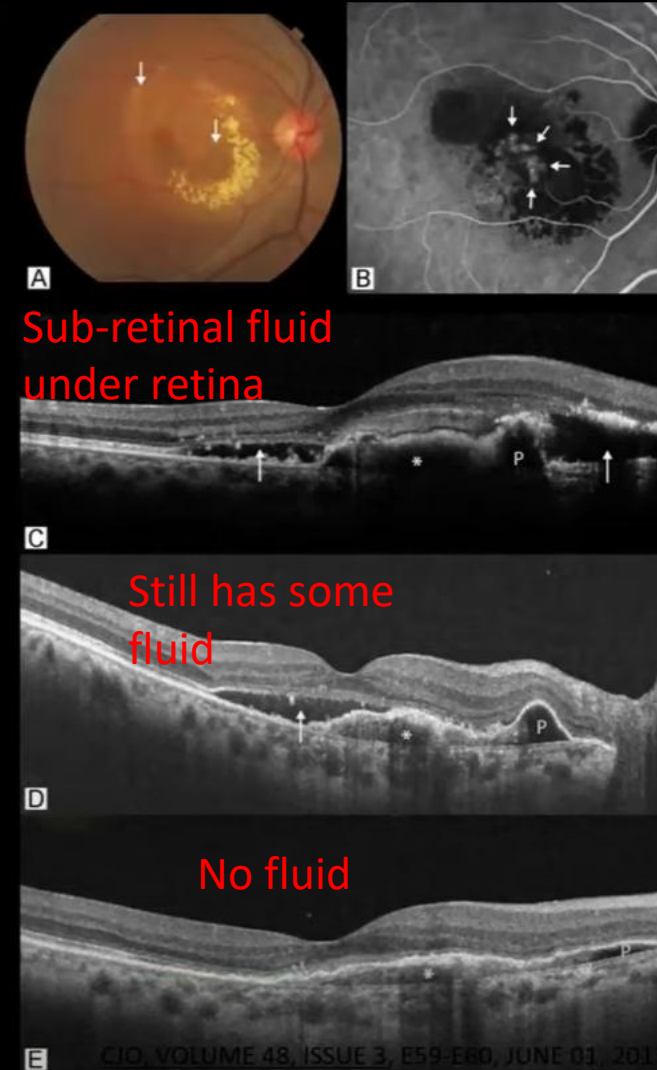
Outermost layer which is retinal pigment epithelium, starts thinning out over the course and this eventually leads to a trophic area which is involving right in the center.

3. Fundus images and OCT



Wet AMD treatment response

OCT scans are taken in a micron level, which can pick up the disease changes very soon and is a backbone for Wet AMD and diabetic macular edema examination.



Subretinal hemorrhage

We perform injections

Treatment response

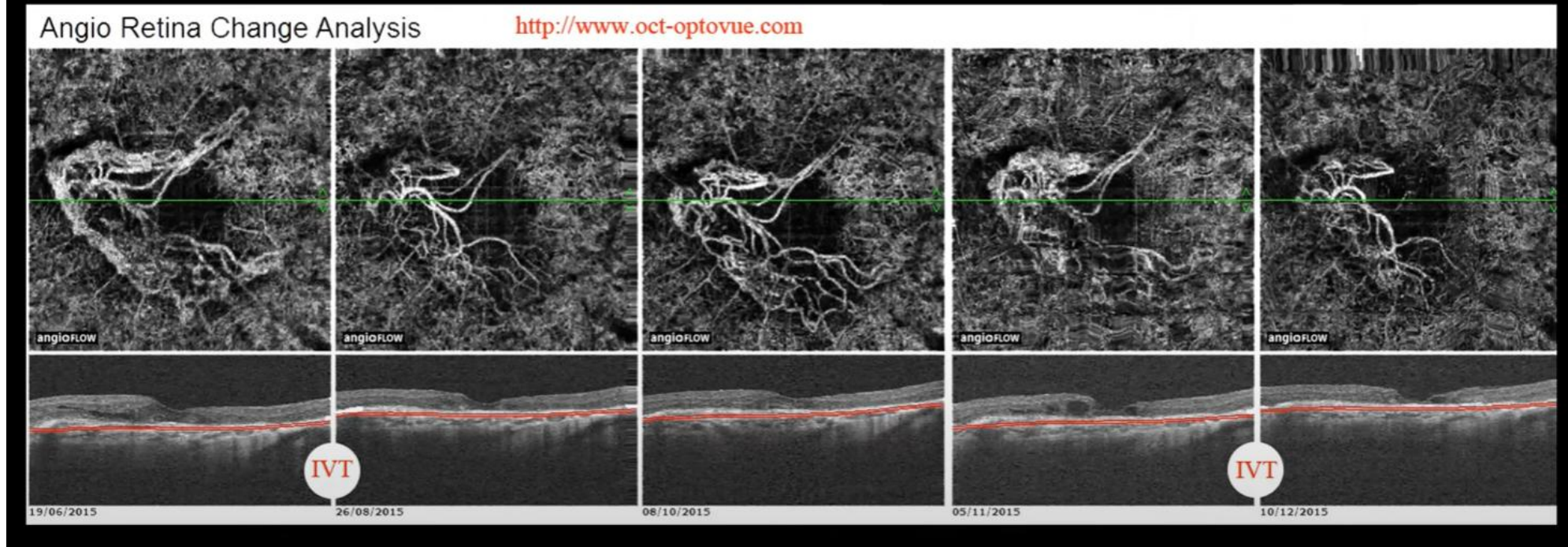
Treatment response

3. Fundus images and OCT



Neovascular membrane progression on OCT angiography

No injection of dye
Can see vessel non-invasively.

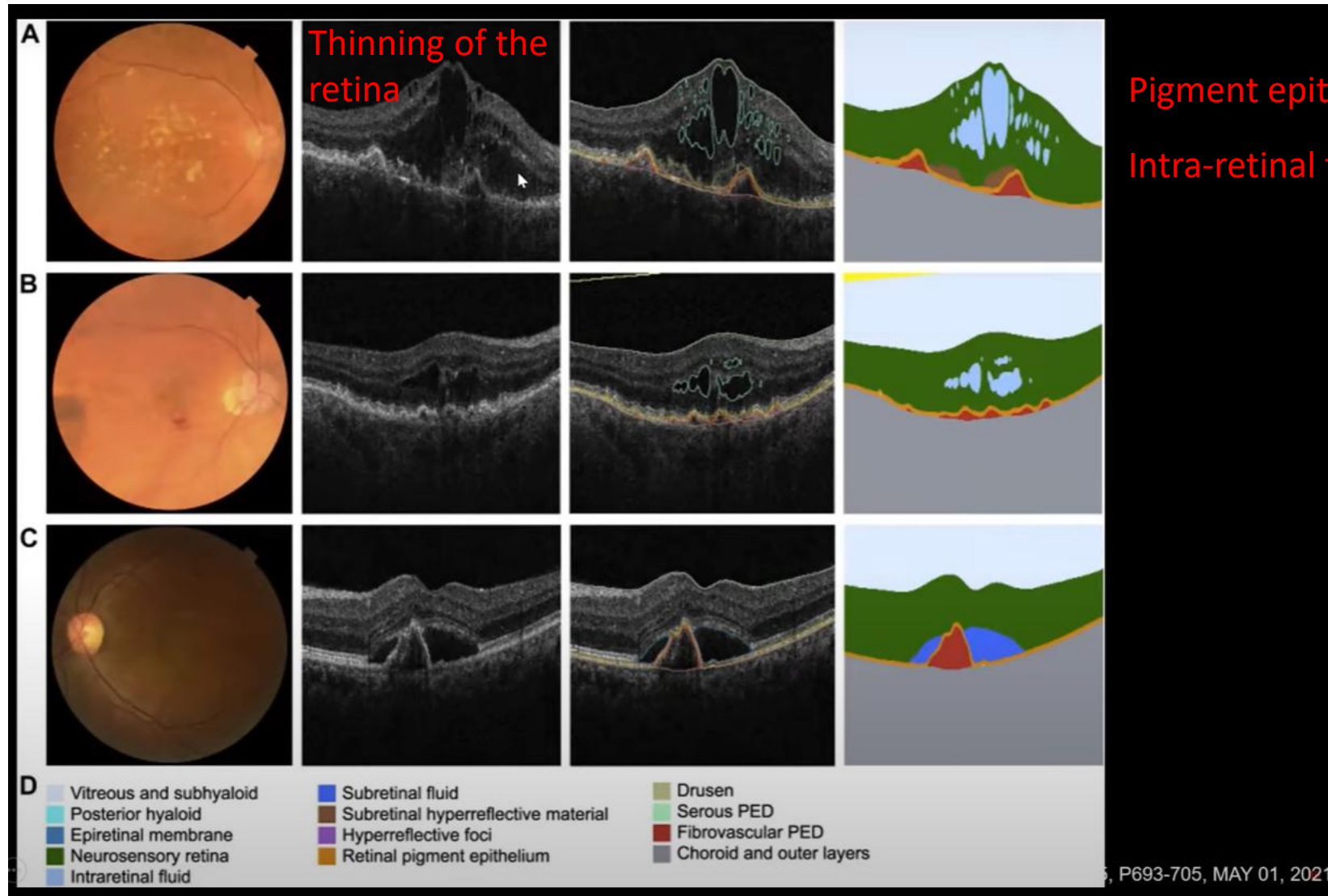


Can help doctors decide whether need to continue the treatment or stop.

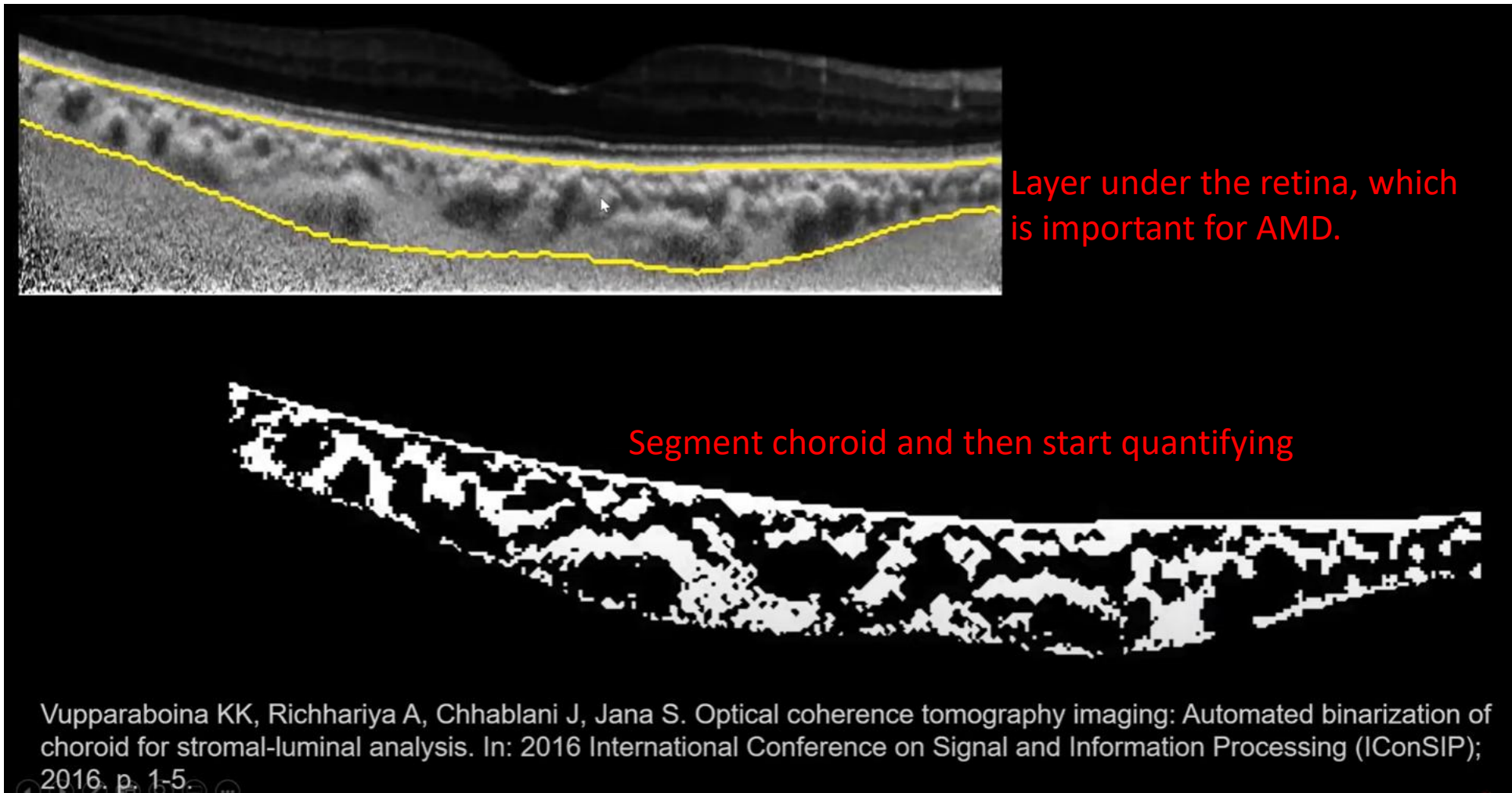
3. Fundus images and OCT



A patient with wet macular degeneration.



3. Fundus images and OCT



3. Fundus images and OCT



How to take retinal photography

[Let's watch a video!](#)

3. Fundus images and OCT



About OCT?

[Let's see some explanations.](#)

4. Deep learning for retinal image analysis

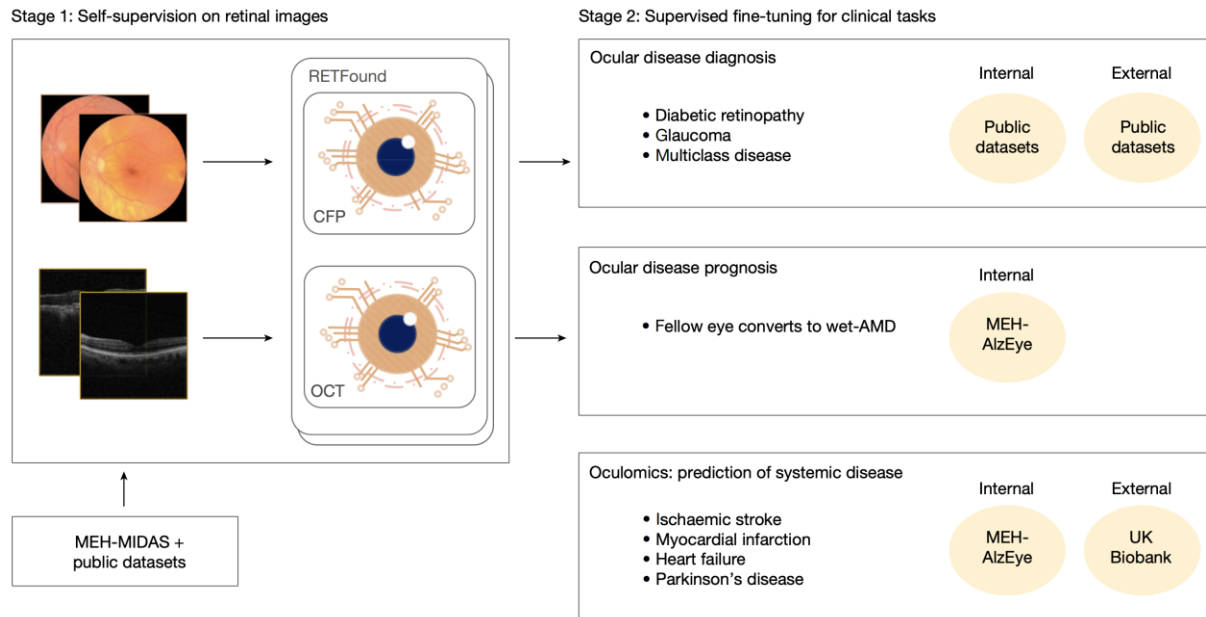


A foundation model for generalizable disease detection from retinal images

Background:

Many retinal disease can be detected from Fundus image and OCT image. Training a deep learning model for each diseases will be resource-demanding. It is also hard to annotate a large number of images for doctors.

How to develop a model that can recognize various retinal diseases without requiring a large amount of labeled data?



Solution: Self-supervised training foundation models!

- **Learns visual feature by utilizing medical image itself**
- **Large model parameters learns robust and generalizable image patterns and features**

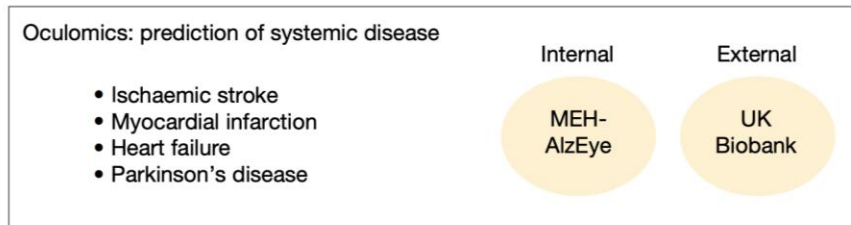
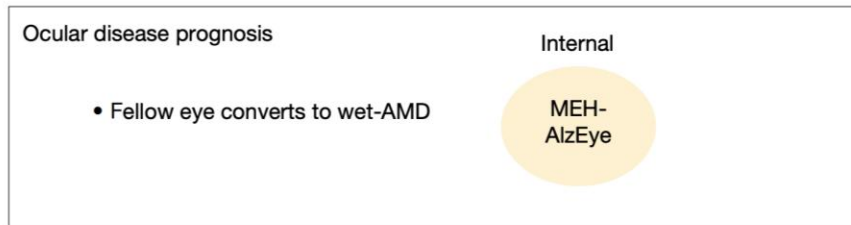
4. Deep learning for retinal image analysis



A foundation model for generalizable disease detection from retinal images

Performed Task - what it can do?

Stage 2: Supervised fine-tuning for clinical tasks



Diagnosis: Recognize certain retinal diseases

- **Diabetic retinopathy:** Visual impairment for diabeto patients.
- **Glaucoma:** optic nerve damage that can cause blindness.
- Multiple Eye Conditions and Diseases: AMD...

Prognosis: Predict whether the retinal disease will become worse.

- **Fellow eye -> wet-AMD prediction:** whether the hypertension in eye will turn into Wet macular degeneration.

Systemic disease: Predict the disease happened in other parts of body.

- **Brain:** Ischaemic stroke & Parkinson's disease
- **Heart:** Myocardial infarction & heart failure

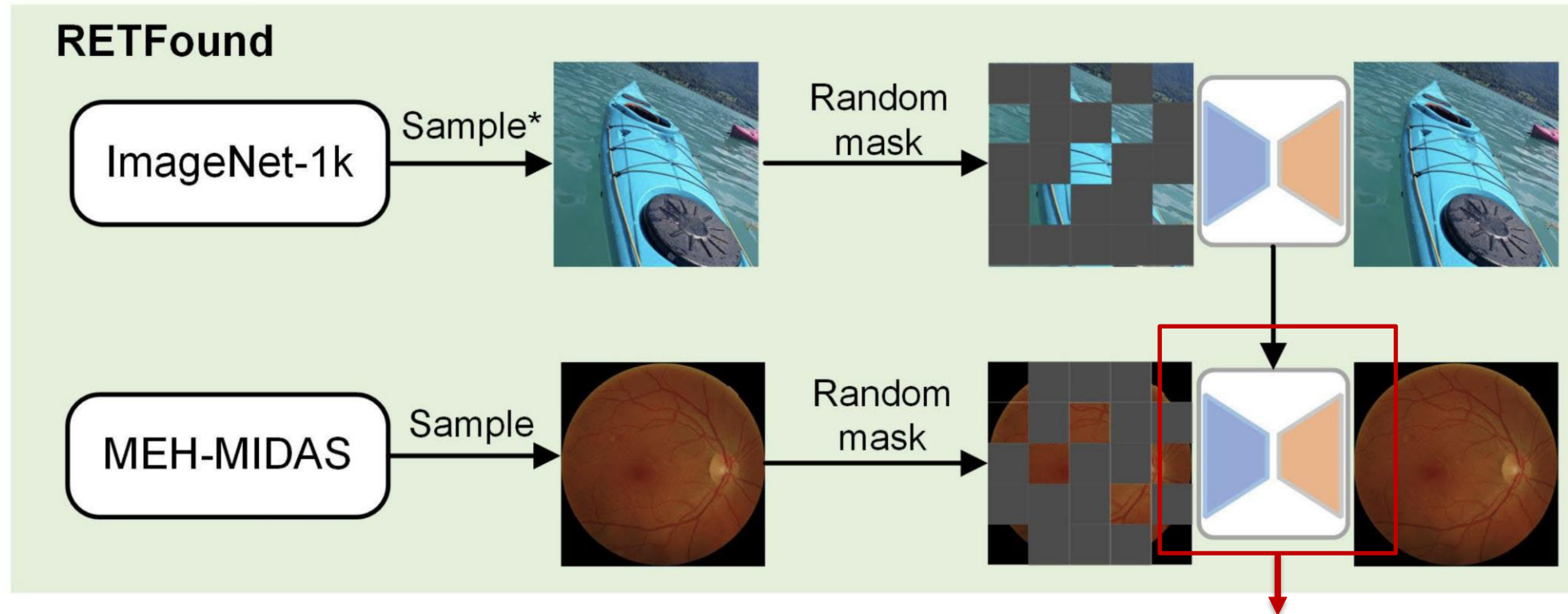
4. Deep learning for retinal image analysis



A foundation model for generalizable disease detection from retinal images

Methodology

Step 1: Self-supervised Pretraining on large-scale Fundus images and OCT images using masked autoencoder (MAE)



The model learns how to recover the masked image patches. This task forces the network learn critical image features.

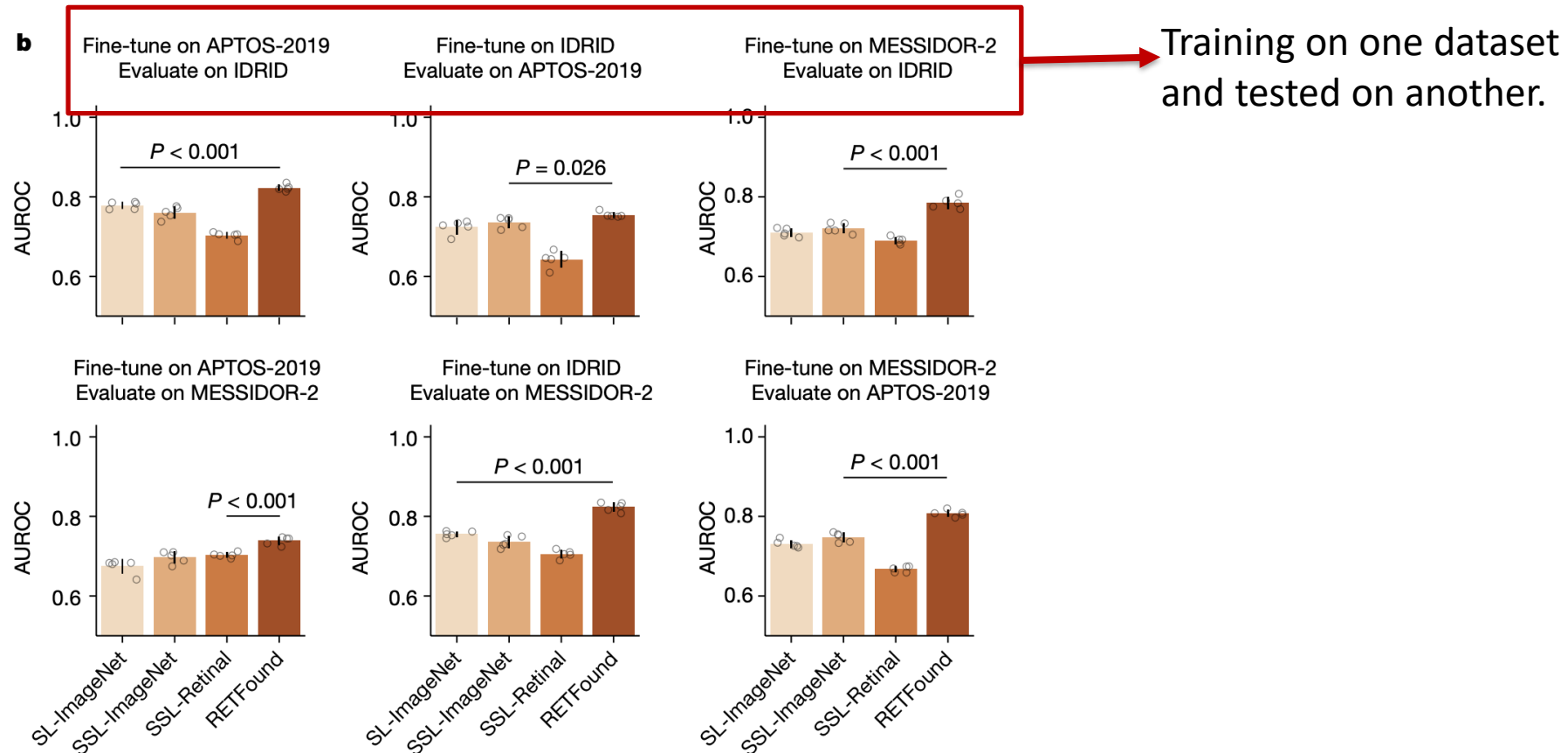
4. Deep learning for retinal image analysis



A foundation model for generalizable disease detection from retinal images

Results and Findings

Finding 1: RETFound performs the best when diagnosing various retinal diseases compared with all small specialized models.



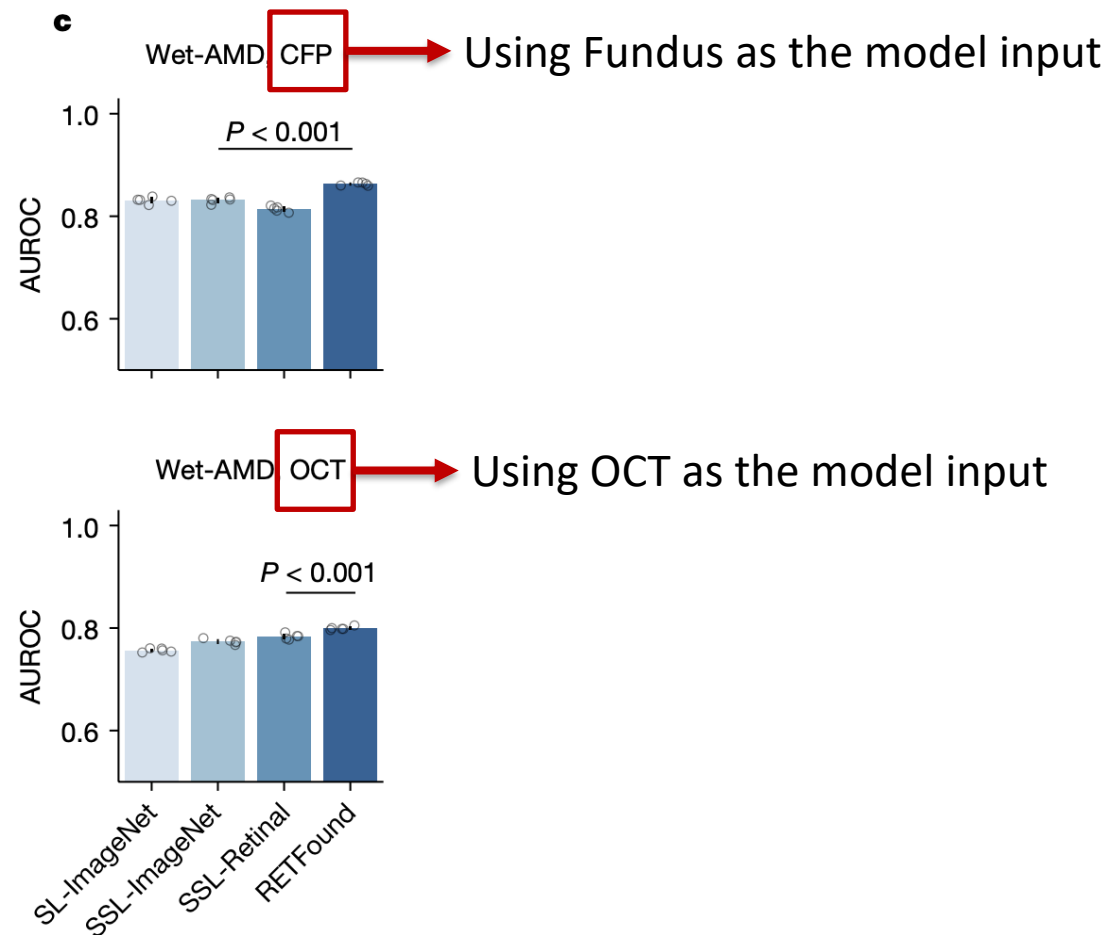
4. Deep learning for retinal image analysis



A foundation model for generalizable disease detection from retinal images

Results and Findings

Finding 2: RETFound performs the best when performing prognosis task (predicting the patient's future event)



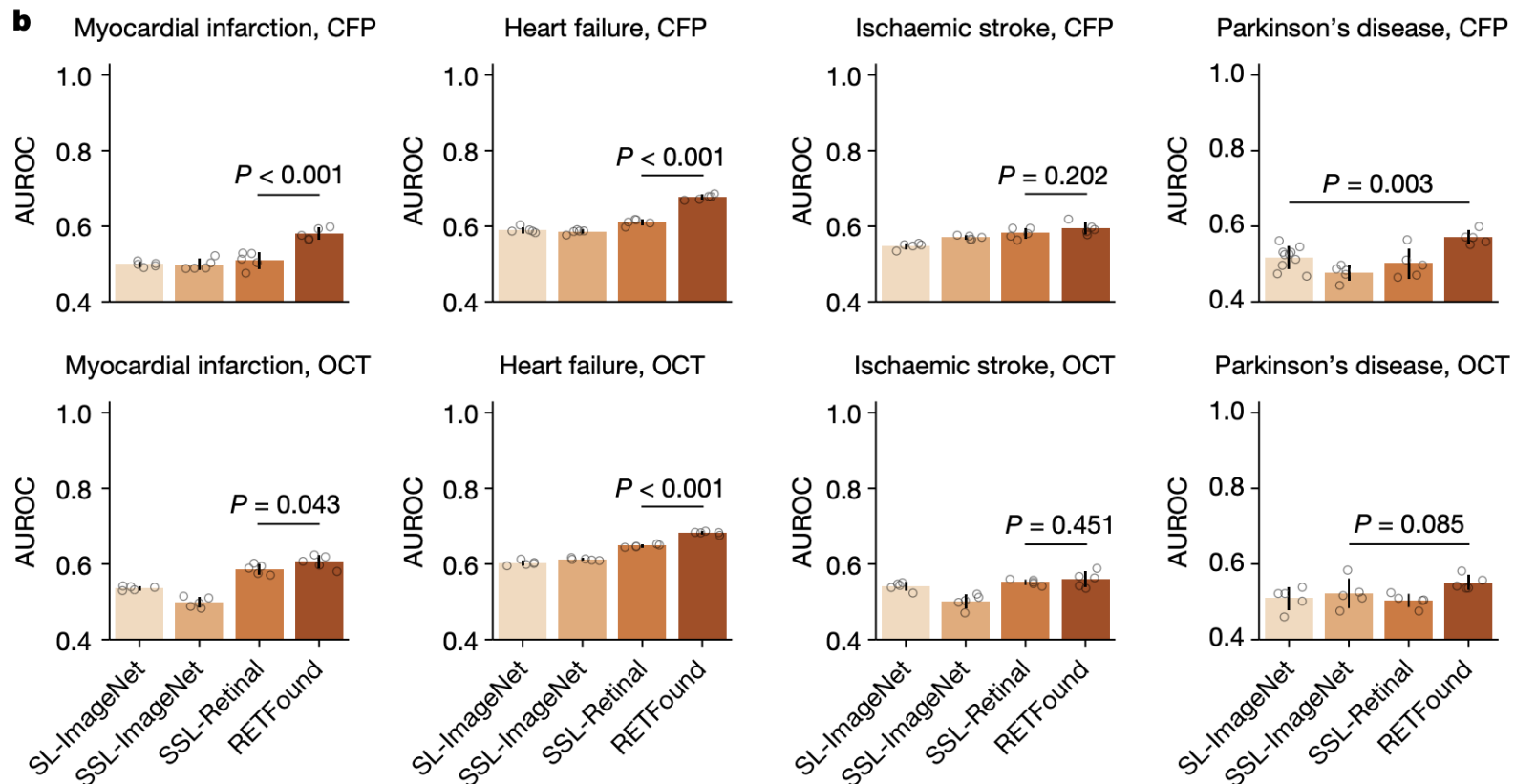
4. Deep learning for retinal image analysis



A foundation model for generalizable disease detection from retinal images

Results and Findings

Finding 3: RETFound can learn effective features that can predict future systemic diseases.



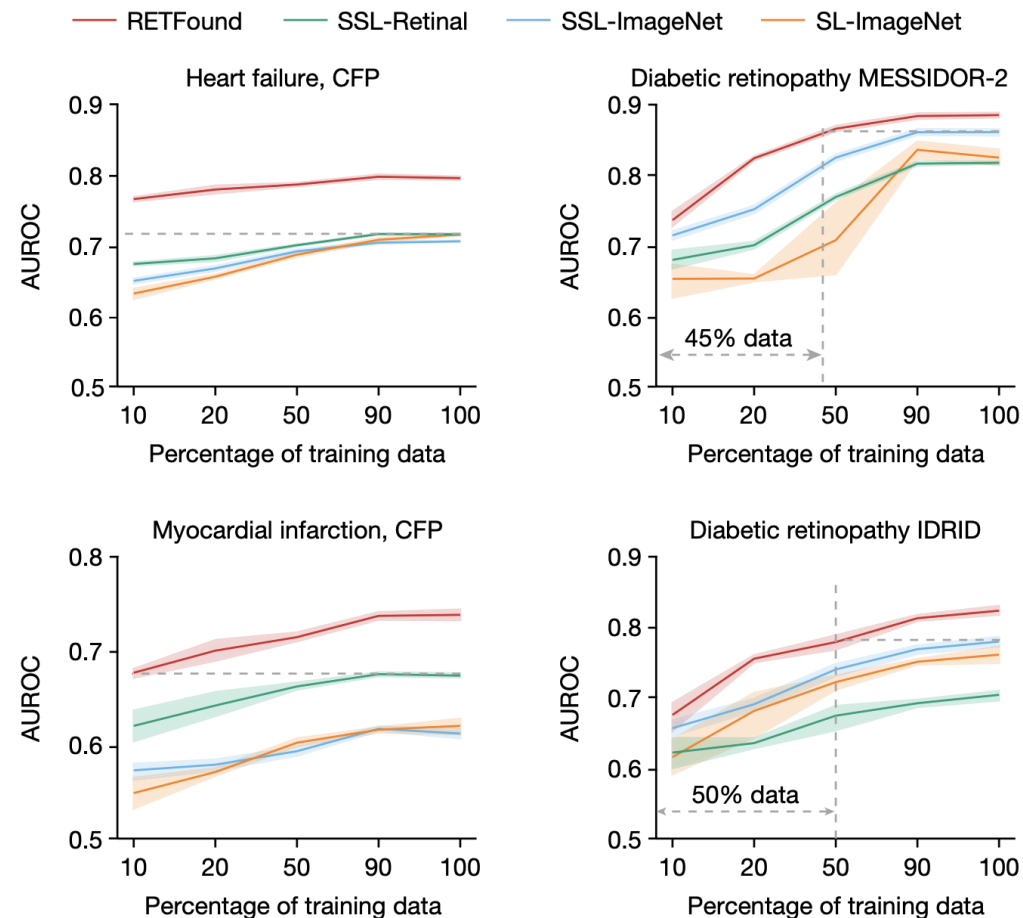
4. Deep learning for retinal image analysis



A foundation model for generalizable disease detection from retinal images

Results and Findings

Finding 4: RETFound requires less labeled data to achieve better performance.



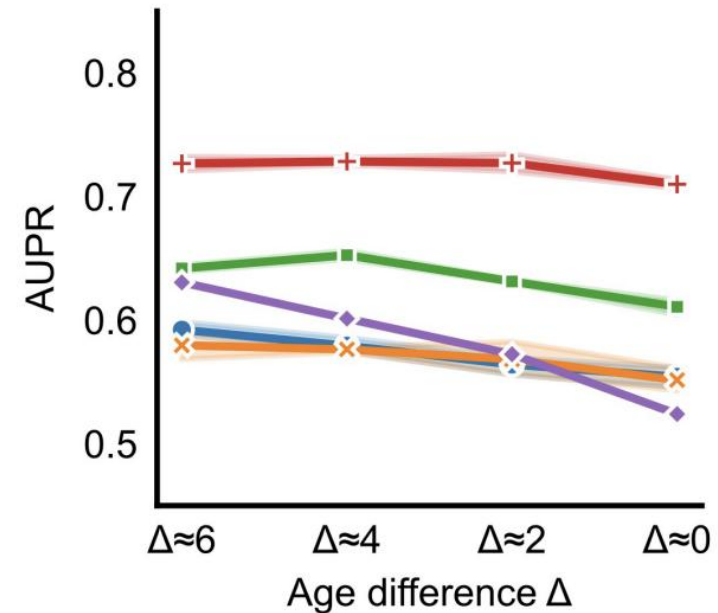
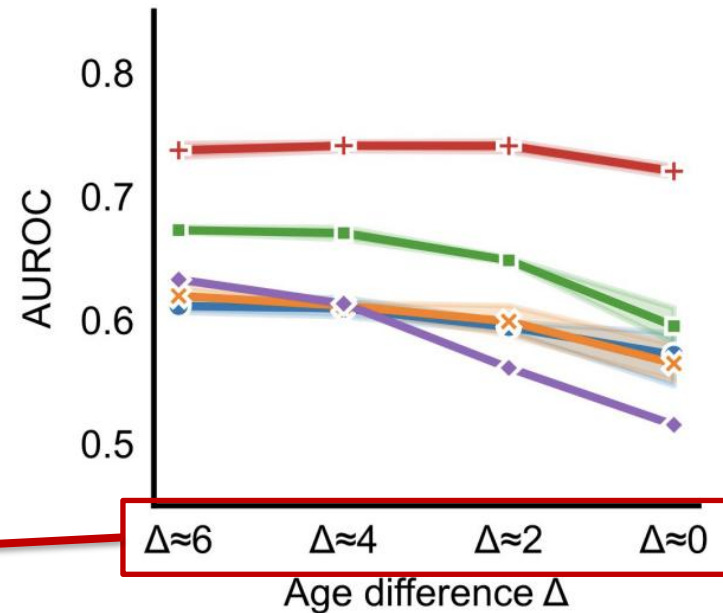
4. Deep learning for retinal image analysis



A foundation model for generalizable disease detection from retinal images

Results and Findings

Finding 5: RETFound is robust to age distribution drifts. For example, patients with different age can enjoy the accurate prediction.



The age difference between training and testing set.

— RETFound — SSL-Retinal — SSL-ImageNet — SL-ImageNet — Logistic regression

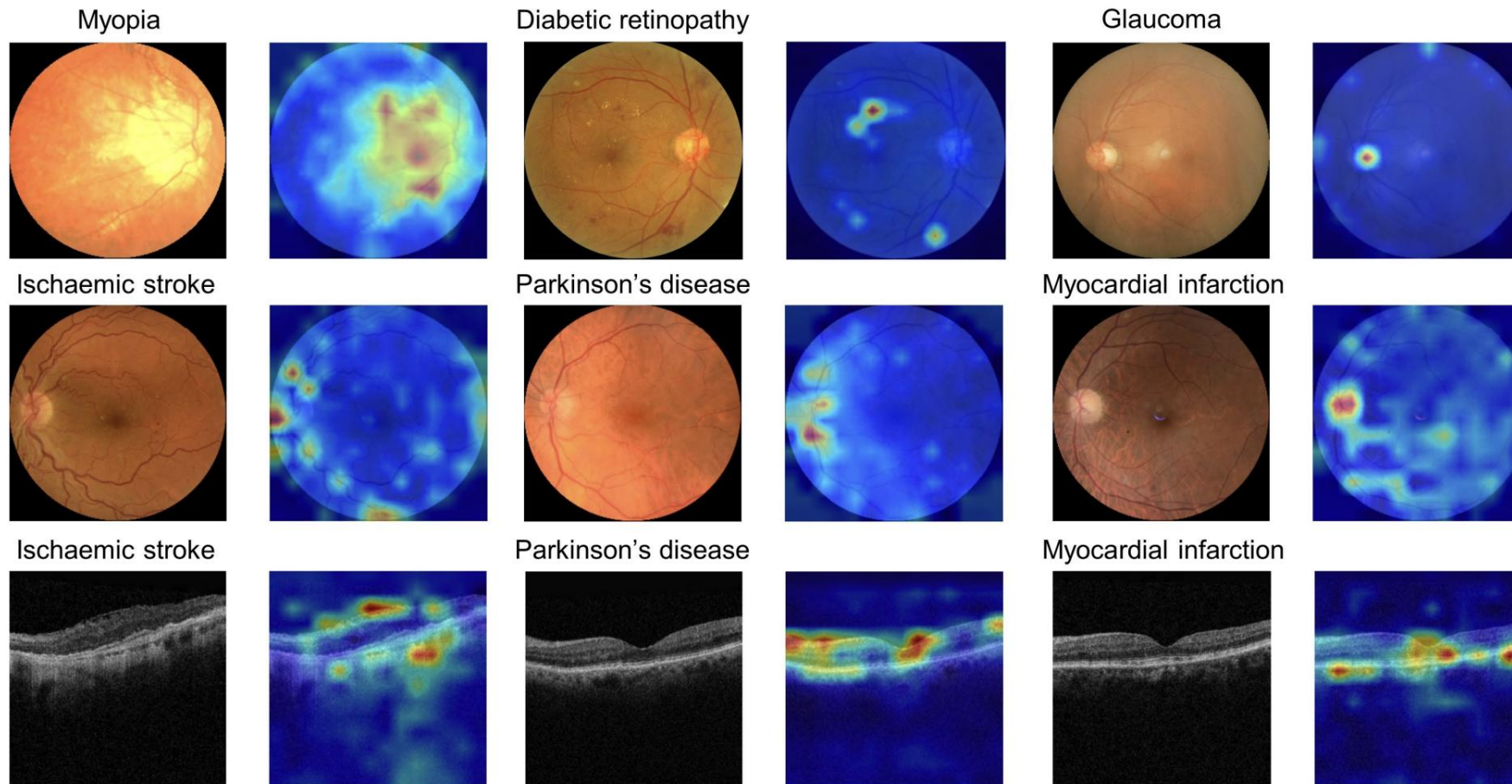
4. Deep learning for retinal image analysis



A foundation model for generalizable disease detection from retinal images

Results and Findings

Finding 6: RETFound can attend to the correct lesions on images.



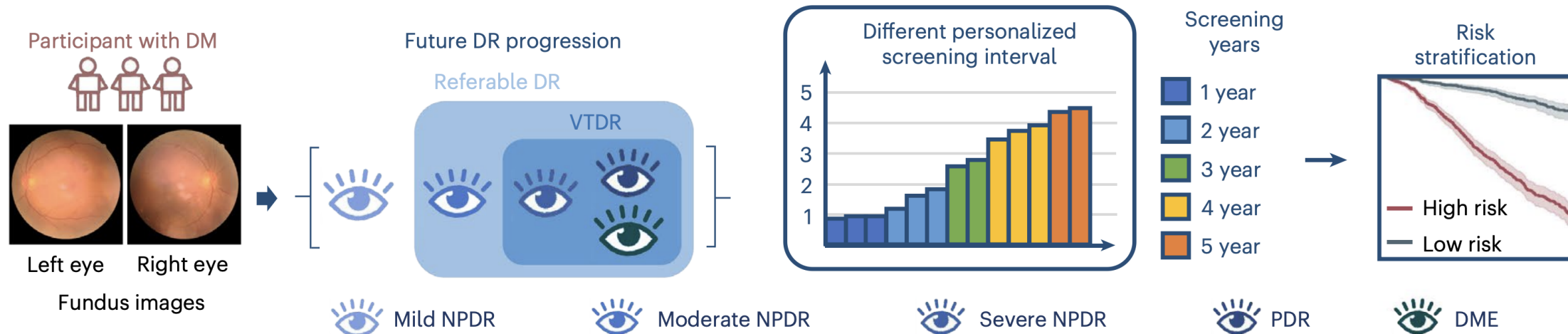
4. Deep learning for retinal image analysis



A deep learning system for predicting time to progression of diabetic retinopathy

Background

If a patient have diabetic retinopathy, it is important to monitor the disease and control it. This prevents the diabetic retinopathy goes worse. An important measure of this is predicting the progression time of DR, which means how long will the DR go worse again. If the time is short (progression risk is high), stronger intervention procedure needs to be made.



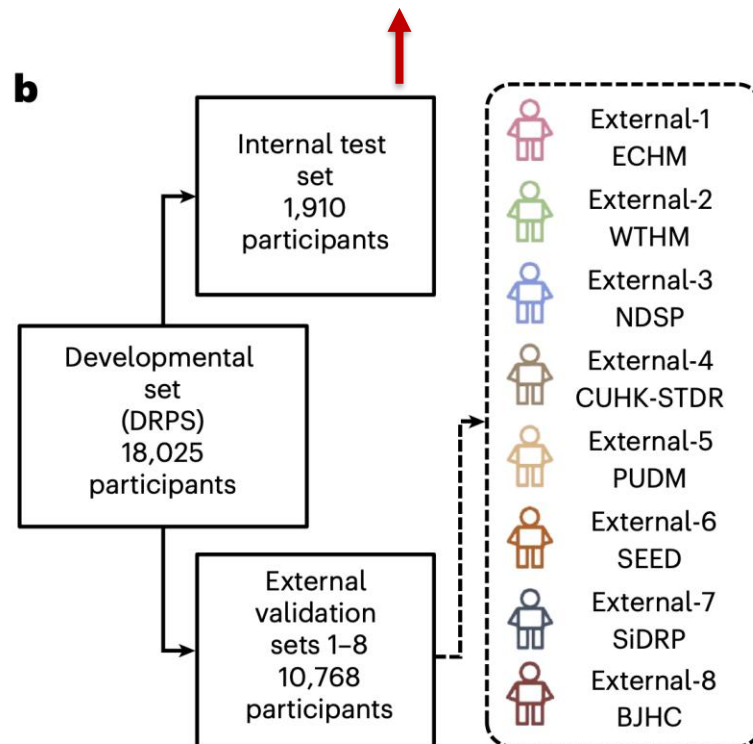
4. Deep learning for retinal image analysis



A deep learning system for predicting time to progression of diabetic retinopathy

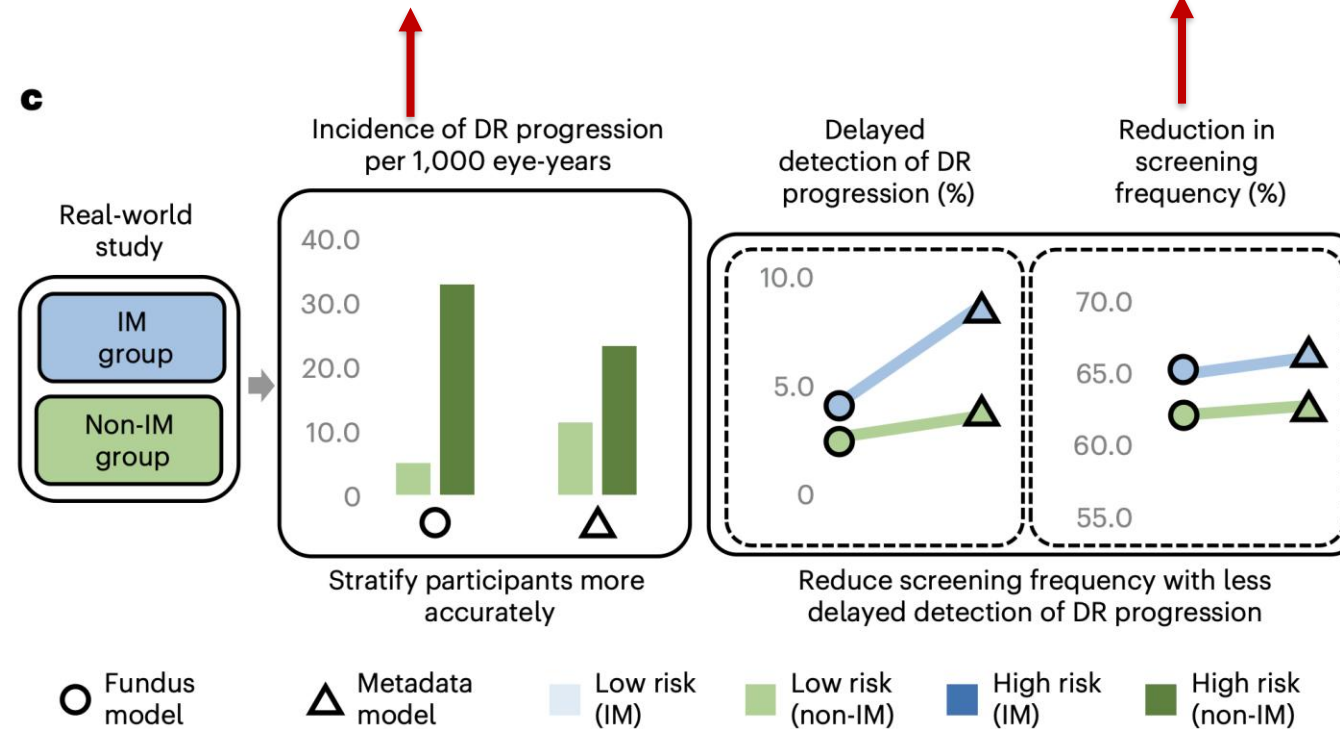
Data Collection

This large study collected a large amount of data from 8 hospitals.



Real world study using this model demonstrate that the model can effectively stratify low-risk and high-risk patient.

Patients can go to hospital less (lower amount of screening examination), but enjoys higher accuracy of progression risk warning.



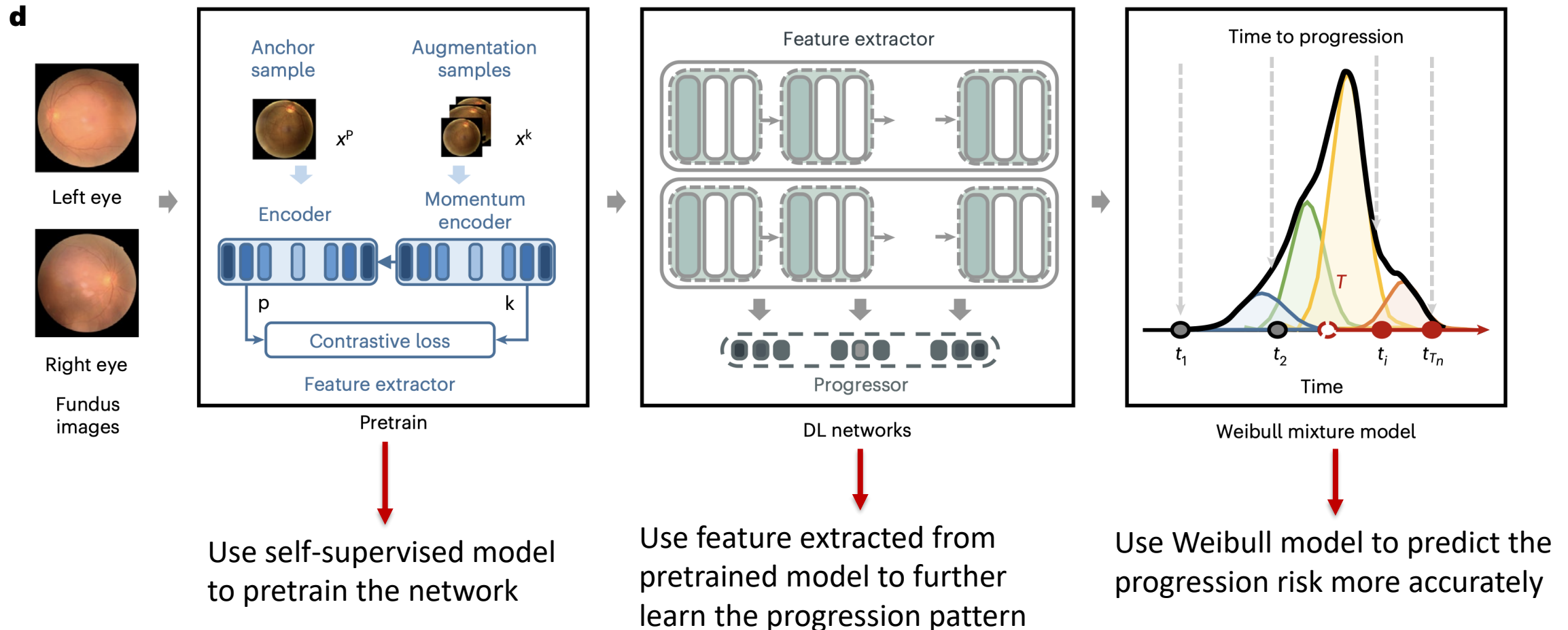
IM: Intelligent management using DeepDRPlus

4. Deep learning for retinal image analysis



A deep learning system for predicting time to progression of diabetic retinopathy

Methodology

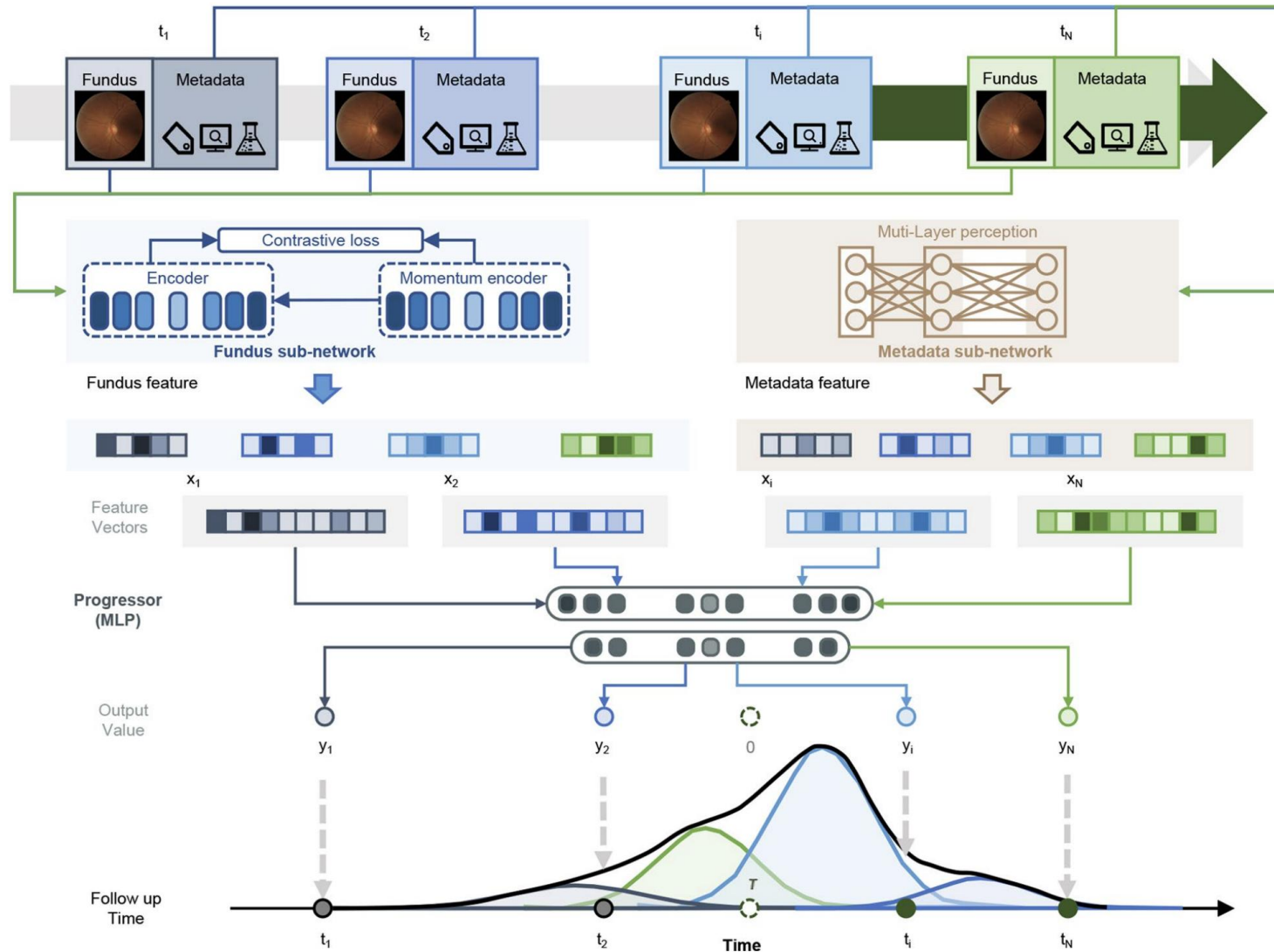


4. Deep learning for retinal image analysis



A deep learning system for predicting time to progression of diabetic retinopathy

Methodology

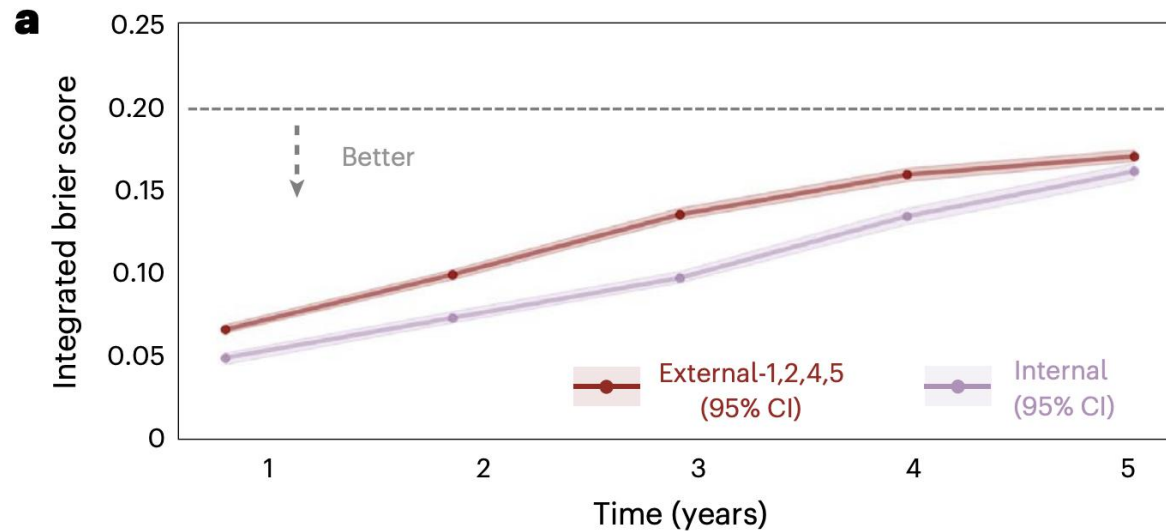


4. Deep learning for retinal image analysis

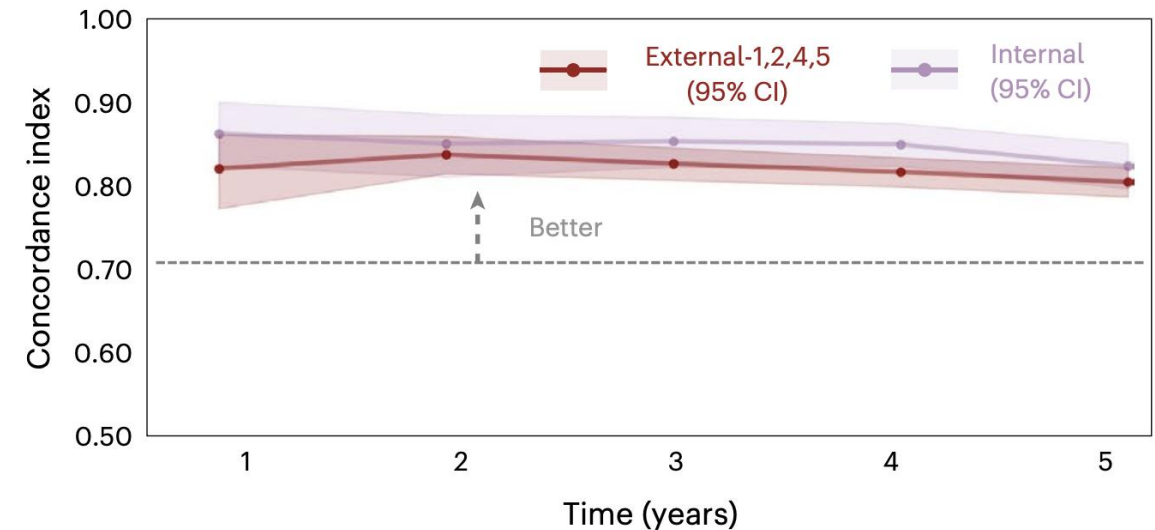


A deep learning system for predicting time to progression of diabetic retinopathy

Finding 1: DeepDR Plus can predicts time to DR progression accurately.



The lower, the more accurate the model predicts the progression risk.



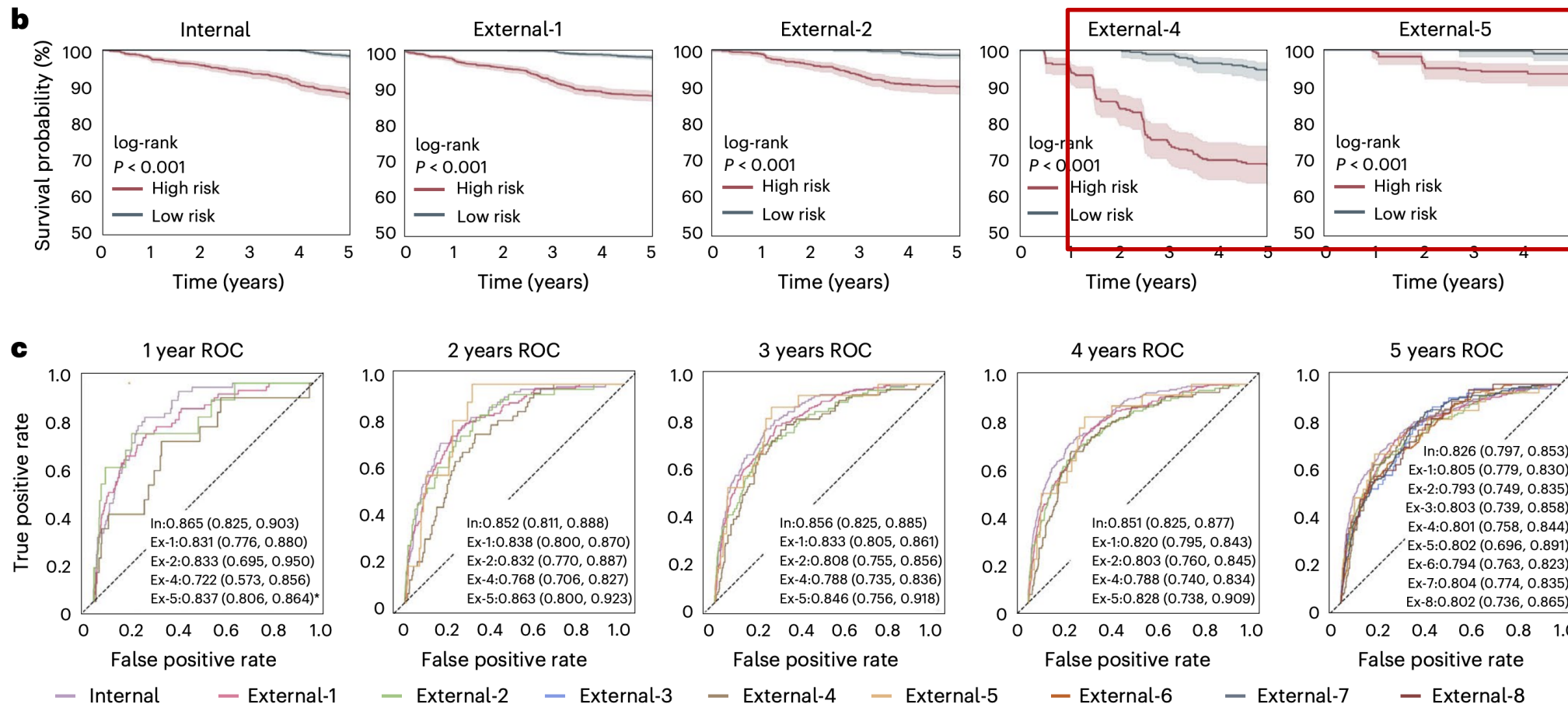
The higher, the more accurate the model predicts the progression risk.

4. Deep learning for retinal image analysis



A deep learning system for predicting time to progression of diabetic retinopathy

Finding 1: DeepDR Plus predicts time to DR progression accurately



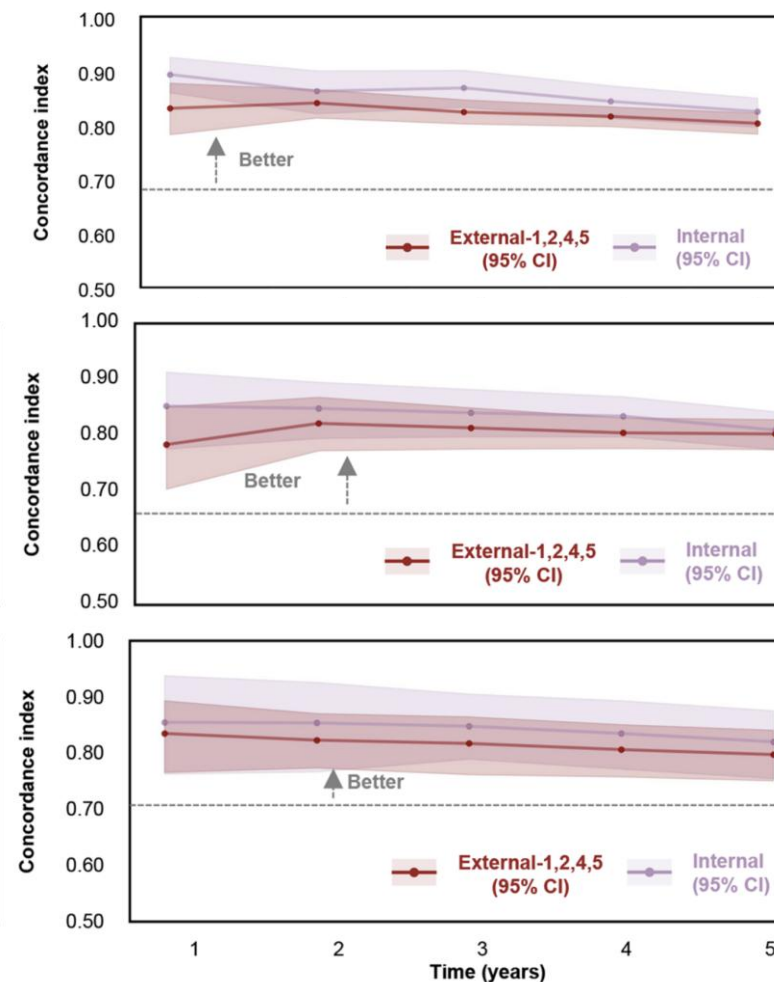
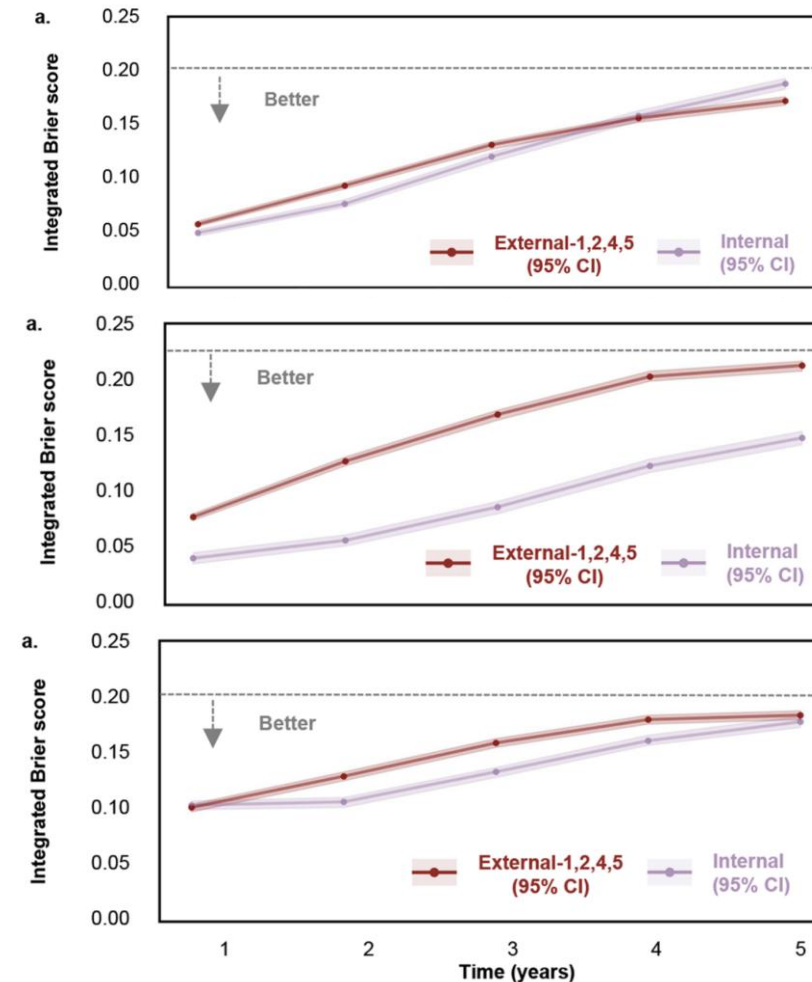
Kaplan Meier Curve:
The wider margin
between the two
lines, the clearer
stratification
between two risk
groups.

4. Deep learning for retinal image analysis



A deep learning system for predicting time to progression of diabetic retinopathy

Finding 2: DeepDR Plus accurately predicts time to progression in three fine-grained subgroups



The risk stratification from non-DR to DR.

The risk stratification from non-referable DR to referable DR (situation goes worse, needed to be treated by senior ophthalmologist)

The risk stratification from non-vision-threatening DR to vision-threatening DR

4. Deep learning for retinal image analysis



A deep learning system for predicting time to progression of diabetic retinopathy

Finding 3: Applying the DeepDR Plus system improves clinical outcomes.

Table 3 | Performance of personalized screening regime recommended by the metadata model or fundus model in DeepDR Plus, compared with fixed annual screening

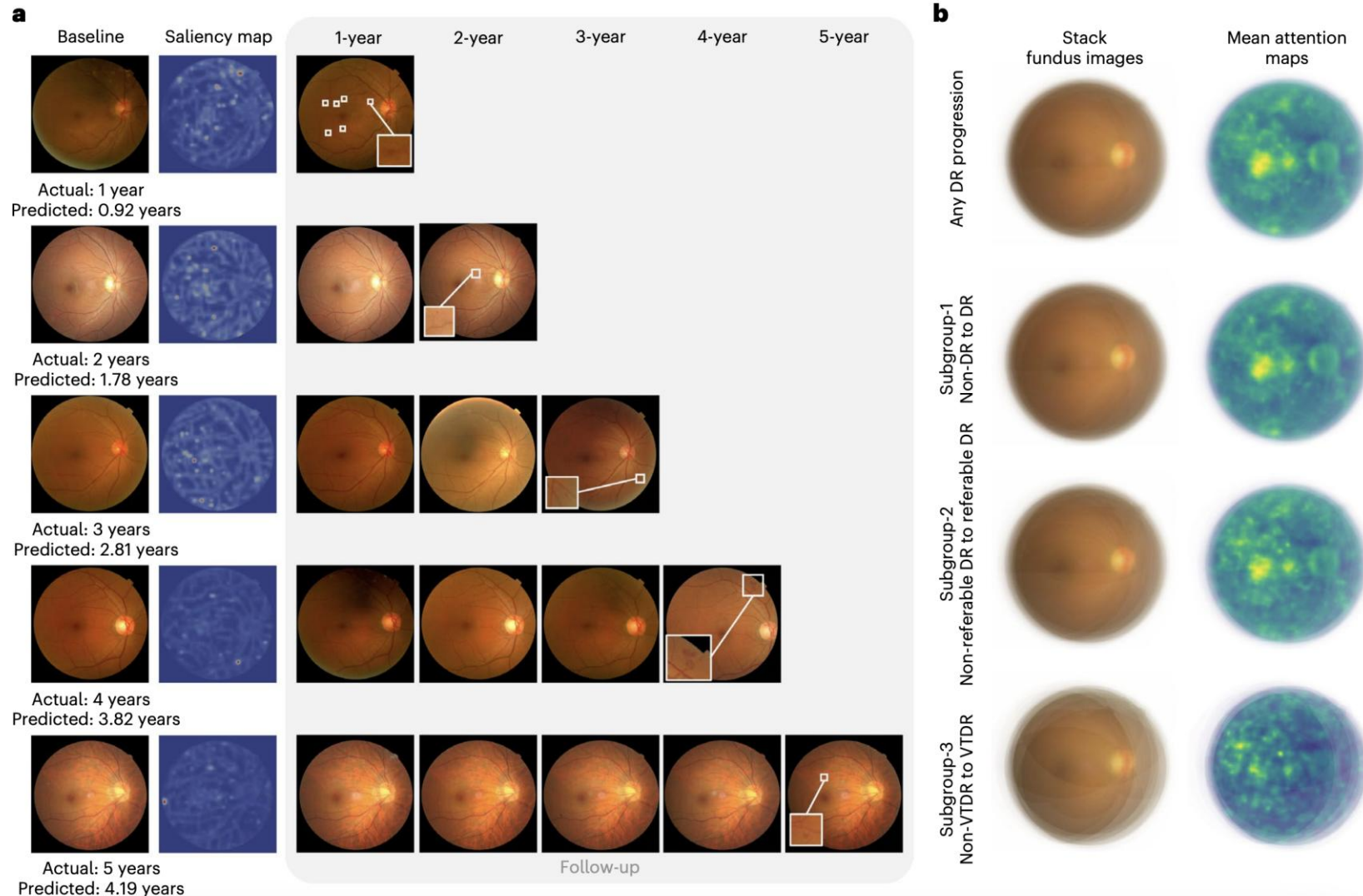
Group	Model	Average screening interval (months)	Reduction in screening frequency (%) ^a	Delayed detection of any DR progression (%) ^b	Delayed detection of progression to VTDR (%)
IM	Metadata	34.06	64.77	1.86	0.93
	Fundus	31.54	61.95	0.37	0.37
Non-IM	Metadata	35.32	66.02	6.01	0.97
	Fundus	32.11	62.63	1.28	0.12
IM and non-IM	Metadata	35.01	65.72	4.99	0.96
	Fundus	31.97	62.46	1.05	0.18

4. Deep learning for retinal image analysis



A deep learning system for predicting time to progression of diabetic retinopathy

Finding 4: DeepDR Plus enjoys explainability.



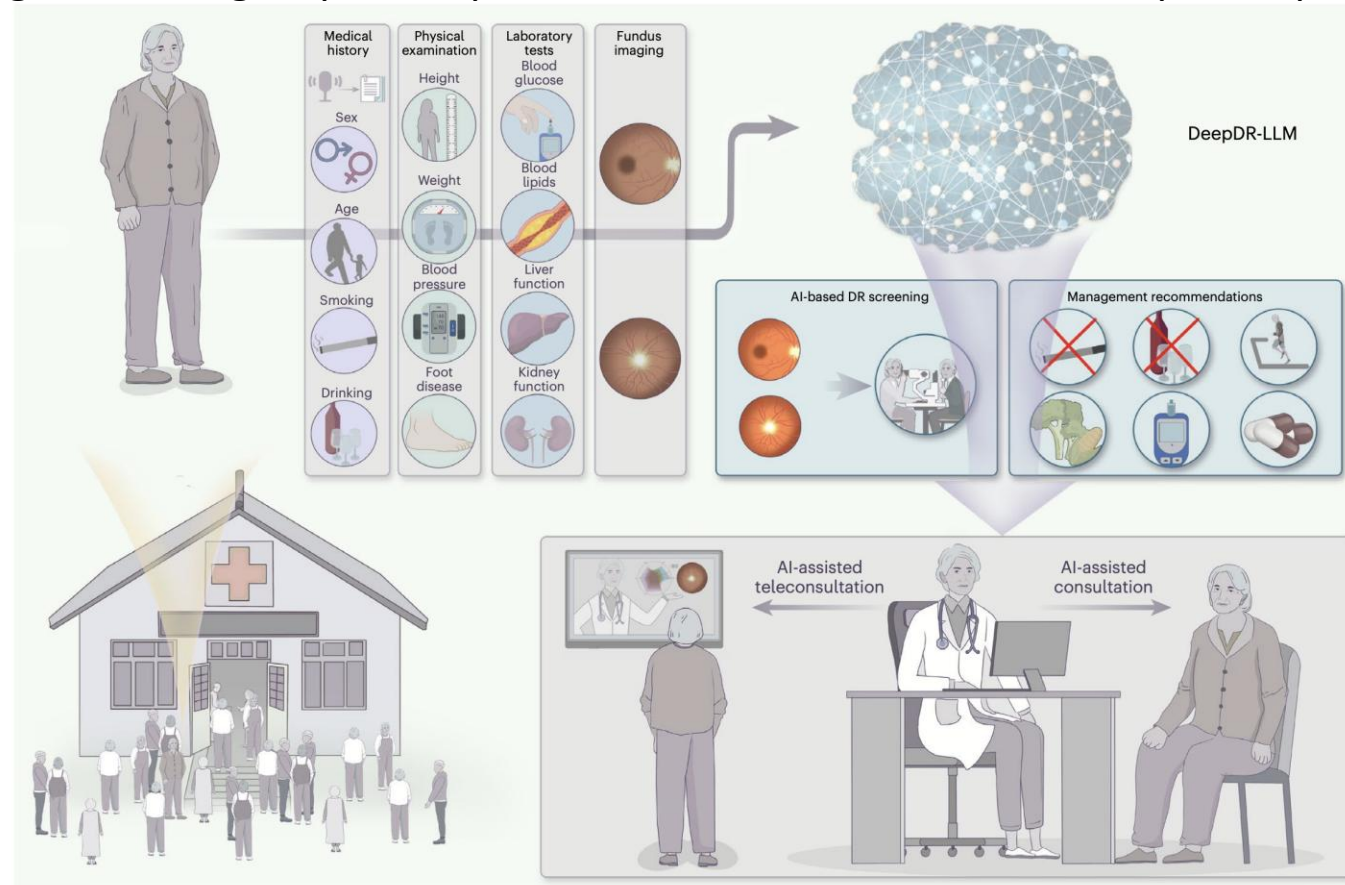
4. Deep learning for retinal image analysis



Integrated image-based deep learning and language models for primary diabetes care

Background

When a patient have diabetes and diabetes retinopathy, it is important to adjust their lifestyle to control the disease, especially in the early stage. How to give patient personalized advice based on their past experience and examination results?

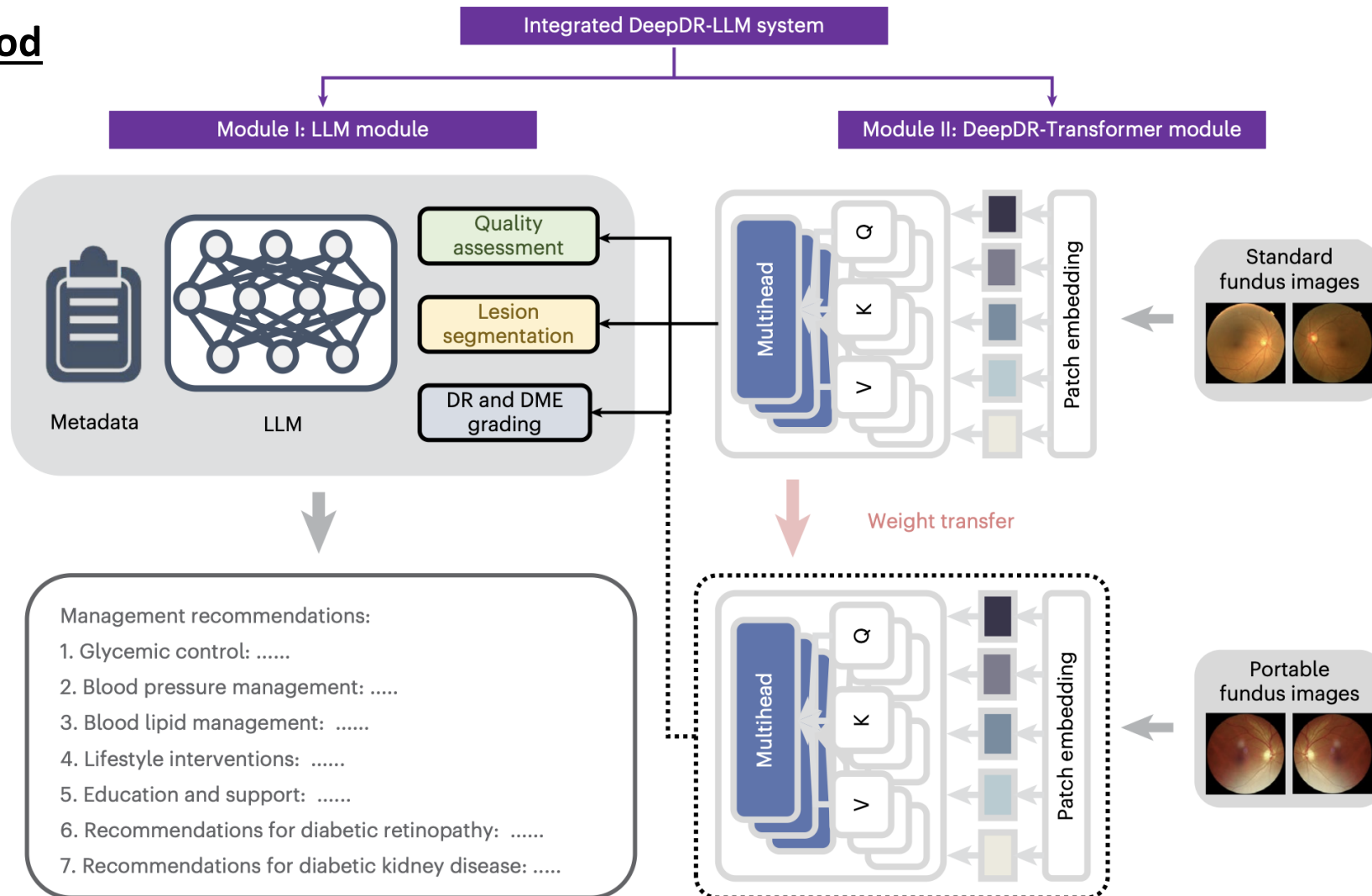


4. Deep learning for retinal image analysis



Integrated image-based deep learning and language models for primary diabetes care

Method



Module I: LLM can output basic advice for patients.

Module II: DeepDR-Transformer Module can assess the patient's fundus images from two sources and output quality assessment, lesion segmentation, and DR and DME grading to make the LLM model output more personalized advice.

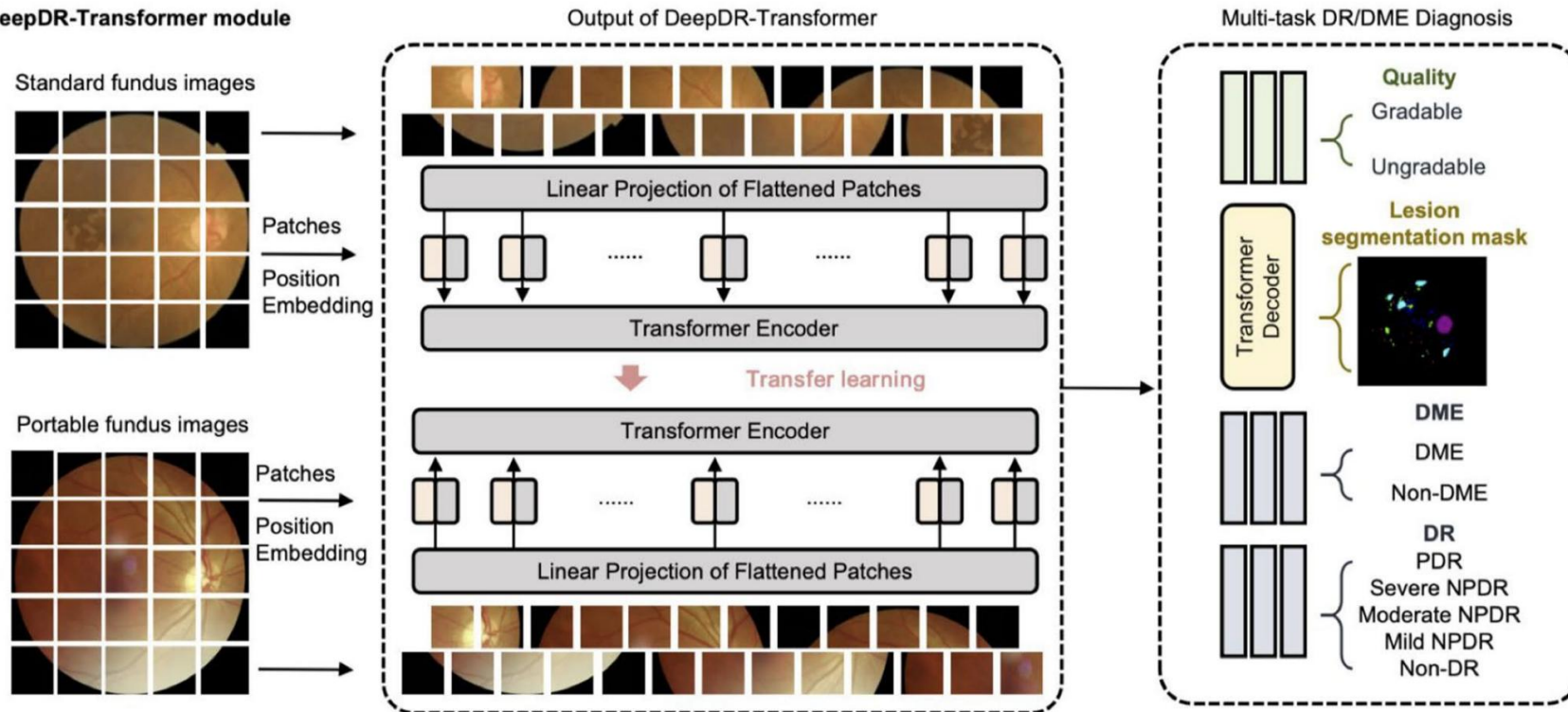
4. Deep learning for retinal image analysis



Integrated image-based deep learning and language models for primary diabetes care

Method

a. DeepDR-Transformer module



Transformer-based feature extraction module trained on standard fundus image, and use this feature to perform multi-task decoding.

The feature extractor module can be further fine-tuned on portable fundus images. A mobile device that you can take a fundus image at home.

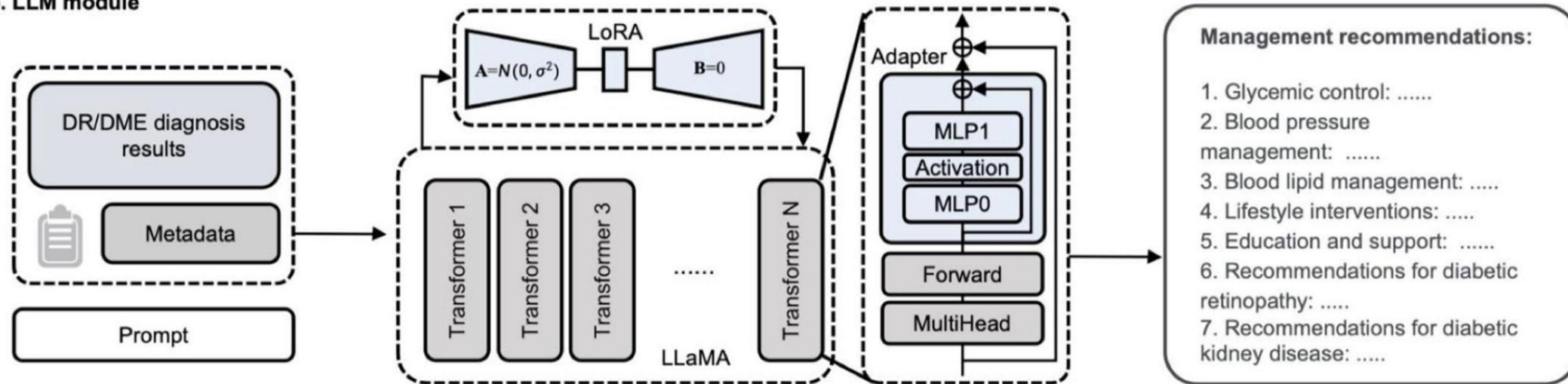
4. Deep learning for retinal image analysis



Integrated image-based deep learning and language models for primary diabetes care

Method

b. LLM module



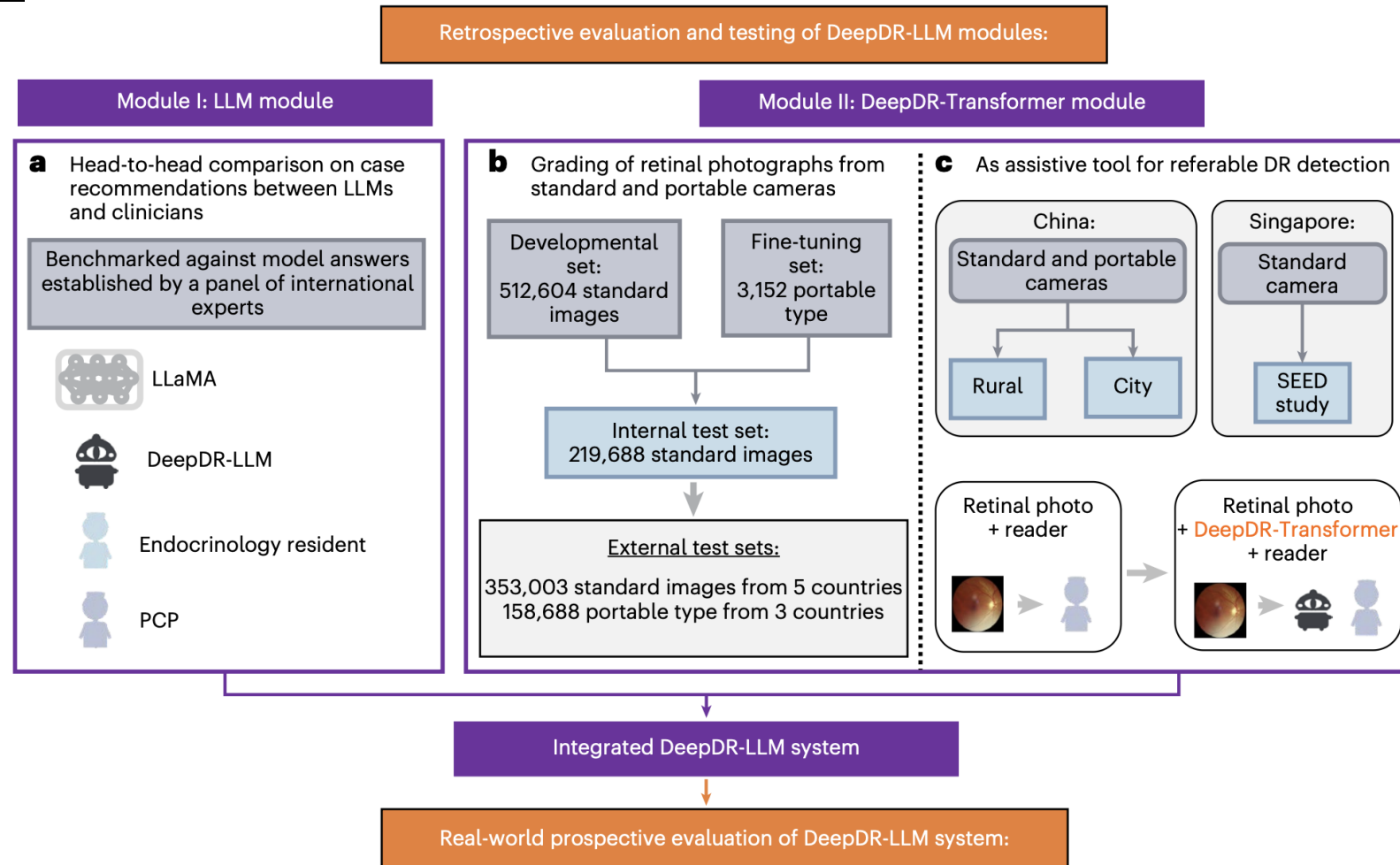
LLM then output the advice based on all available inputs.

4. Deep learning for retinal image analysis



Integrated image-based deep learning and language models for primary diabetes care

Method - Evaluation



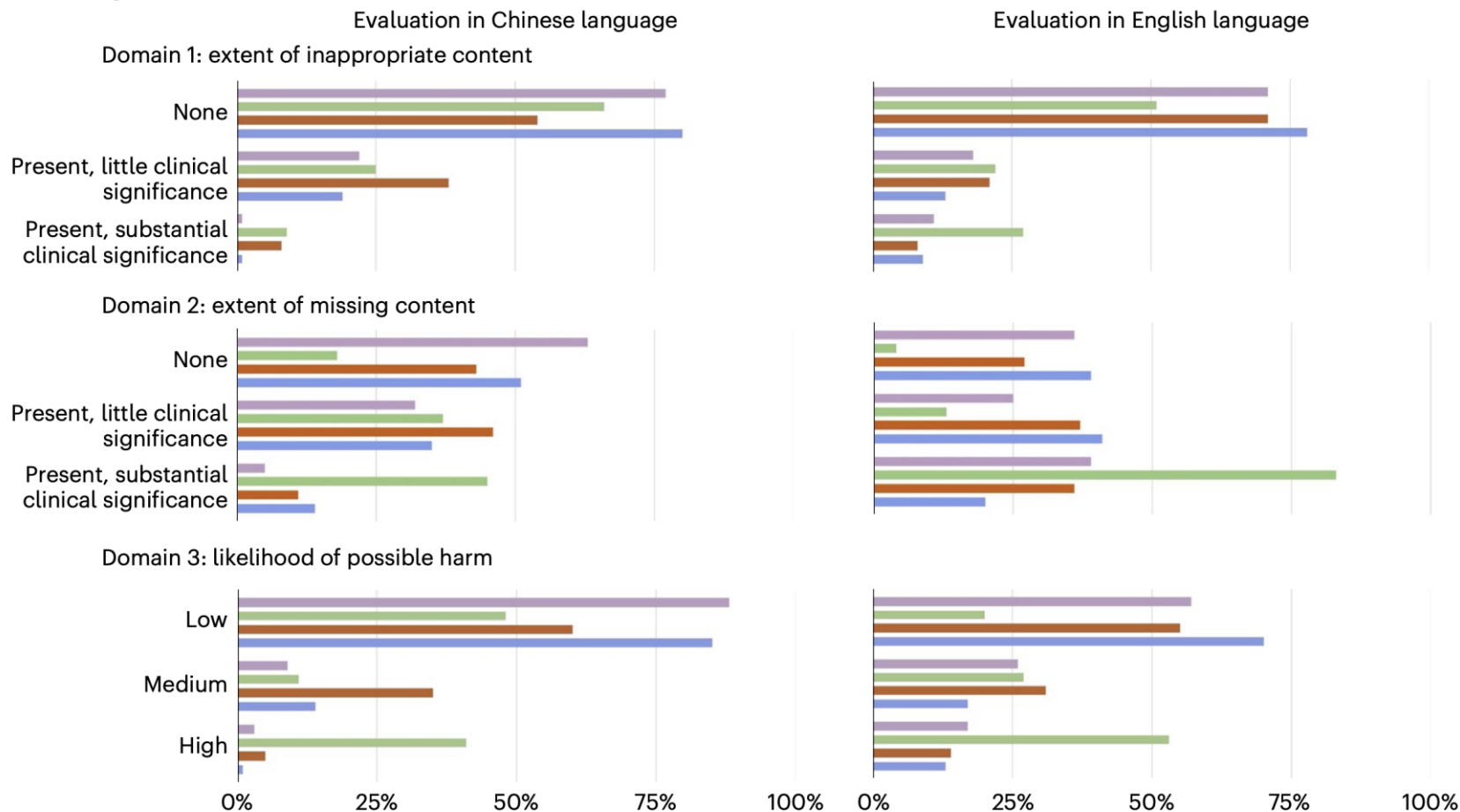
4. Deep learning for retinal image analysis



Integrated image-based deep learning and language models for primary diabetes care

Finding 1: The LLM's output is appropriate from three perspectives and two languages.

a Rating in three domains



4. Deep learning for retinal image analysis



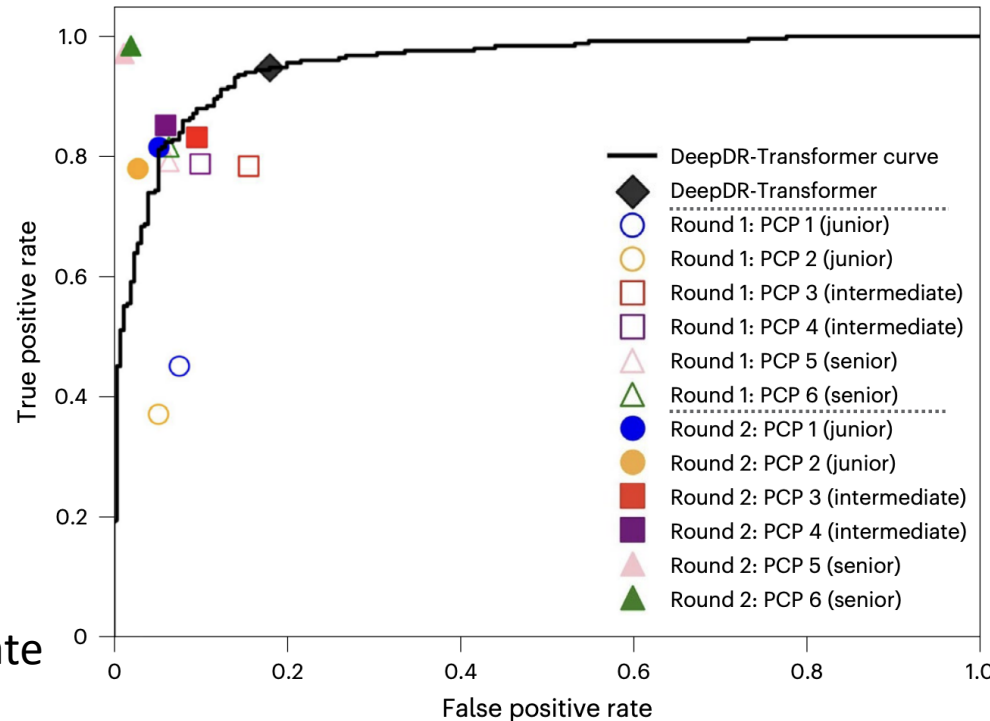
Integrated image-based deep learning and language models for primary diabetes care

Finding 2: DeepDR-Transformer as an assistive tool for clinicians.

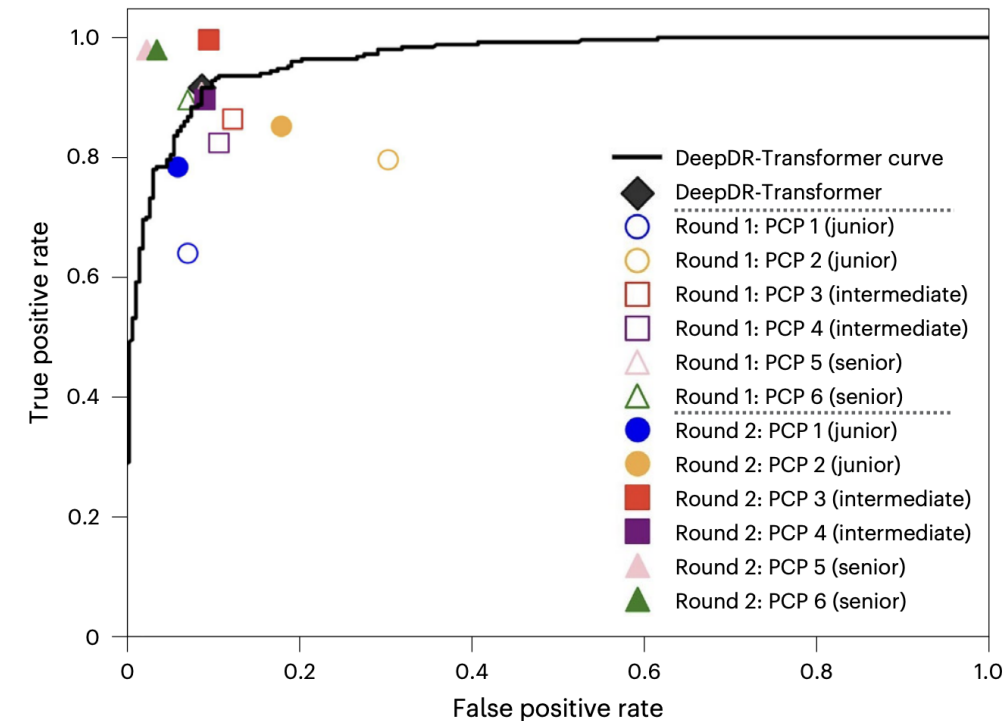
Round 1, AI and doctor grade the fundus image individually: DeepDR-Transformer performs better than junior and intermediate doctors.

Round 2: Doctor and AI grade the fundus image together: DeepDR-Transformer + doctor will provide even more accurate diagnosis.

a Standard fundus images graded by PCPs in the urban area



b Portable fundus images graded by PCPs in the urban area



4. Deep learning for retinal image analysis



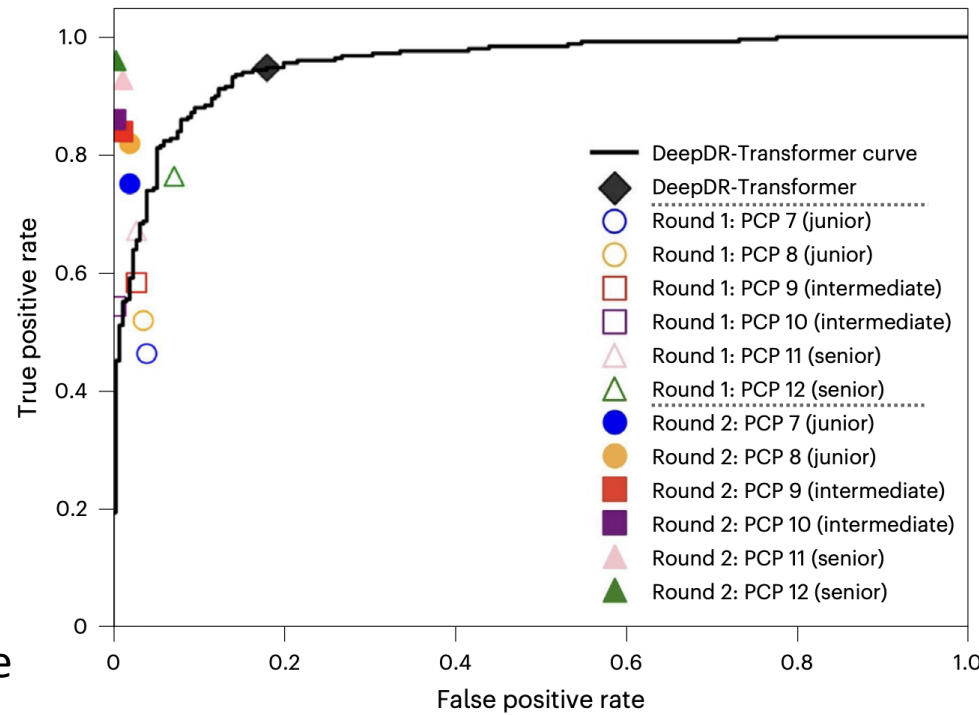
Integrated image-based deep learning and language models for primary diabetes care

Finding 2: DeepDR-Transformer as an assistive tool

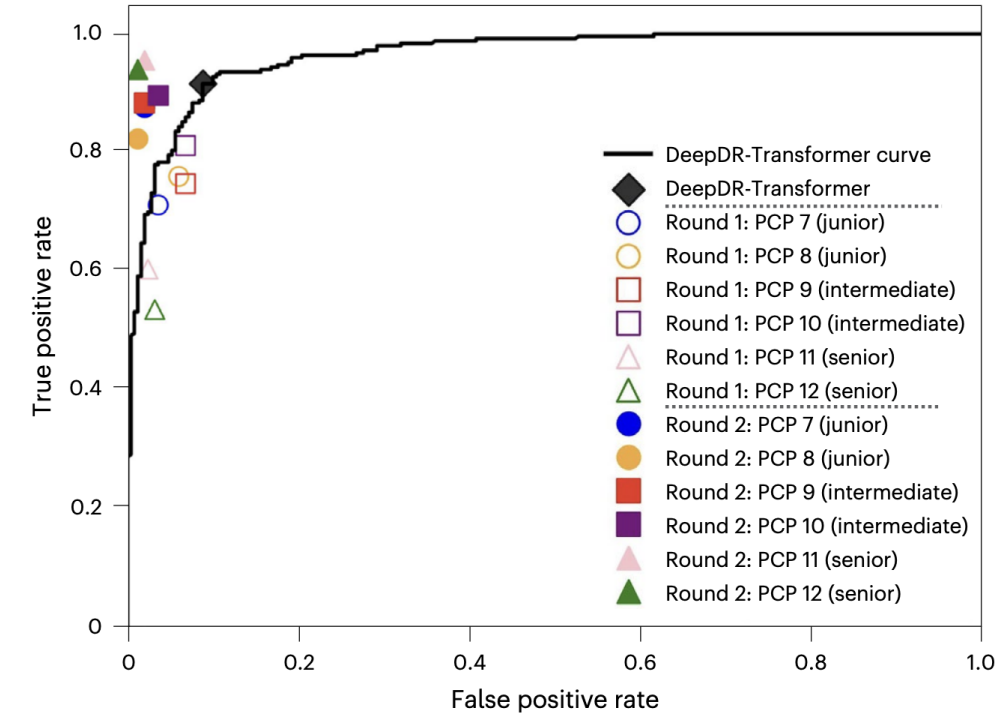
Round 1, AI and doctor grade the fundus image individually: DeepDR-Transformer performs better than junior and intermediate doctors.

Round 2: Doctor and AI grade the fundus image together: DeepDR-Transformer + doctor will provide even more accurate diagnosis.

c Standard fundus images graded by PCPs in the rural area



d Portable fundus images graded by PCPs in the rural area

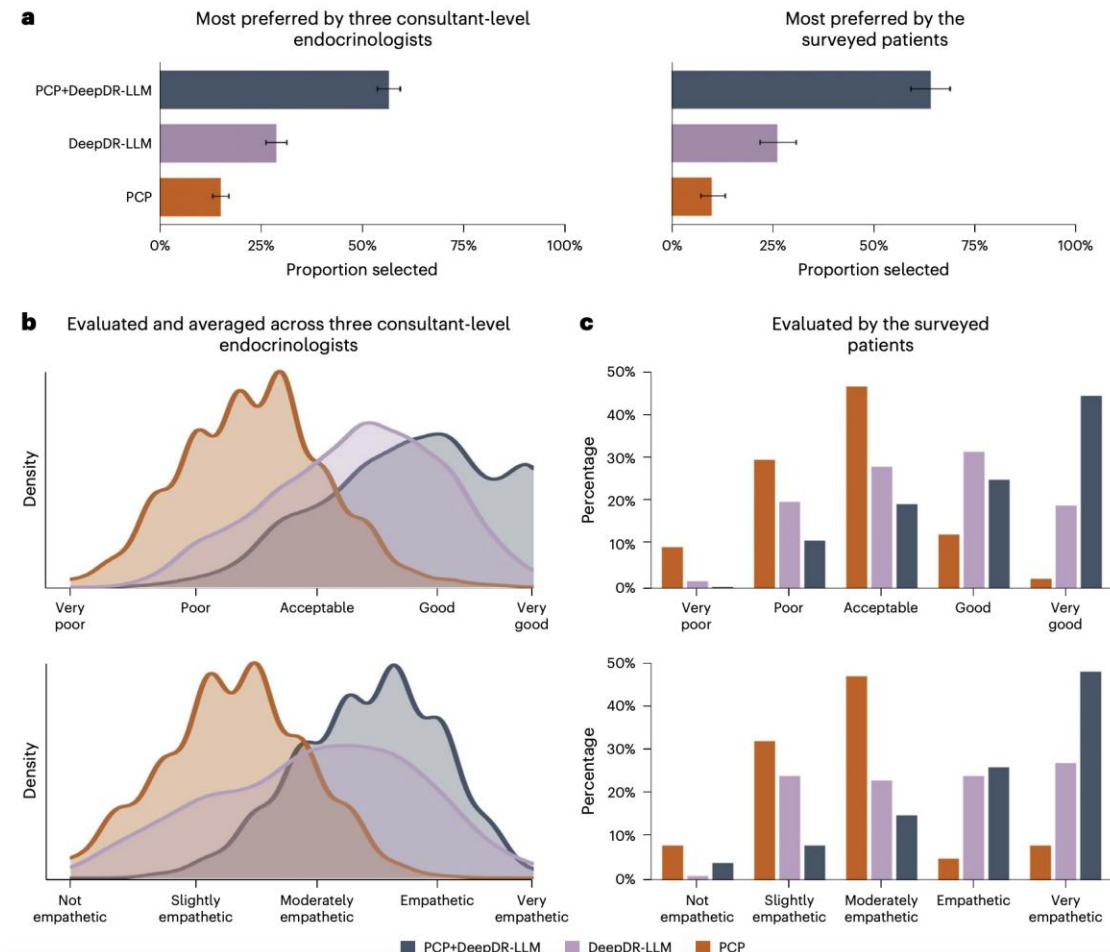
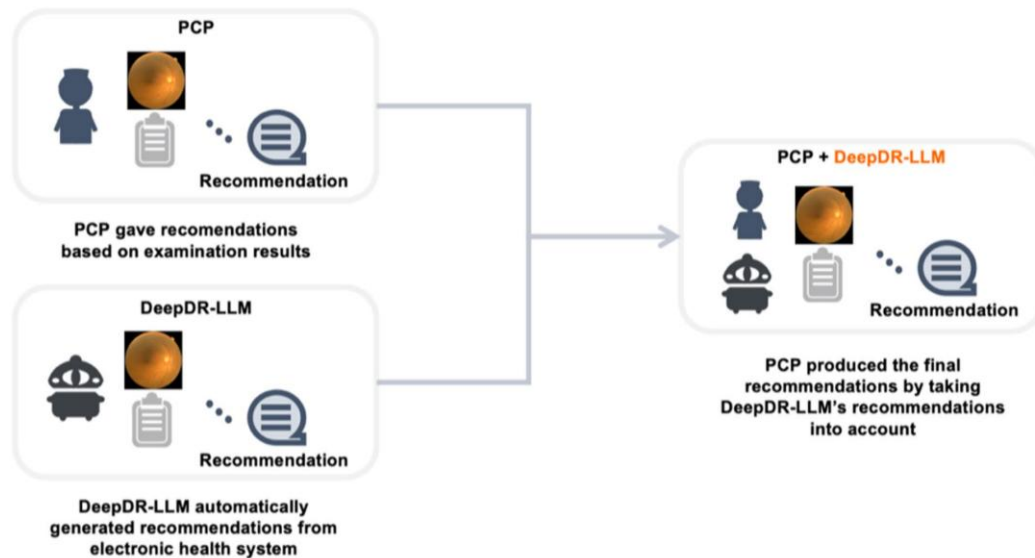


4. Deep learning for retinal image analysis



Integrated image-based deep learning and language models for primary diabetes care

Finding 3: Prospective real-world study of DeepDR-LLM. Quality and empathy ratings of the diabetes management recommendations by three consultant-level endocrinologists and 372 surveyed patients in the PCP+DeepDR-LLM arm.



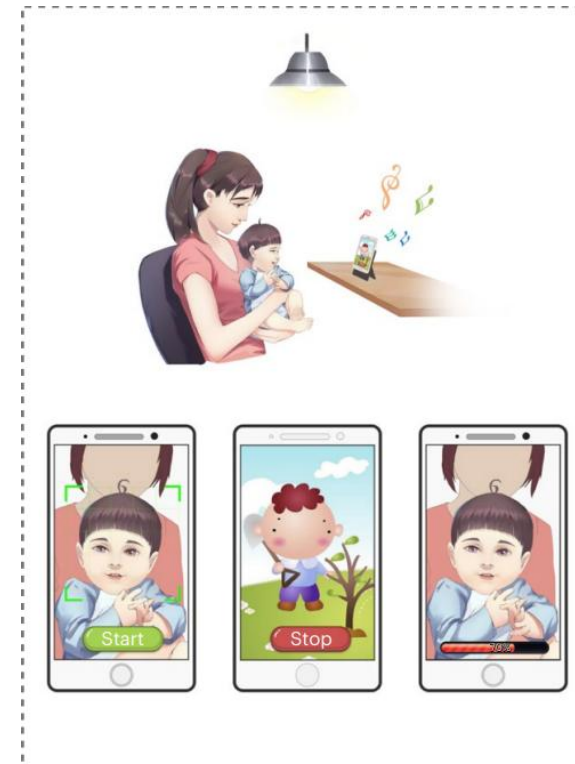
4. Deep learning for retinal image analysis



Early detection of visual impairment in young children using a smartphone-based deep learning system

Background

- Visual impairment is one of the most important causes of long-term disability in children worldwide and has a detrimental impact on education and socioeconomic achievements.
- Infancy and toddlerhood (early childhood) are critical periods for visual development, during which early detection and prompt treatment of ocular pathology can prevent irreversible visual loss.
- However, babies can not fully cooperate in the examination.



4. Deep learning for retinal image analysis



Early detection of visual impairment in young children using a smartphone-based deep learning system

Method How can a smartphone screen potential visual impairment risk for babies?

1. Choose a quiet environment with appropriate lighting and avoid interference



2. Keep the smartphone in a stable position



3. Use the guiding box to adjust the position and the distance from the child to the smartphone



4. Watch the cartoon-like video



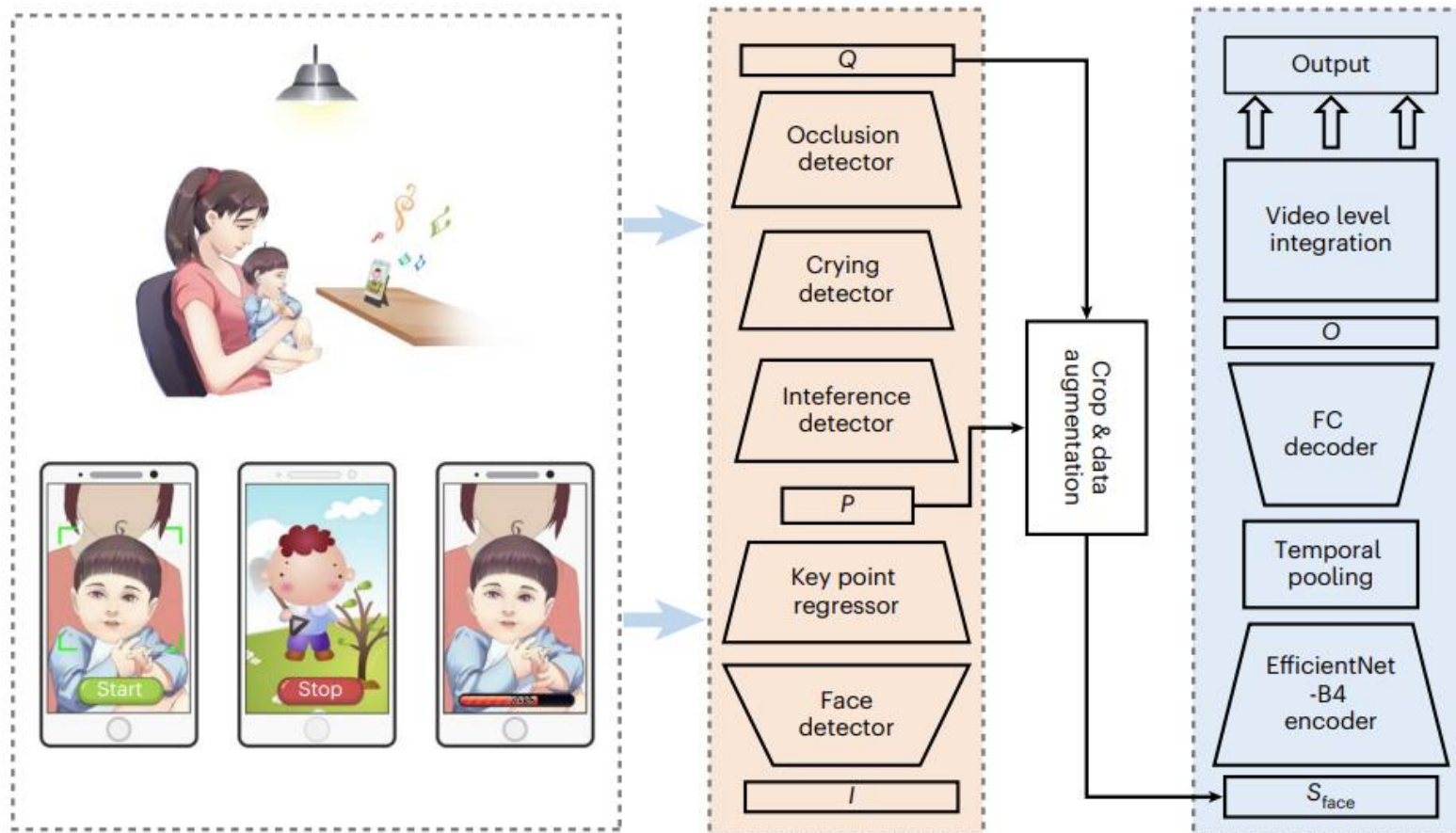
Use WeChat, Search for Ai宝睛灵 to have a look at the demo.

4. Deep learning for retinal image analysis



Early detection of visual impairment in young children using a smartphone-based deep learning system

Method Then, the collected videos will be sent to AI for a series of analysis.



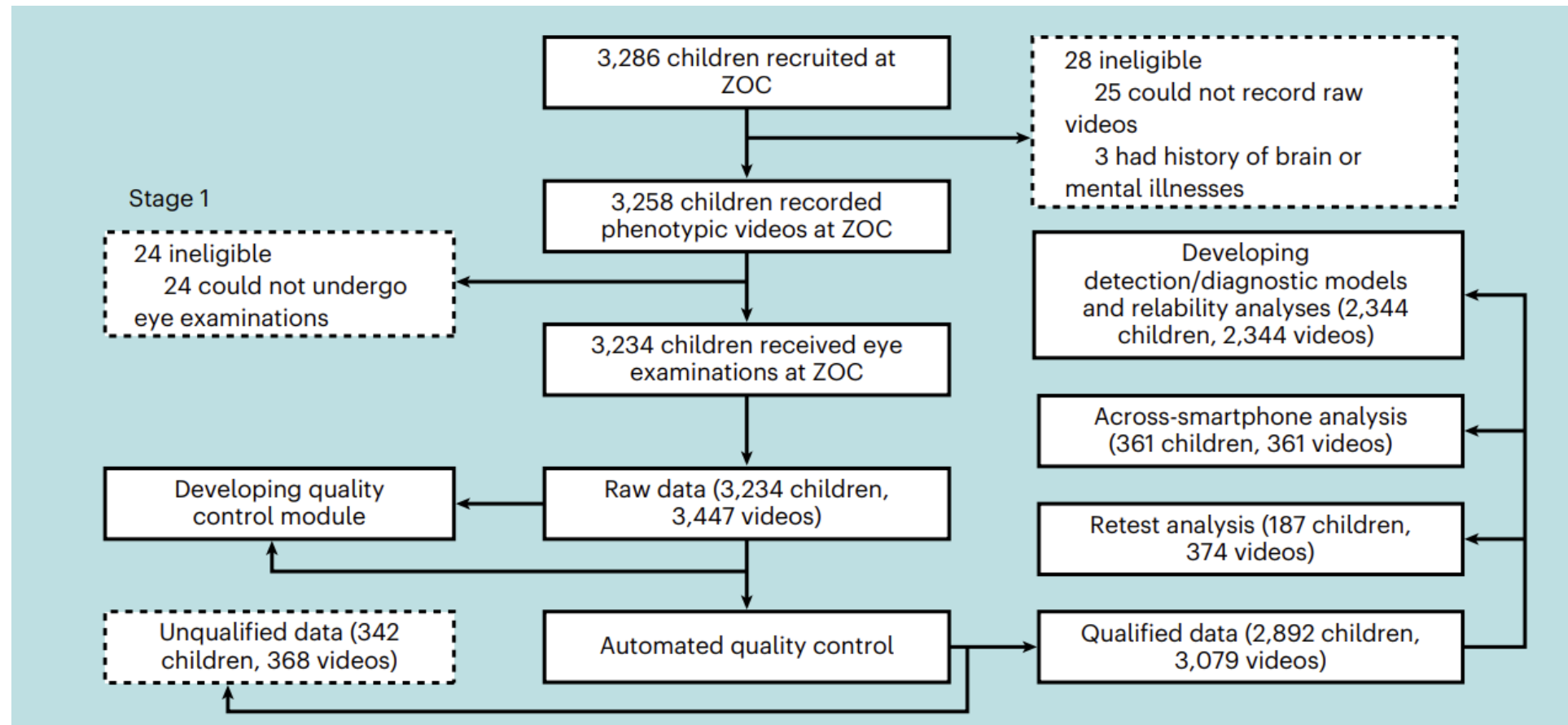
- Are there any occlusion?
- Is the baby crying?
- Did any interference happened?
- Where are the face key points?
- Where is the baby's face?

4. Deep learning for retinal image analysis



Early detection of visual impairment in young children using a smartphone-based deep learning system

Method 2344 children data are collected for developing models.

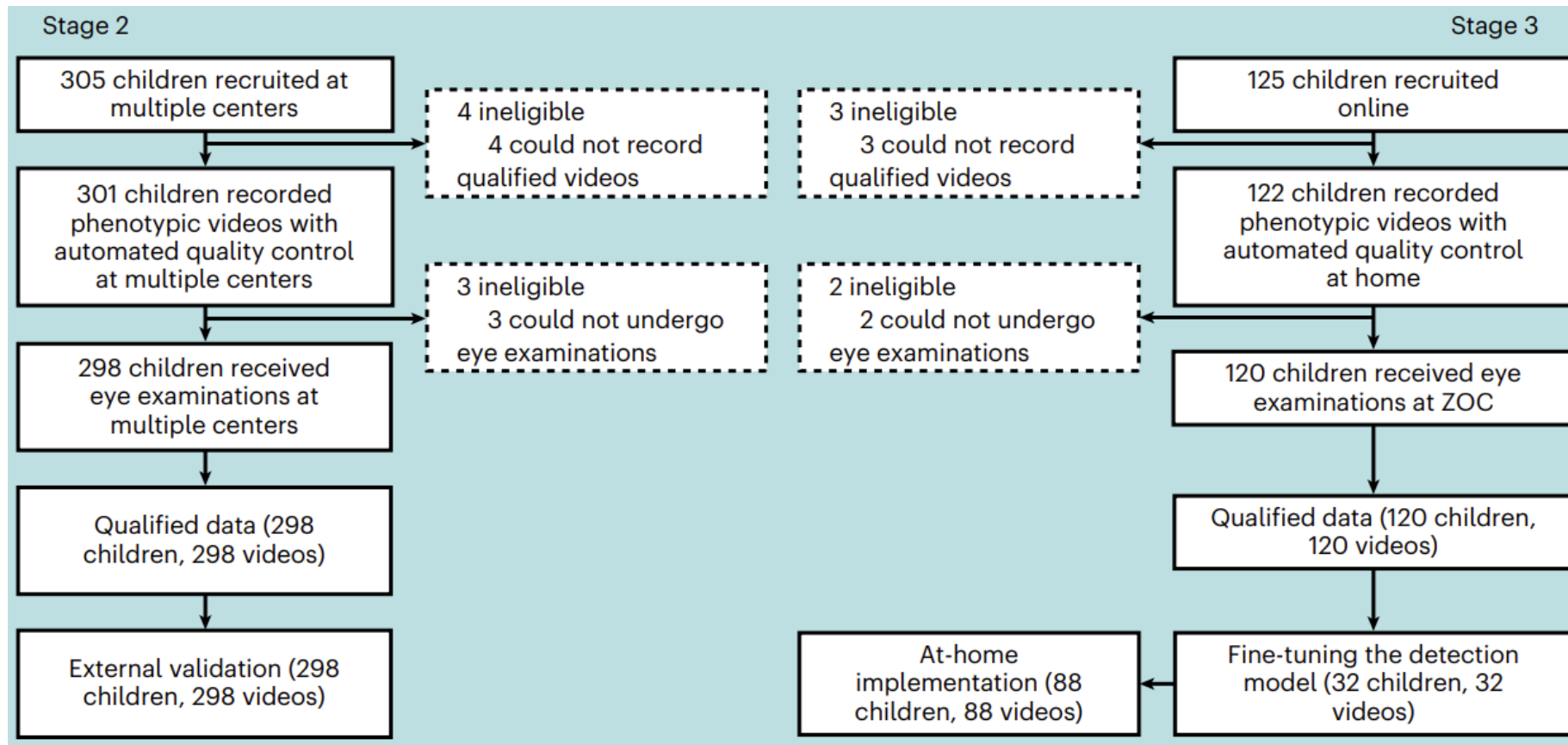


4. Deep learning for retinal image analysis



Early detection of visual impairment in young children using a smartphone-based deep learning system

Method 305 children are recruited for validation, and 125 children are “remote volunteers”

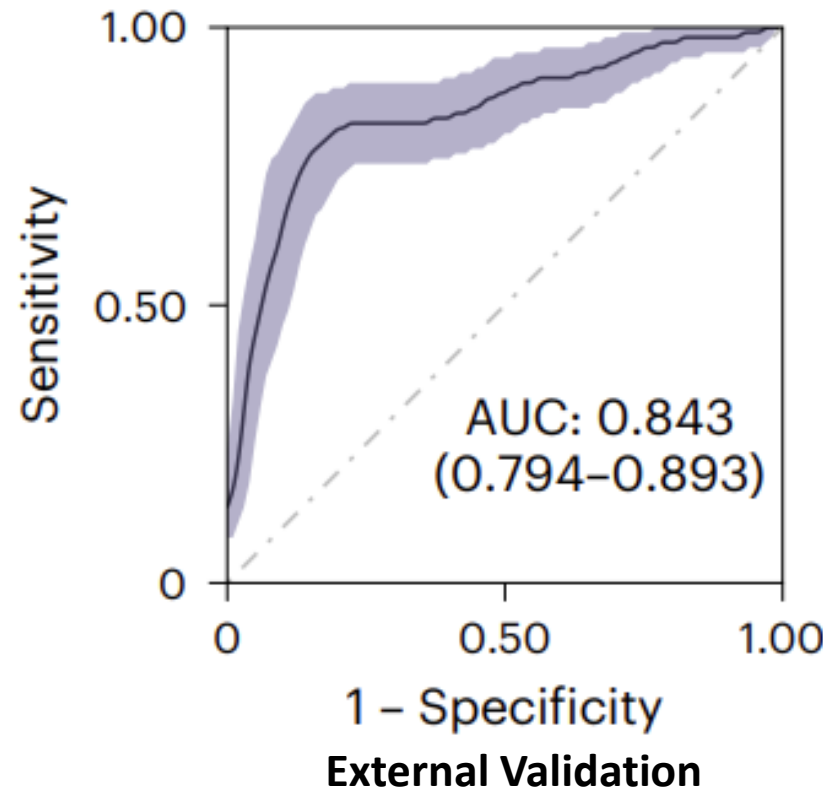
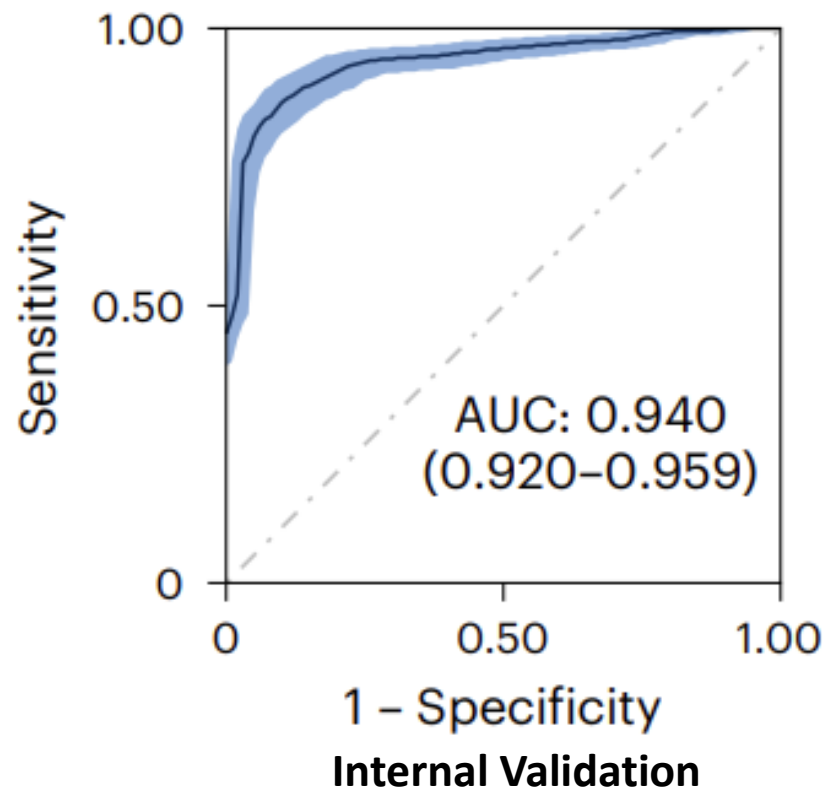


4. Deep learning for retinal image analysis



Early detection of visual impairment in young children using a smartphone-based deep learning system

Finding 1: mHealth AIS, the detection model, can accurately detect babies with potential visual impairment in real clinical settings with trained volunteers. Volunteers can guide the baby and patients to perform standard recording procedures.



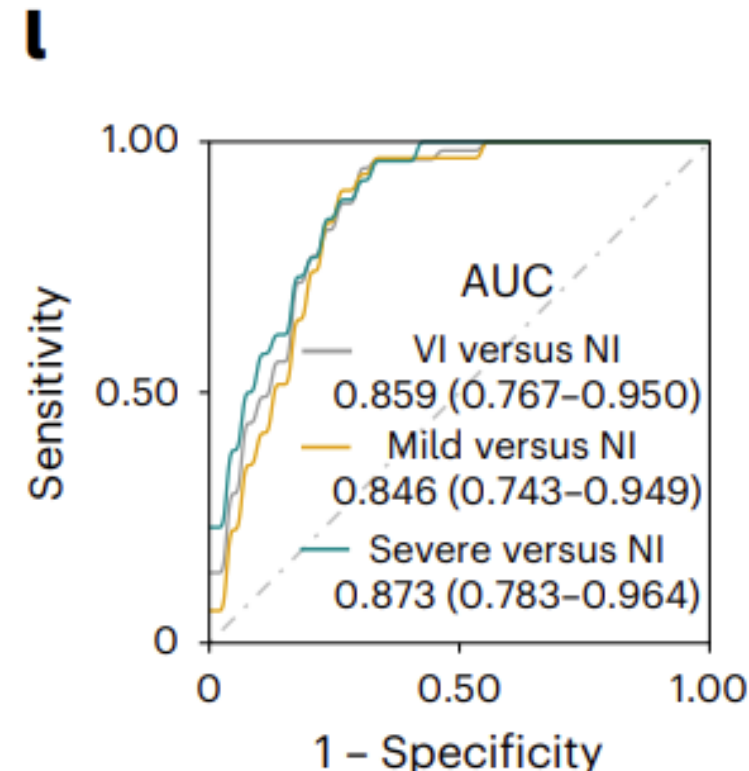
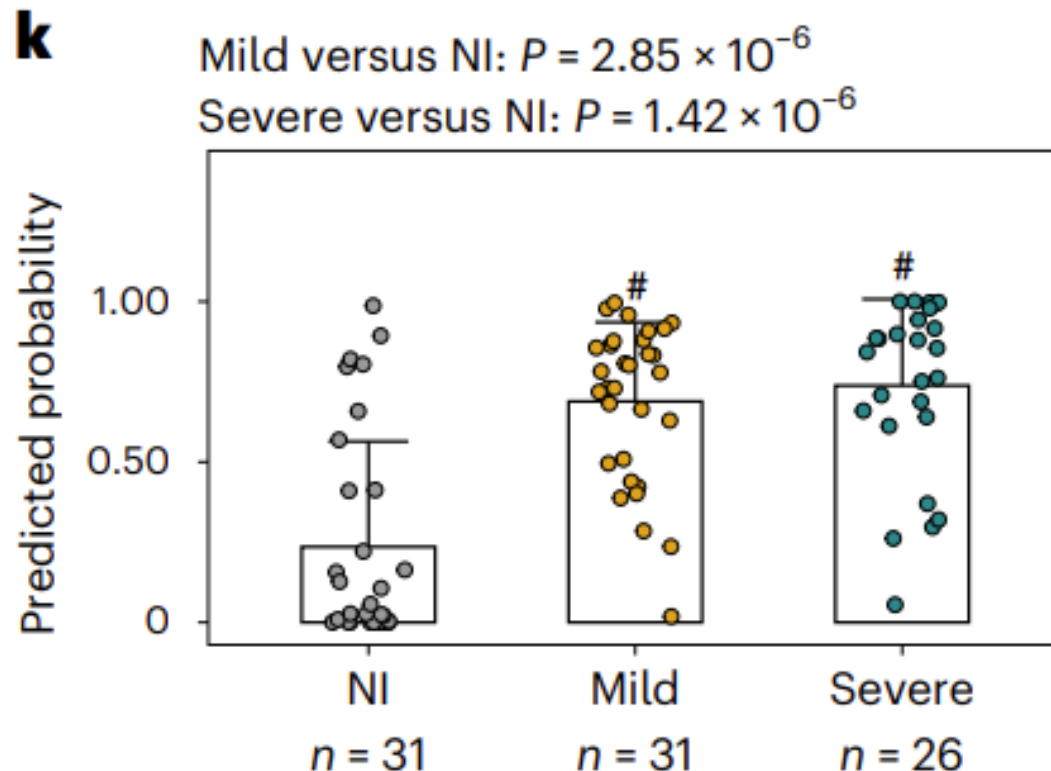
Distinguish from not Vision Impaired to Vision Impaired

4. Deep learning for retinal image analysis



Early detection of visual impairment in young children using a smartphone-based deep learning system

Finding 2: mHealth AIS, the detection model, can accurately detect babies with potential visual impairment even simply using the app at home by patients.



NI: Not Impaired
VI: Vision Impaired
Mild: Mild Impair
Severe: Severe Impair

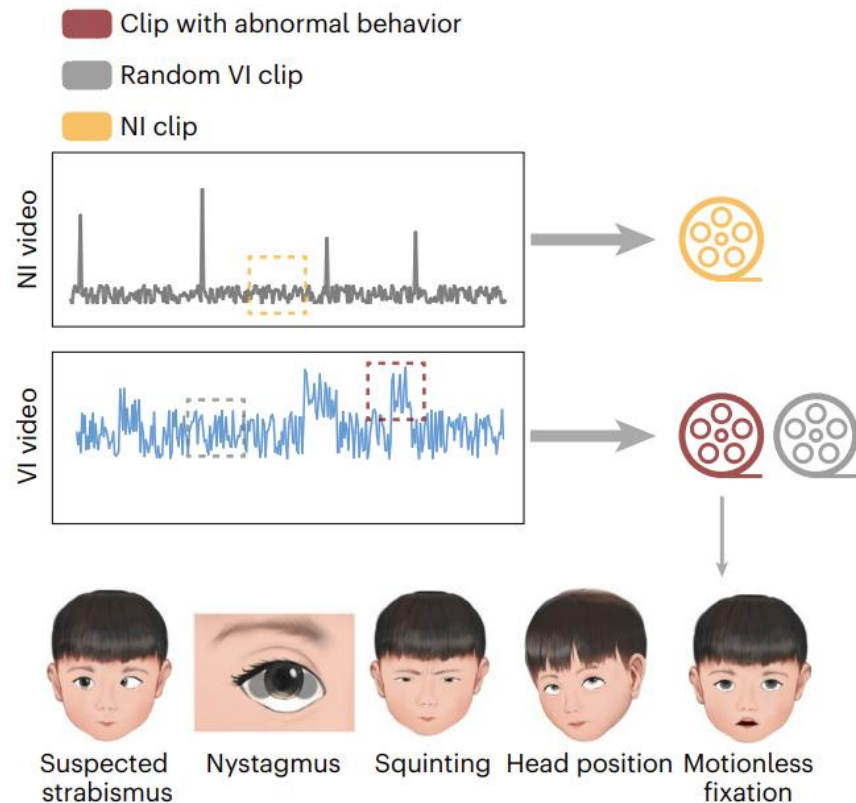
4. Deep learning for retinal image analysis



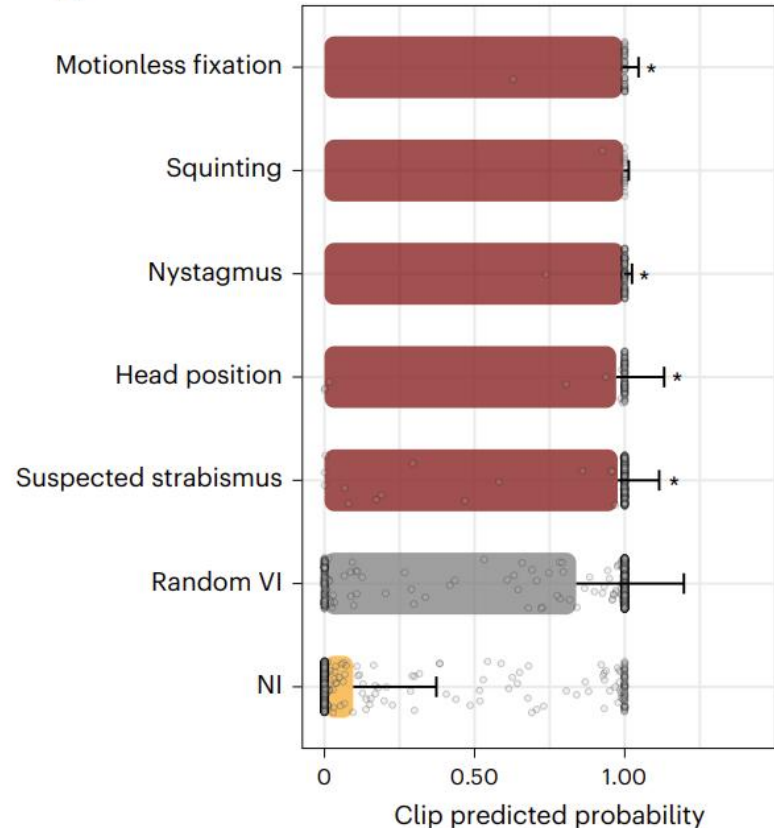
Early detection of visual impairment in young children using a smartphone-based deep learning system

Finding 3: mHealth AIS can analyze abnormal behaviors in vision-impaired children.

d



e



VI children usually have:

- motionless fixation,
- squinting,
- nystagmus,
- abnormal head position,
- suspected strabismus

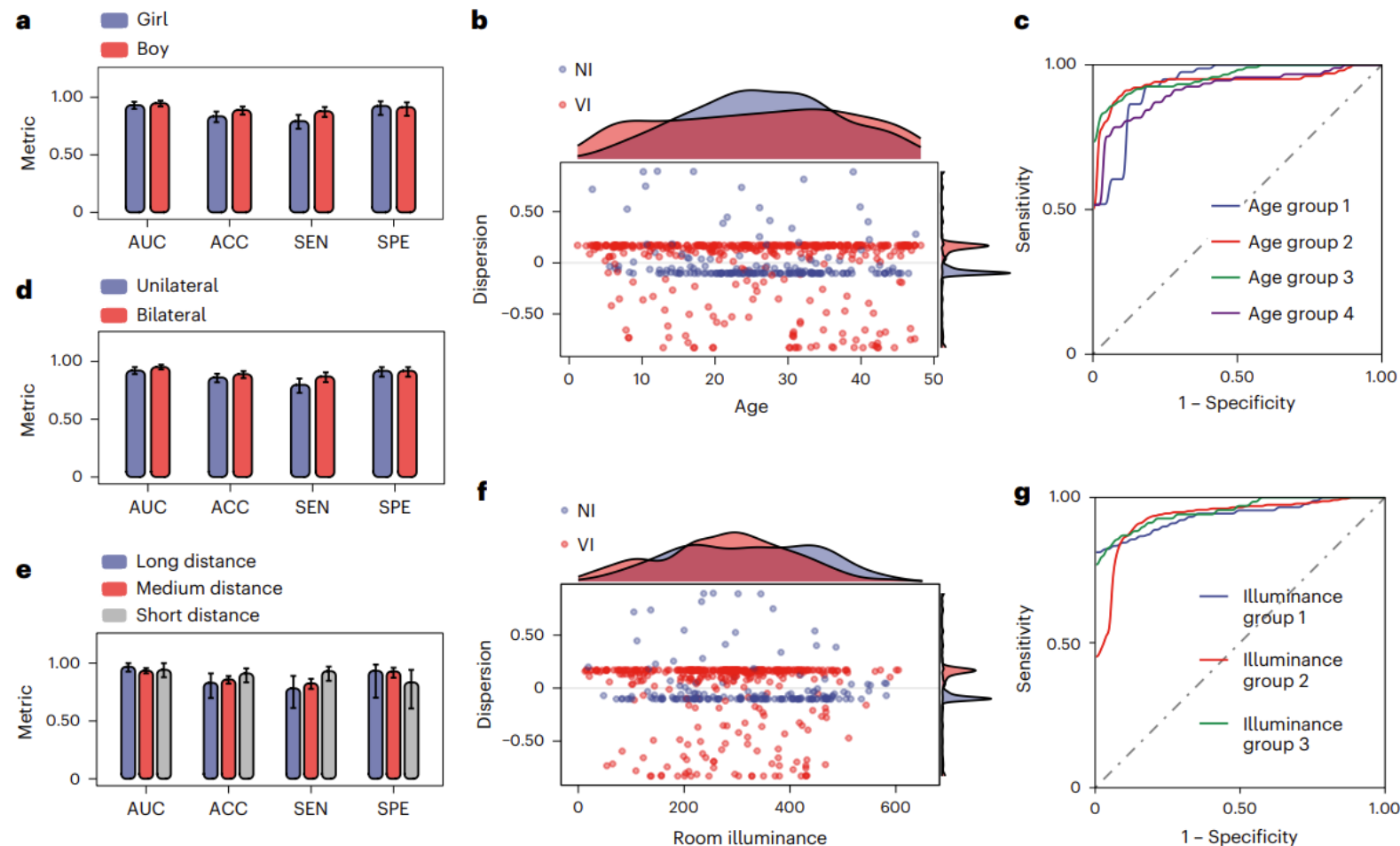
When watching the video.

4. Deep learning for retinal image analysis



Early detection of visual impairment in young children using a smartphone-based deep learning system

Finding 4: mHealth AIS has stable performance across different age, gender, recording environment, record distance.



4. Deep learning for retinal image analysis



Accurate prediction of disease-risk factors from volumetric medical scans by a deep vision model pre-trained with 2D scans

Background

- Many diagnoses rely on volumetric biomedical imaging (for instance, 3D OCT or 3D MRI scans), and transfer learning is not directly applicable, as in contrast to the 2D domain there is no large annotated 'ImageNet-like' dataset of structured 3D scans.
- Moreover, annotating 3D biomedical images is far more labour prohibitive than 2D images. For example, a 3D OCT scan that is composed of 97 2D frames (usually referred to as B-scans) normally requires a 5–10 min inspection of a highly trained clinical retina specialist to detect retinal-disease biomarkers, such as the volume of a drusen lesion.
- Therefore, considering the resources typically devoted to such a task, it is practically infeasible to annotate 100,000 (or more) volumes to eliminate the necessity of supervised transfer learning.

Solution: Use two dimensional pretrained model to extract feature slice by slice, and then aggregate the 3D feature!

4. Deep learning for retinal image analysis



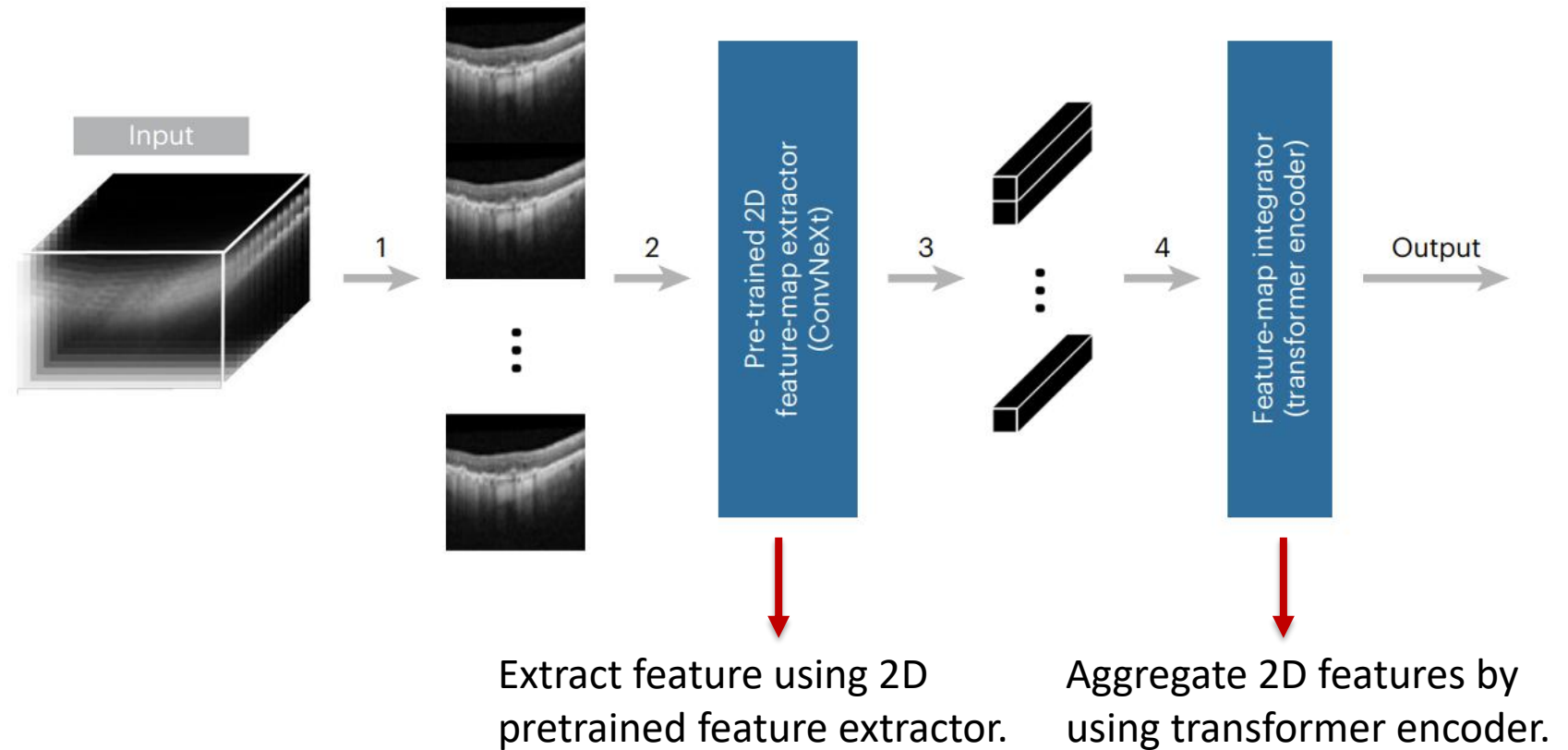
Accurate prediction of disease-risk factors from volumetric medical scans by a deep vision model pre-trained with 2D scans

Method

Pretraining:

Step 1: Initialize and trained with Image-Net pretrained encoder.

Step 2: Pretrain on 3D OCT data.

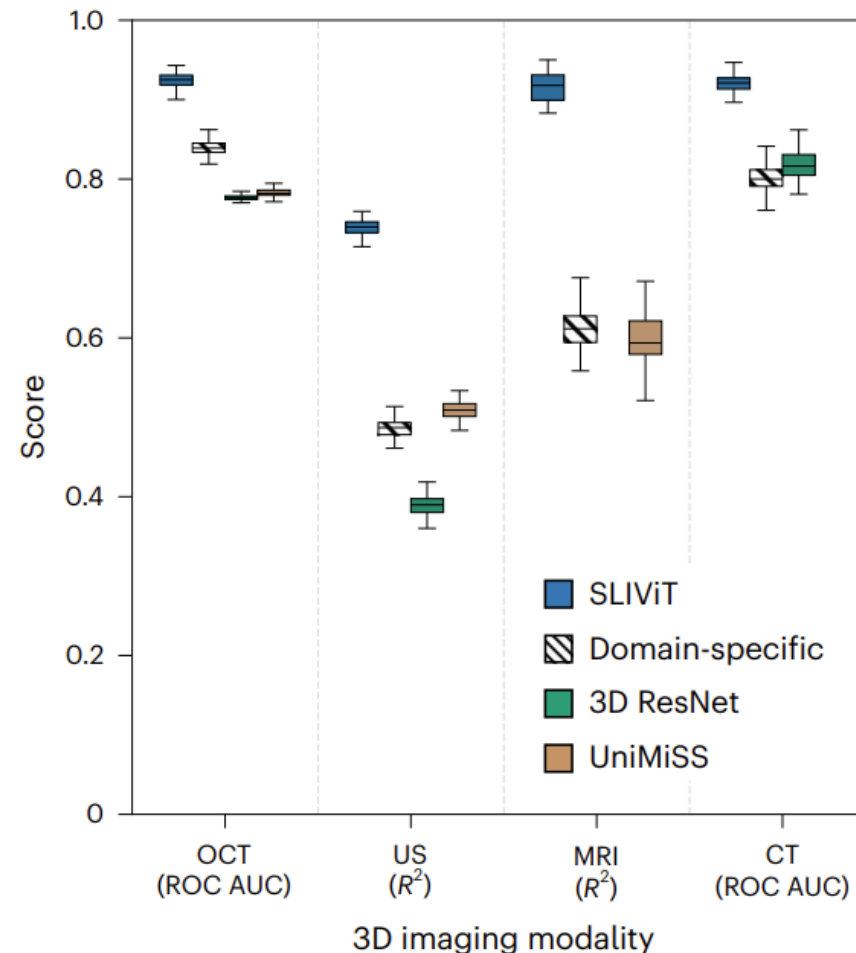


4. Deep learning for retinal image analysis



Accurate prediction of disease-risk factors from volumetric medical scans by a deep vision model pre-trained with 2D scans

Finding 1: Pretrained SLIViT have strong performance on OCT, Ultrasound, MRI, CT 3D classification and regression tasks.



4. Deep learning for retinal image analysis

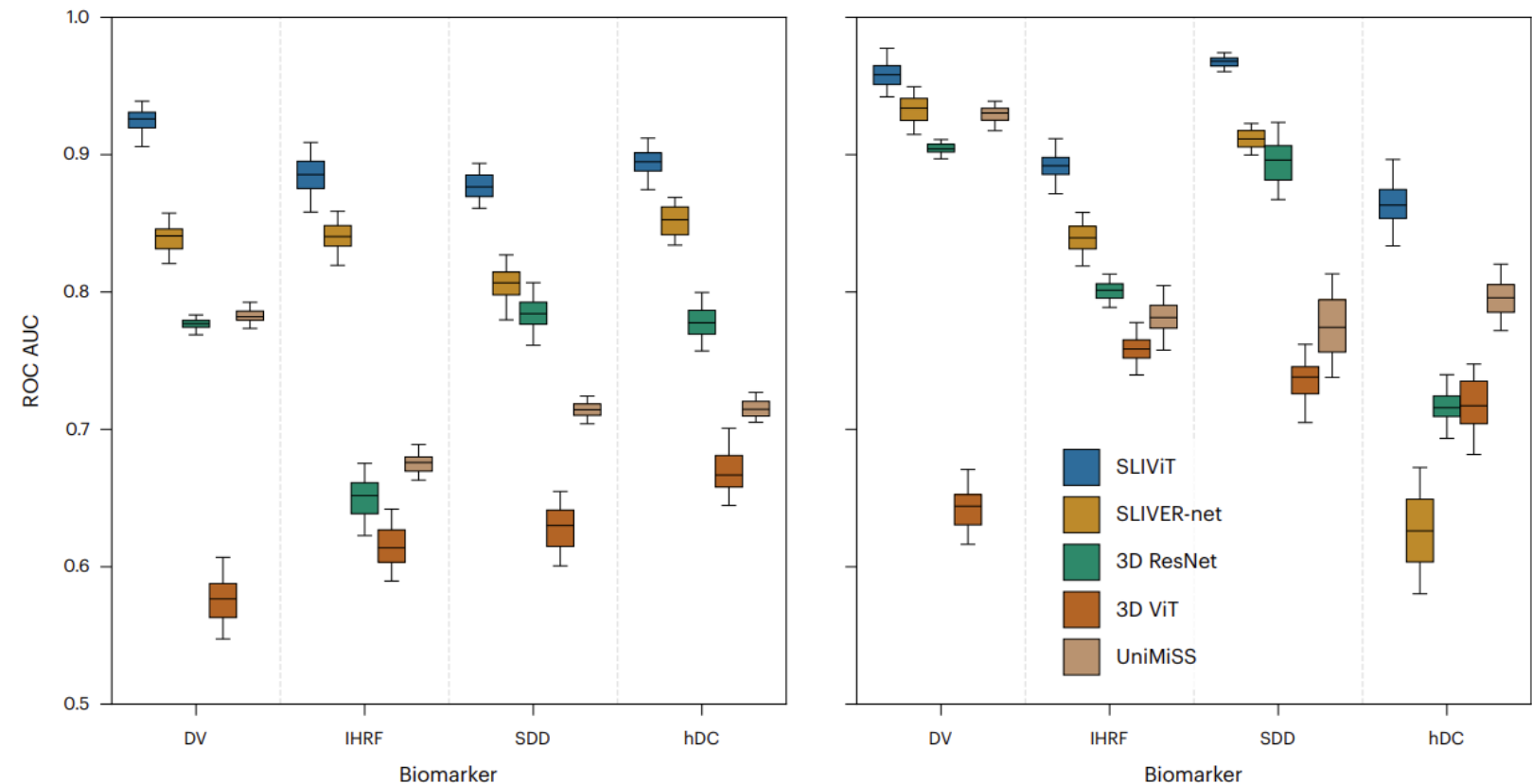


Accurate prediction of disease-risk factors from volumetric medical scans by a deep vision model pre-trained with 2D scans

Finding 2: SLIViT can accurately detect ocular disease high-risk factors when trained on less than 700 OCT volumes.

Key high-risk factors that can be detected in OCT:

- Drusen Volume
- Intraretinal hyperreflective foci
- Subretinal drusen deposits
- Hyporefective drusen cores

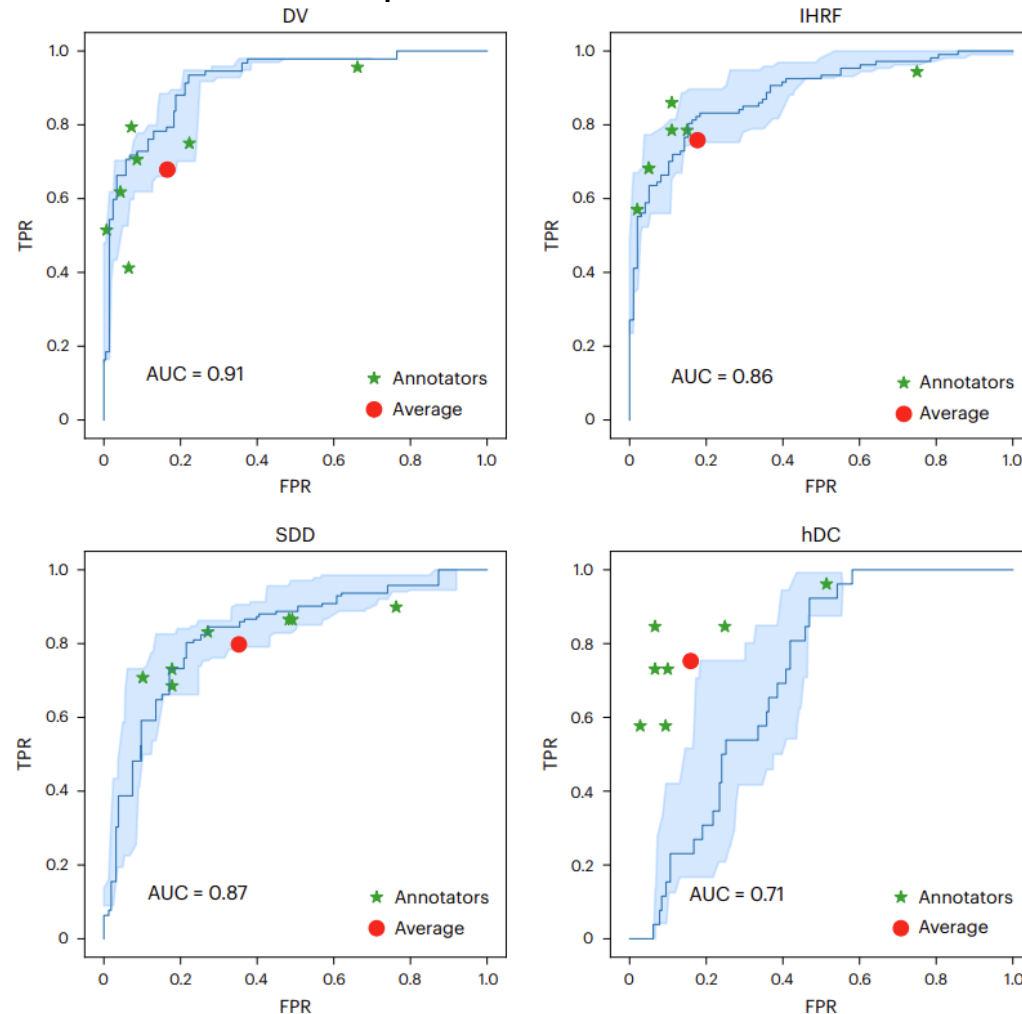


4. Deep learning for retinal image analysis



Accurate prediction of disease-risk factors from volumetric medical scans by a deep vision model pre-trained with 2D scans

Finding 3: The SLIViT can perform better when compared with clinicians.

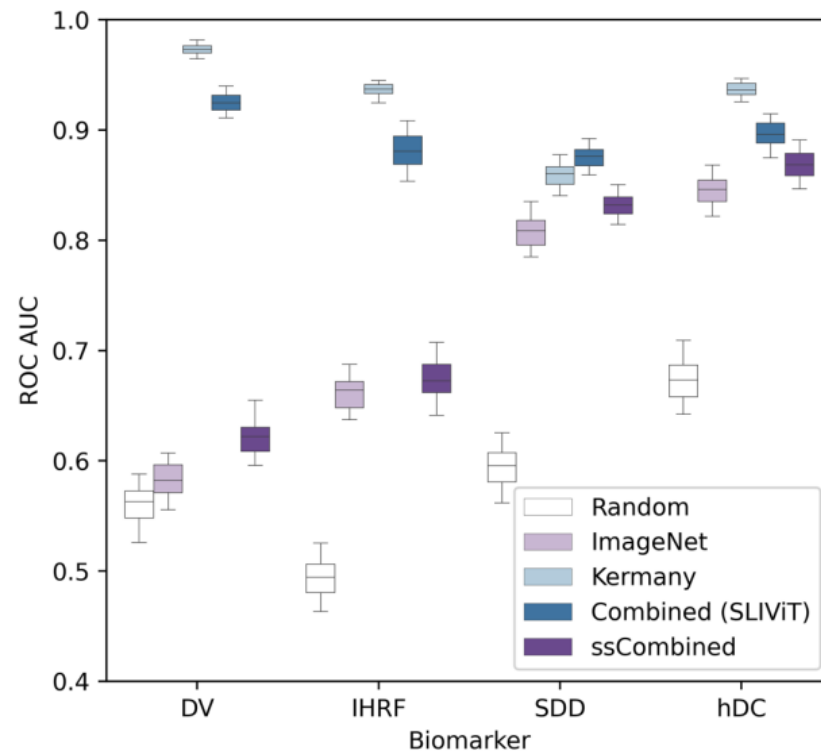


4. Deep learning for retinal image analysis

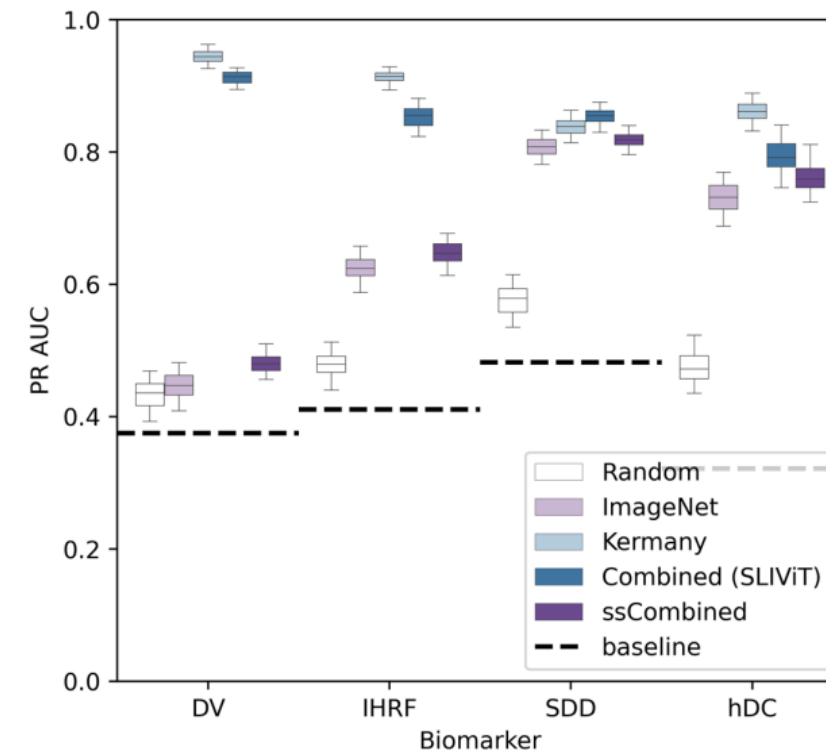


Accurate prediction of disease-risk factors from volumetric medical scans by a deep vision model pre-trained with 2D scans

Finding 4: In this scenario, two-stage supervised pretraining performs better than supervised combined pretraining.



Extended Data Fig. 4 | ImageNet and OCT B-scans pre-training contribution for OCT-related downstream learning tasks. Shown are the ROC (left) and PR (right) AUC scores across different fine-tuned models for volumetric-OCT classification tasks initialized with five different sets of weights. Combined, the proposed SLiViT's initialization, is ImageNet weights initialization followed by supervised pre-training on the Kermany Dataset. ssCombined is an ImageNet



weights initialization followed by self-supervised pre-training on an unlabeled version of the Kermany Dataset. The expected ROC AUC score of a random model is 0.5. The dashed lines represent the corresponding biomarker's positive-label prevalence, which is the expected PR AUC score of a random model. Box plot whiskers represent a 90% CI.



- Last time: 3D and videos
- Introduction to eye physiology
- Fundus images and OCT
- Deep learning for retinal image analysis